NEBULAOPS

v22.1 - Manuale Tecnico Enterprise

Edizione completa aggiornata da v21.4 con Keycloak SSO, OAuth2 Proxy, GitLab opzionale, WSL/Docker, CI/CD, GitOps, Observability e Troubleshooting operativo.

Angular 18 - Spring Boot 3.3 - Java 21 - Go 1.23 - Python 3.12 - Docker Compose - Keycloak - OAuth2 Proxy - MongoDB - RabbitMQ - Redis - Prometheus - Loki - Tempo - Grafana

Nota editoriale v22.1

Questa edizione estesa mantiene la struttura operativa del manuale NebulaOps v21.4 e la aggiorna alla release v22.1. Le sezioni storiche sono conservate per non perdere procedure, tabelle, controller, script e troubleshooting gia documentati; le pagine iniziali aggiungono il delta tecnico v22.1, la nuova matrice SSO e le procedure corrette di avvio.

L obiettivo e fornire un manuale completo, non un estratto: quick start, repository map, runtime architecture, gateway, script, compose, autenticazione, console UI, Docker Desktop, Kubernetes/OpenLens, observability, DevSecOps, CI/CD, GitOps, Terraform, tasks, RabbitMQ, AI Ops, demo Spring Boot, API reference, troubleshooting, hardening e appendici rimangono nel corpo del documento.

Nelle sezioni ereditate da v21.4, i riferimenti di versione sono stati normalizzati a v22.1. Quando una procedura e stata modificata nella release corrente, la versione aggiornata e riportata nelle pagine di delta v22.1 prima del corpo completo.

Criteri di aggiornamento

- Aggiornamento della versione visuale e del naming operativo da v21.4/v21-4 a v22.1/v22-1.
- Keycloak trattato come identity provider centrale per frontend, gateway, microservizi, Grafana e strumenti amministrativi via OAuth2 Proxy.
- GitLab CE mantenuto opzionale tramite profilo Compose per evitare overhead nello sviluppo quotidiano.
- RabbitMQ, Mongo Express e Redis Commander protetti con OAuth2 Proxy solo quando si usa --with-sso-proxy.
- Health check e smoke test allineati agli endpoint autenticati: quando Keycloak e attivo, una chiamata anonima che riceve 401/403 non e piu un errore di piattaforma.

Indice esteso v22.1

0A. Delta tecnico v22.1 e modalita operative

La release v22.1 consolida l autenticazione centralizzata e introduce una separazione netta tra stack base e stack SSO esteso. Il comando normale avvia una piattaforma piu leggera e stabile; il profilo SSO aggiunge OAuth2 Proxy e bridge Nginx per proteggere le UI amministrative che non supportano direttamente OIDC.

Comando	Componenti aggiunti	Uso consigliato	Effetto su RabbitMQ/Mongo/Redis
./scripts/wsl/start.sh	Stack base NebulaOps, Keycloak, Grafana OIDC, gateway e microservizi protetti	Sviluppo quotidiano, demo leggere, troubleshooting rapido	UI native o basic proxy; niente OAuth2 Proxy sui tool
./scripts/wsl/start.sh --with-sso-proxy	Stack base + OAuth2 Proxy + Nginx bridge per RabbitMQ/Mongo/Redis	Demo security/SSO completa e validazione auth centralizzata	Accesso browser rediretto a Keycloak prima della UI
./scripts/wsl/start.sh --with-gitlab	Stack base + GitLab CE	Solo quando serve CI/CD GitLab locale completo	GitLab usa OIDC Keycloak ma resta opzionale per peso runtime

Regola operativa

```
cd /mnt/d/workspace/personal/NebulaOps/nebulaops-v22.1
./scripts/wsl/stop.sh
./scripts/wsl/start.sh

# oppure, per SSO completo sui tool:
./scripts/wsl/stop.sh
./scripts/wsl/start.sh --with-sso-proxy
```

Il cambio tra modalita base e modalita SSO deve sempre partire da uno stop pulito. In caso contrario, container proxy o porte pubblicate da un avvio precedente possono rimanere attivi e generare falsi 502 o redirect non coerenti.

0B. Quick start aggiornato WSL/Docker

Il percorso primario per Windows 11 + WSL2 resta scripts/wsl/start.sh. La release v22.1 aggiunge la preparazione automatica del tema Keycloak, la gestione della rete esterna nebulaops-network, la verifica del datasource Grafana e il supporto ai profili GitLab/SSO.

Avvio base

```
cd /mnt/d/workspace/personal/NebulaOps/nebulaops-v22.1
./scripts/wsl/check-wsl.sh
./scripts/wsl/start.sh
./scripts/wsl/health.sh
```

Avvio con SSO tools

```
cd /mnt/d/workspace/personal/NebulaOps/nebulaops-v22.1
./scripts/wsl/stop.sh
./scripts/wsl/start.sh --with-sso-proxy
./scripts/wsl/health.sh
./scripts/wsl/diagnose-sso.sh
```

Endpoint principali

Risorsa	URL	Autenticazione v22.1
Frontend	http://localhost:4200	Keycloak OIDC/PKCE
Gateway	http://localhost:8080	JWT Resource Server + Bearer token
Keycloak	http://localhost:8180	admin/admin, realm nebulaops
Grafana	http://localhost:3000	Generic OAuth verso Keycloak
Prometheus	http://localhost:9090	Endpoint health pubblicato; UI separabile da SSO se abilitato
RabbitMQ Management	http://localhost:15672	Nativo nel base, Keycloak via OAuth2 Proxy con --with-sso-proxy
Mongo Express	http://localhost:8088	Nativo/basic nel base, Keycloak via OAuth2 Proxy con --with-sso-proxy
Redis Commander	http://localhost:8089	Nativo/basic nel base, Keycloak via OAuth2 Proxy con --with-sso-proxy
GitLab	http://localhost:8929	Disabilitato di default; profilo gitlab

0C. Matrice SSO: nativo, JWT, OAuth2 Proxy

Non tutti i componenti parlano lo stesso linguaggio di autenticazione. La configurazione v22.1 usa il meccanismo piu adatto al componente: OIDC nativo dove disponibile, JWT resource server per API applicative, OAuth2 Proxy per UI legacy o tool amministrativi.

Componente	Pattern auth	Client Keycloak	Note operative
Frontend Angular	OIDC Authorization Code + PKCE	nebulaops-frontend	Il browser ottiene token e chiama il gateway con Bearer.
Gateway Spring	JWT Resource Server	nebulaops-gateway / nebulaops-api	Valida issuer/JWKS e propaga Authorization ai microservizi.
Servizi Spring	JWT Resource Server	nebulaops-api	Actuator health resta raggiungibile; API business richiedono token.
FastAPI AI Engine	JWT/JWKS middleware	nebulaops-api	Verifica token prima delle route applicative.
Go services	JWT/JWKS middleware	nebulaops-api	Health pubblico, API protette.
Grafana	Generic OAuth	grafana	Login Keycloak nativo, mapping ruoli su org/admin.
RabbitMQ UI	OAuth2 Proxy + Nginx bridge	nebulaops-oauth2-proxy	OAuth2 Proxy autentica, bridge soddisfa basic auth legacy verso RabbitMQ.
Mongo Express	OAuth2 Proxy + Nginx bridge	nebulaops-oauth2-proxy	OAuth2 Proxy davanti, Nginx aggiunge basic auth interna.
Redis Commander	OAuth2 Proxy + Nginx bridge	nebulaops-oauth2-proxy	OAuth2 Proxy davanti, Nginx aggiunge basic auth interna.
GitLab	OIDC omniauth	gitlab	Solo profilo gitlab; non parte nello stack base.

Flusso OAuth2 Proxy per UI legacy

```
Browser -> http://localhost:15672
OAuth2 Proxy -> redirect Keycloak /realms/nebulaops
Keycloak -> login -> callback /oauth2/callback
OAuth2 Proxy -> session cookie
OAuth2 Proxy -> Nginx bridge interno
Nginx bridge -> rabbitmq:15672 con basic auth interna
```

0D. Catalogo servizi v22.1 e profili Compose

La tabella seguente e generata dal docker-compose.yml della release v22.1 corrente. Il profilo default corrisponde all avvio normale; i profili gitlab, sso-proxy e sso-prometheus si attivano solo quando richiesti dagli script.

Servizio	Porte host	Profilo	Immagine/build	Nota v22.1
keycloak-db	-	default	postgres:16-alpine	
keycloak	8180:8080	default	quay.io/keycloak/keycloak:24.0.4	
mongodb	27017:27017	default	mongo:7	
mongo-express	-	default	mongo-express:1.0.2-20	UI nativa con override oppure dietro SSO
redis-commander	-	default	rediscommander/redis-commander:latest	UI nativa con override oppure dietro SSO
auth-service	8081:8081	default	nebulaops-v22-1-auth-service:latest	
task-service	8082:8082	default	nebulaops-v22-1-task-service:latest	
notification-service	8083:8083	default	nebulaops-v22-1-notification-service:latest	
file-service	8084:8084	default	nebulaops-v22-1-file-service:latest	
ai-engine	8095:8095	default	nebulaops-v22-1-ai-engine:latest	
ai-ops-service	8085:8085	default	nebulaops-v22-1-ai-ops-service:latest	
devsecops-service	8086:8086	default	nebulaops-v22-1-devsecops-service:latest	
pipeline-engine-service	8087:8087	default	nebulaops-v22-1-pipeline-engine-service:latest	
observability-service	8092:8092	default	nebulaops-v22-1-observability-service:latest	

Servizio	Porte host	Profilo	Immagine/build	Nota v22.1
gitops-control-service	8093:8093	default	nebulaops-v22-1-gitops-control-service:latest	
environment-manager-service	8094:8094	default	nebulaops-v22-1-environment-manager-service:latest	
terraform-studio-service	8096:8096	default	nebulaops-v22-1-terraform-studio-service:latest	
gateway-service	8080:8080	default	nebulaops-v22-1-gateway-service:latest	
frontend	4200:80	default	nebulaops-v22-1-frontend:latest	
prometheus	-	default	prom/prometheus:v2.53.0	UI nativa con override oppure dietro SSO
loki	3100:3100	default	grafana/loki:2.9.10	
tempo	3200:3200, 4317:4317	default	grafana/tempo:2.5.0	
otel-collector	4318:4318	default	otel/opentelemetry-collector-contrib:0.104.0	
grafana	3000:3000	default	grafana/grafana:11.1.0	
redis	6379:6379	default	redis:7-alpine	
rabbitmq	5672:5672	default	rabbitmq:3.13-management-alpine	UI nativa con override oppure dietro SSO
go-cache-service	8091:8091	default	nebulaops-v22-1-go-cache-service:latest	
go-event-worker	-	default	nebulaops-v22-1-go-event-worker:latest	

Servizio	Porte host	Profilo	Immagine/build	Nota v22.1
gitlab	8929:8929, 2222:22	gitlab	gitlab/gitlab-ce:17.11.3-ce.0	Opzionale; profilo gitlab
cost-analytics-service	8097:8097	default	nebulaops-v22-1-cost-analytics-service:latest	
rabbitmq-basic-proxy	-	sso-proxy	${NEBULAOPS_SSO_NGINX_IMAGE:-nginx:1.27-alpine}	Layer SSO Keycloak/OAuth2
mongo-express-basic-proxy	-	sso-proxy	${NEBULAOPS_SSO_NGINX_IMAGE:-nginx:1.27-alpine}	Layer SSO Keycloak/OAuth2
redis-commander-basic-proxy	-	sso-proxy	${NEBULAOPS_SSO_NGINX_IMAGE:-nginx:1.27-alpine}	Layer SSO Keycloak/OAuth2
rabbitmq-management-sso	15672:4180	sso-proxy	${OAUTH2_PROXY_IMAGE:-quay.io/oauth2-proxy/oauth2-proxy:v7.6.0}	Layer SSO Keycloak/OAuth2
prometheus-sso	9090:4180	sso-prometheus	${OAUTH2_PROXY_IMAGE:-quay.io/oauth2-proxy/oauth2-proxy:v7.6.0}	Layer SSO Keycloak/OAuth2
mongo-express-sso	8088:4180	sso-proxy	${OAUTH2_PROXY_IMAGE:-quay.io/oauth2-proxy/oauth2-proxy:v7.6.0}	Layer SSO Keycloak/OAuth2
redis-commander-sso	8089:4180	sso-proxy	${OAUTH2_PROXY_IMAGE:-quay.io/oauth2-proxy/oauth2-proxy:v7.6.0}	Layer SSO Keycloak/OAuth2

0E. Script operativi v22.1

Rispetto alla v21.4, gli script WSL sono il contratto operativo principale. La tabella indica i comandi da usare in routine e quelli da usare solo per diagnostica o reset controllato.

Script	Esiste	Uso operativo v22.1
scripts/wsl/start.sh	si	Avvio primario WSL. Supporta stack base, --with-sso-proxy per RabbitMQ/Mongo/Redis dietro Keycloak e --with-gitlab per GitLab CE opzionale.
scripts/wsl/stop.sh	si	Stop robusto del progetto Compose, inclusi container opzionali e orfani.
scripts/wsl/health.sh	si	Health check Keycloak-aware: token admin, actuator, gateway /api, Prometheus, Grafana, Loki e SSO redirect per tool UI.
scripts/wsl/smoke-test.sh	si	Smoke test API con Bearer token Keycloak verso gateway e microservizi.
scripts/wsl/diagnose-sso.sh	si	Diagnostica per OAuth2 Proxy, Nginx bridge, DNS Docker e log dei container SSO.
scripts/wsl/start-sso-tools.sh	si	Alias operativo per avvio con --with-sso-proxy.
scripts/keycloak-ensure-sso-clients.sh	si	Crea o aggiorna i client Keycloak necessari senza reset distruttivo del volume.
scripts/keycloak-sso-reset.sh	si	Reset controllato del realm/theme per reimportare client SSO e login custom.
scripts/oauth2-proxy-preflight.sh	si	Verifica DNS e immagini oauth2-proxy/nginx prima dell avvio SSO.
scripts/wsl/docker-network-fix.sh	no	Ripara la rete nebulaops-network quando preesiste senza label Compose corretta.
scripts/wsl/docker-cache-repair.sh	si	Pulisce cache BuildKit/immagini in caso di snapshot corrotti.
scripts/test-all.sh	si	Validazioni statiche e test opportunistici su YAML, Bash, Go, frontend e Maven se presenti.

Sequenza di validazione consigliata

```
./scripts/wsl/start.sh --with-sso-proxy
./scripts/wsl/health.sh
./scripts/wsl/diagnose-sso.sh
./scripts/wsl/smoke-test.sh
```

health.sh deve mostrare Keycloak token obtained, Gateway /api verde, Prometheus verde e SSO redirect OK per RabbitMQ, MongoDB UI e Redis UI. diagnose-sso.sh entra nel dettaglio di DNS Docker, bridge Nginx e upstream reali.

0F. Keycloak realm e client applicativi

La release v22.1 importa o assicura i client Keycloak necessari all avvio. Quando un volume Keycloak esiste gia, il realm non viene reimportato automaticamente: per questo sono presenti keycloak-ensure-sso-clients.sh e keycloak-sso-reset.sh.

Client	Protocollo	Public	Redirect URI principali
nebulaops-frontend	-	True	http://localhost:4200/*, http://localhost:4201/*
nebulaops-gateway	-	False	
gitlab	-	False	http://localhost:8929/users/auth/openid_connect/callback
grafana	-	False	http://localhost:3000/login/generic_oauth
nebulaops-oauth2-proxy	-	False	http://localhost:15672/oauth2/callback, http://localhost:8088/oauth2/callback, http://localhost:8089/oauth2/callback
nebulaops-api	-	False	

Errore Client not found

Se Grafana, frontend o OAuth2 Proxy mostrano Client not found, il browser sta arrivando a Keycloak ma il client non e presente nel realm attivo. In genere il volume Keycloak contiene una vecchia importazione.

```
./scripts/keycloak-ensure-sso-clients.sh
# se non basta:
./scripts/keycloak-sso-reset.sh
```

0G. OAuth2 Proxy per RabbitMQ, Mongo Express e Redis Commander

RabbitMQ Management, Mongo Express e Redis Commander non sono trattati come microservizi applicativi. Sono UI amministrative legacy. In v22.1 vengono protette con OAuth2 Proxy solo nel profilo --with-sso-proxy; in questo modo lo stack base resta leggero e non dipende dal download delle immagini oauth2-proxy/nginx quando non servono.

UI	Porta host	Container SSO	Bridge interno	Upstream reale	Health bridge
RabbitMQ	15672	rabbitmq-management-sso	rabbitmq-basic-proxy	rabbitmq:15672	/__nebulaops_proxy_healt
Mongo Express	8088	mongo-express-sso	mongo-express-basic-proxy	mongo-express:8081	/__nebulaops_proxy_healt
Redis Commander	8089	redis-commander-sso	redis-commander-basic-proxy	redis-commander:8081	/__nebulaops_proxy_healt

Perche esiste il bridge Nginx

OAuth2 Proxy autentica l utente con Keycloak, ma le UI native possono richiedere ancora basic auth o header specifici. Il bridge Nginx e interno alla rete Docker, non pubblico, e aggiunge gli header necessari verso l upstream. Se il bridge non risolve il nome del servizio o l upstream non e healthy, il browser vede 502 Bad Gateway dopo il login.

Diagnostica 502

```
./scripts/wsl/diagnose-sso.sh
docker compose ps | grep -E "rabbit|mongo|redis|sso|proxy"
docker logs nebulaops-v22-1-rabbitmq-basic-proxy-1 --tail=80
docker logs nebulaops-v22-1-mongo-express-basic-proxy-1 --tail=80
docker logs nebulaops-v22-1-redis-commander-basic-proxy-1 --tail=80
```

0H. Troubleshooting decisionale v22.1

Sintomo	Causa probabile	Primo comando	Esito atteso
Client not found in Keycloak	Realm vecchio o client non creato	./scripts/keycloak-ensure-sso-clients.sh	Client grafana/oauth2/frontend presenti
502 dopo login SSO su RabbitMQ/Mongo/Redis	Bridge Nginx o upstream interno non healthy	./scripts/wsl/diagnose-sso.sh	DNS e upstream OK
oauth2-proxy image non scaricabile	DNS WSL/Docker o quay.io non raggiungibile	docker pull quay.io/oauth2-proxy/oauth2-proxy:v7.6.0	Pull completato oppure usare OAUTH2_PROXY_IMAGE alternativo
Prometheus rosso in health	Porta non pubblicata o profilo errato	docker compose ps prometheus	9090 pubblicato quando richiesto
GitLab rosso	GitLab disabilitato di default	./scripts/wsl/start.sh --with-gitlab	Container gitlab avviato solo se serve
Gateway /api 401/403	Smoke test anonimo su API protetta	./scripts/wsl/smoke-test.sh	Token Keycloak ottenuto e API verdi
network label incorrect	Rete esterna preesistente senza label Compose	./scripts/wsl/docker-network-fix.sh	nebulaops-network pronta
COPY target/*.jar app.jar fallisce	Dockerfile non multi-stage o target escluso	docker compose build --no-cache <service>	JAR creato nel builder stage

Comandi di recovery rapidi

```
./scripts/wsl/stop.sh
./scripts/wsl/docker-network-fix.sh
./scripts/keycloak-sso-reset.sh
./scripts/wsl/start.sh --with-sso-proxy
./scripts/wsl/health.sh
```

0I. Validazione e criteri di rilascio v22.1

La qualita di rilascio non si misura solo con docker compose up. La piattaforma e considerata pronta quando sono verdi le verifiche statiche, l avvio runtime, l autenticazione Keycloak, gli endpoint applicativi e gli strumenti amministrativi nella modalita scelta.

Area	Comando	Criterio OK
YAML e package	python3 scripts/validate-yaml.py docker-compose.yml .gitlab-ci.yml && python3 scripts/validate-package.py	Nessun errore sintattico o file mancante critico
Bash	find scripts -name "*.sh" -print0 | xargs -0 -I{} bash -n {}	Tutti gli script parsabili
Go	(cd go/cache-service && go test ./...) && (cd go/event-worker && go test ./...)	Test passati
Frontend	cd frontend && npm install && npm run build	Build Angular completata
Backend	mvn -q -f backend/pom.xml test	Test Spring/Maven passati
Runtime base	./scripts/wsl/start.sh && ./scripts/wsl/health.sh	Stack base verde
Runtime SSO	./scripts/wsl/start.sh --with-sso-proxy && ./scripts/wsl/diagnose-sso.sh	SSO redirect e bridge upstream OK
Smoke API	./scripts/wsl/smoke-test.sh	Token Keycloak e chiamate gateway OK

0J. Migrazione da v21.4 a v22.1

La migrazione consigliata e una sostituzione pulita della directory applicativa mantenendo separati i backup dei volumi dati. I volumi Keycloak possono conservare realm vecchi: in caso di incoerenze SSO, usare il reset dedicato e non modificare manualmente il database.

```
cd /mnt/d/workspace/personal/NebulaOps
./nebulaops-v21.4/scripts/wsl/stop.sh || true
mv nebulaops-v21.4 nebulaops-v21.4.backup.$(date +%Y%m%d-%H%M%S)
unzip nebulaops-v22.1-sso-bridge-healthfix.zip
cd nebulaops-v22.1
./scripts/wsl/start.sh
./scripts/wsl/health.sh
```

Migrazione con SSO tools

```
./scripts/wsl/stop.sh
./scripts/keycloak-sso-reset.sh
./scripts/wsl/start.sh --with-sso-proxy
./scripts/wsl/diagnose-sso.sh
```

Se si riutilizzano volumi di una versione precedente, la causa piu frequente di problemi e un realm Keycloak non allineato. Il reset SSO reimporta client e tema e rende coerenti frontend, Grafana, OAuth2 Proxy e GitLab opzionale.

0K. Estratti di configurazione v22.1

Nginx bridge health endpoint

```
events {}
http {
resolver 127.0.0.11 valid=10s ipv6=off;
resolver_timeout 5s;

server {
listen 8080;

location = /__nebulaops_proxy_health {
access_log off;
add_header Content-Type text/plain;
return 200 'rabbitmq-basic-proxy ok\n';
}

location / {
set $upstream_rabbitmq http://rabbitmq:15672;
proxy_pass $upstream_rabbitmq;
proxy_http_version 1.1;
proxy_connect_timeout 10s;
proxy_send_timeout 60s;
proxy_read_timeout 60s;
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header X-Forwarded-Host $host;
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection "upgrade";
proxy_redirect off;
# Satisfy the legacy RabbitMQ basic-auth challenge after Keycloak SSO.
proxy_set_header Authorization "Basic Z3Vlc3Q6Z3Vlc3Q=";
proxy_hide_header WWW-Authenticate;
proxy_buffering off;
}
}
}
```

Preflight OAuth2 Proxy

```
#!/usr/bin/env bash
# v22.1 — Check/pull the images required by OAuth2 Proxy protected tool UIs.
set -euo pipefail

OAUTH2_IMAGE="${OAUTH2_PROXY_IMAGE:-quay.io/oauth2-proxy/oauth2-proxy:v7.6.0}"
NGINX_IMAGE="${NEBULAOPS_SSO_NGINX_IMAGE:-nginx:1.27-alpine}"

pull_or_ok() {
local image="$1"
if docker image inspect "$image" >/dev/null 2>&1; then
echo "OK image already present: $image"
return 0
fi

echo "Pulling image: $image"
docker pull "$image"
}

if pull_or_ok "$OAUTH2_IMAGE" && pull_or_ok "$NGINX_IMAGE"; then
echo "OK OAuth2 Proxy prerequisites are ready"
exit 0
fi

cat >&2 <<TIP

[errore] Impossibile scaricare una delle immagini SSO.

Immagini richieste:
- $OAUTH2_IMAGE
- $NGINX_IMAGE

Nel tuo WSL/Docker l'errore più probabile è DNS verso quay.io.
Puoi provare una registry alternativa per OAuth2 Proxy:

export OAUTH2_PROXY_IMAGE=docker.io/bitnami/oauth2-proxy:7.6.0
```

```
./scripts/wsl/start.sh --with-sso-proxy

Oppure pre-scarica manualmente le immagini e rilancia lo start.
TIP
exit 1
```

0L. Checklist finale operativa

Check	Stato atteso
Versione visuale e naming	v22.1 in frontend, script, compose, immagini e manuale
Keycloak	Login custom HTTP 200; realm nebulaops pronto; client grafana/frontend/api/oauth2 presenti
Gateway	/actuator/health verde; /api/health verde con Bearer token
Microservizi	Actuator health verde e API protette da JWT
Grafana	Login Keycloak e /api/health verde
RabbitMQ/Mongo/Redis base	UI raggiungibili senza layer SSO esteso
RabbitMQ/Mongo/Redis SSO	Redirect Keycloak e upstream interno OK con --with-sso-proxy
Prometheus/Loki/Tempo	Health verdi e datasource Grafana coerente
GitLab	Disabilitato di default; attivabile con --with-gitlab
Smoke test	Tasks, K8s snapshot, Docker containers, Helm releases, Environments verdi

Il corpo seguente conserva il manuale tecnico completo, con le sezioni estese della v21.4 aggiornate e integrate nel contesto v22.1. Le informazioni operative nuove sono da considerare prevalenti quando differiscono da procedure storiche.

Indice ragionato

L indice e organizzato per uso operativo: ogni capitolo contiene scopo, file sorgente, endpoint, comandi, esito atteso e problemi tipici. Le schede UI sono collegate alle API e ai componenti backend che alimentano i dati mostrati nella console.

0. Perimetro tecnico e metodo di verifica

viene descritto come piattaforma DevOps cloud-native locale composta da frontend Angular, gateway Spring Boot, microservizi applicativi, data services, stack di osservabilita, integrazione Docker, funzioni Kubernetes/OpenLens-like, CI/CD, GitOps, Terraform e demo applicativa Spring Boot.

Il contenuto tecnico e collegato a evidenze verificabili nel repository: file sorgente, classi Java, script shell, manifest YAML, Docker Compose, chart Helm, asset grafici, endpoint REST e procedure CLI. Le sezioni con maturita parziale sono marcate come foundation o backlog tecnico per evitare ambiguita operative.

Ambito analizzato

- docker-compose.yml root e configurazione runtime dei servizi .
- config/platform.yml, frontend/src/app/api.config.ts, app.component.html e app.component.ts.
- Controller Spring Boot in backend/*/src/main/java e servizi Python/Go collegati.
- Script WSL/Linux/Terraform in scripts/ e configurazioni sotto infrastructure/.
- Chart Helm, manifest Kubernetes, ArgoCD project/application e stack observability.
- Pacchetto demo nebulaops-spring-demo- con Docker, Kubernetes, Helm, GitLab CI/CD e ArgoCD.

Criteri di tracciabilita

- Ogni feature importante e associata a un file, una classe, uno script, un endpoint o un manifesto verificabile.
- Ogni procedura operativa include comando, risultato atteso e possibili errori.
- Ogni scheda UI indica origine dati, chiamate backend, azioni disponibili e limiti di maturita.
- Le API sono estratte dai controller Java presenti in backend/*/src/main/java.
- Le incongruenze di versione e i componenti foundation sono trattati come backlog tecnico, non come funzioni pienamente consolidate.

Stato del pacchetto

Il pacchetto root e il percorso WSL sono il riferimento principale per la . Alcuni file secondari conservano naming legacy o versioni storiche; tali elementi sono documentati nella sezione hardening/backlog e non incidono necessariamente sul percorso di avvio consigliato.

Check	Risultato	Impatto operativo
python3 scripts/validate-yaml.py	OK su .gitlab-ci.yml, docker-compose.yml, ArgoCD project/application/applicationset	YAML sintatticamente valido per CI e GitOps
python3 scripts/validate-package.py	README da aggiornare: GitLab, Argo CD e riferimenti ai diagrammi runtime/service-port	Gap documentale/SEO, non failure runtime
grep version mismatch	config/platform.yml v21.3, backend POM 21.3.0, alcuni script Linux v17, infrastructure/docker-compose.yml v20-2	Usare root docker-compose.yml e scripts/wsl/start.sh come percorso primario

1. Quick Start WSL2 e Docker

Il percorso consigliato per su Windows 11 e WSL2 passa dallo script scripts/wsl/start.sh, non da infrastructure/docker-compose.yml. Il root docker-compose.yml contiene Keycloak, porte e immagini ; il compose sotto infrastructure conserva nomi v20-2 ed e da trattare come legacy/Helm support artifact.

1.1 Comandi essenziali

```
cd /mnt/d/workspace/personal/portfolio/nebulaops-
./scripts/wsl/check-wsl.sh
./scripts/wsl/start.sh
./scripts/wsl/health.sh
./scripts/wsl/logs.sh gateway-service --lines 120
./scripts/wsl/stop.sh
```

1.2 Cosa deve succedere al primo avvio

- Lo script crea o riusa la rete Docker esterna nebulaops-network.
- Copia il kubeconfig host in .kube/config e riscrive server localhost verso IP WSL.
- Copia docker/kubectl/helm/trivy/terraform/argocd in .runtime-tools, cosi i container possono eseguire tool reali.
- Rimuove template.ftl custom Keycloak potenzialmente rotto e genera un login.ftl standalone NebulaOps.
- Verifica che Grafana abbia esattamente un datasource con isDefault: true.
- Avvia tutti i servizi compose in modalita detached e poi prova Keycloak OIDC e gateway /api/health.

1.3 Credenziali e URL locali

Risorsa	URL	Credenziali/Note
Frontend	http://localhost:4200	Login via Keycloak OIDC oppure auth-service dev admin in alcuni flussi
Gateway	http://localhost:8080/actuator/health	Health Spring Boot
Keycloak	http://localhost:8180	admin/admin; realm nebulaops
Grafana	http://localhost:3000	admin/admin
Prometheus	http://localhost:9090	/-/healthy
RabbitMQ	http://localhost:15672	guest/guest
Mongo Express	http://localhost:8088	admin/admin
Redis Commander	http://localhost:8089	Dipende da redis service

2. Repository map e ownership tecnica

La mappa seguente e basata sul contenuto reale del pacchetto. Serve per capire dove intervenire quando una sezione UI mostra dati non aggiornati, quando una rotta 502 cade nel gateway o quando una pipeline non sincronizza ArgoCD.

Area	Path	Responsabilita reale
Frontend Angular	frontend/src/app/app.component.ts/html/css, api.config.ts	Single page cockpit, tab routing, API client, Keycloak PKCE, UI Docker/K8s actions
Gateway	backend/gateway-service	BFF, proxy microservizi, Docker socket, kubectl/helm/terraform toolchain, API live platform
Auth	backend/auth-service	JWT dev auth e user account MongoDB; health ancora version 21.3
Task domain	backend/task-service + notification-service	CRUD MongoDB, eventi RabbitMQ, notifiche
Runtime tools	scripts/wsl/prepare-runtime-tools.sh + .runtime-tools	Binari host copiati e montati nei container backend
Kubernetes manifests	infrastructure/kubernetes	Deploy NebulaOps su cluster, namespace, configmap, deployments, services, ingress
Helm chart	infrastructure/helm/nebulaops	Packaging Kubernetes della piattaforma
GitOps	infrastructure/argocd	ArgoCD Project, Application, ApplicationSet
Observability	infrastructure/observability	Prometheus, Grafana dashboards, OTel collector, Tempo
Terraform	terraform/ e infrastructure/terraform	Esempi IaC; attenzione a path differenziati nei vecchi script
Demo app	nebulaops-spring-demo-	Applicazione Spring Boot con Docker, K8s, Helm, GitLab CI/CD, ArgoCD

2.1 File primari verificati

File	Esiste	Uso nel manuale
README.md	si	fonte primaria
START_HERE.md	si	fonte primaria
PROJECT_METADATA.md	si	fonte primaria
RELEASE_NOTES_ md	si	fonte primaria
KEYCLOAK_CUSTOM_LOGIN_ md	si	fonte primaria
CLEAN_PACKAGE_ md	si	fonte primaria
docker-compose.yml	si	fonte primaria
config/platform.yml	si	fonte primaria
frontend/src/app/api.config.ts	si	fonte primaria
frontend/src/app/app.component.ts	si	fonte primaria
frontend/src/app/app.component.html	si	fonte primaria
backend/gateway-service/src/main/resources/application.yml	si	fonte primaria
backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/ProxyController.java	si	fonte primaria
backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java	si	fonte primaria
backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/DockerActionsController.java	si	fonte primaria
backend/gateway-service/src/main/java/dev/nebulaops/gateway/client/DockerSocketClient.java	si	fonte primaria
backend/gateway-service/src/main/java/dev/nebulaops/gateway/client/ToolCommandClient.java	si	fonte primaria
infrastructure/keycloak/realm-nebulaops.json	si	fonte primaria
.gitlab-ci.yml	si	fonte primaria
infrastructure/argocd/application.yaml	si	fonte primaria
infrastructure/helm/nebulaops/values.yaml	si	fonte primaria
infrastructure/observability/prometheus/prometheus.yml	si	fonte primaria

3. Architettura runtime e service catalog

e una piattaforma local-first che gira principalmente in Docker Compose ma prepara anche un percorso Kubernetes/Helm/GitOps. Il frontend Angular e servito da nginx, chiama /api sul gateway, e il gateway inoltra a microservizi interni o esegue direttamente Docker/kubectl/helm/terraform quando serve accesso runtime.

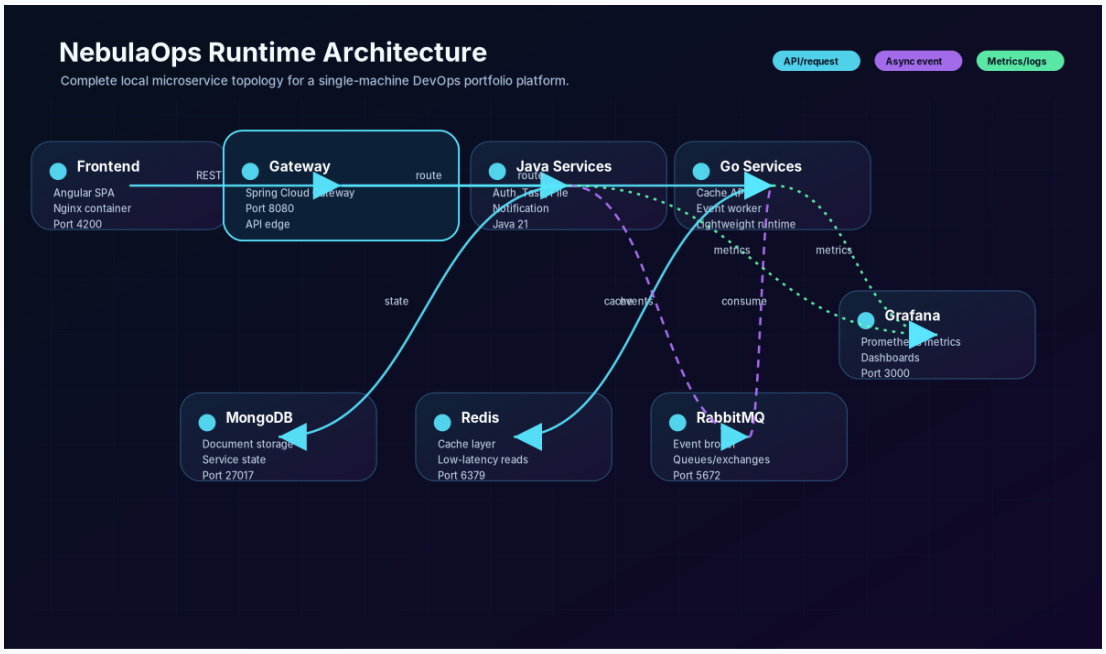

Figura 1 - docs/diagrams/runtime-architecture.svg: separa frontend, gateway BFF, microservizi, data services e stack observability; utile per capire dove cercare un 502.

3.1 Catalogo servizi da docker-compose.yml

Servizio	Porte host	Immagine	Dockerfile/Origine
keycloak-db	-	postgres:16-alpine	image esterna
keycloak	8180:8080	quay.io/keycloak/keycloak:24.0.4	image esterna
mongodb	27017:27017	mongo:7	image esterna
mongo-express	8088:8081	mongo-express:1.0.2-20	image esterna
redis-commander	8089:8081	rediscommander/redis-commander:latest	image esterna
auth-service	8081:8081	·auth-service:latest	backend/auth-service/Dockerfile
task-service	8082:8082	·task-service:latest	backend/task-service/Dockerfile
notification-service	8083:8083	·notification-service:latest	backend/notification-service/Dockerfile
file-service	8084:8084	·file-service:latest	backend/file-service/Dockerfile
ai-engine	8095:8095	·ai-engine:latest	ai-engine/Dockerfile
ai-ops-service	8085:8085	·ai-ops-service:latest	backend/ai-ops-service/Dockerfile
devsecops-service	8086:8086	·devsecops-service:latest	backend/devsecops-service/Dockerfile
pipeline-engine-service	8087:8087	·pipeline-engine-service:latest	backend/pipeline-engine-service/Dockerfile
observability-service	8092:8092	·observability-service:latest	backend/observability-service/Dockerfile
gitops-control-service	8093:8093	·gitops-control-service:latest	backend/gitops-control-service/Dockerfile
environment-manager-service	8094:8094	·environment-manager-service:latest	backend/environment-manager-service/Dockerfile
terraform-studio-service	8096:8096	·terraform-studio-service:latest	backend/terraform-studio-service/Dockerfile
gateway-service	8080:8080	·gateway-service:latest	backend/gateway-service/Dockerfile
frontend	4200:80	·frontend:latest	Dockerfile
prometheus	9090:9090	prom/prometheus:v2.53.0	image esterna
loki	3100:3100	grafana/loki:2.9.10	image esterna
tempo	3200:3200, 4317:4317	grafana/tempo:2.5.0	image esterna
otel-collector	4318:4318	otel/opentelemetry-collector-contrib:0.104.0	image esterna
grafana	3000:3000	grafana/grafana:11.1.0	image esterna
redis	6379:6379	redis:7-alpine	image esterna
rabbitmq	5672:5672, 15672:15672	rabbitmq:3.13-management-alpine	image esterna
go-cache-service	8091:8091	·go-cache-service:latest	go/cache-service/Dockerfile
go-event-worker	-	·go-event-worker:latest	go/event-worker/Dockerfile

3.2 Service port map reale

La mappa porte non e un elenco decorativo: e il contratto operativo per aprire console, scrivere smoke test e distinguere un problema di frontend da un problema di microservizio.

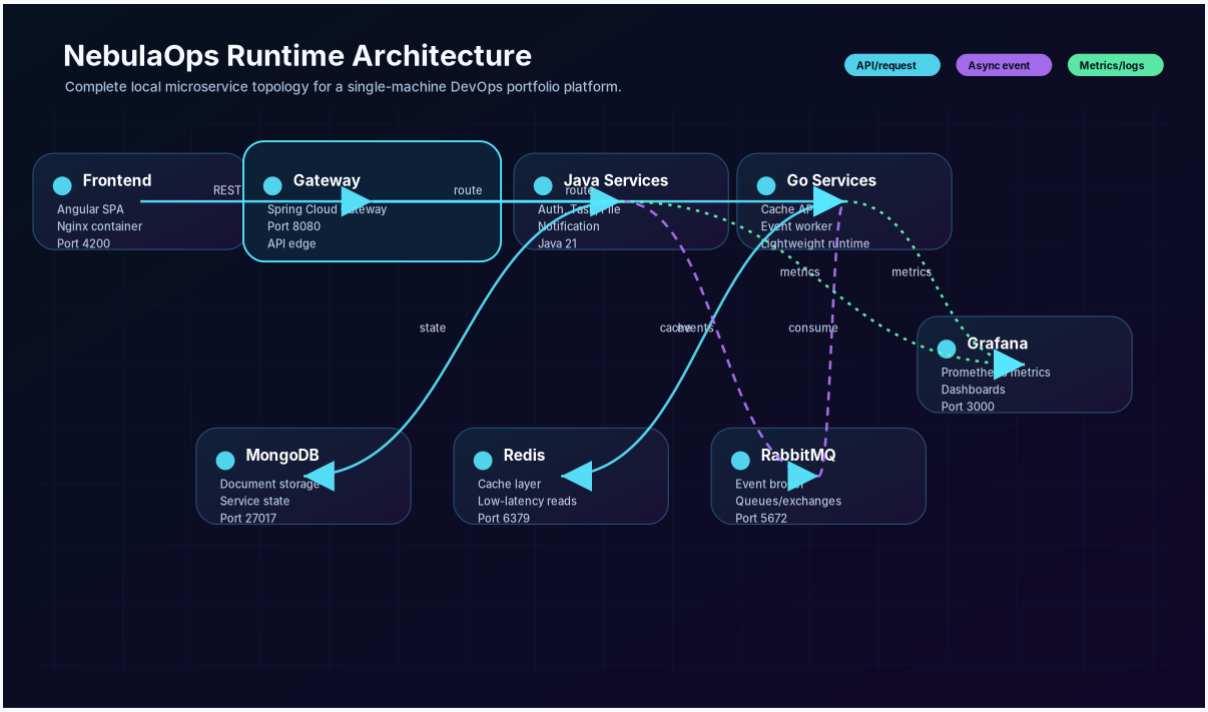

Figura 2 - docs/diagrams/service-port-map.svg: mapping tra host port, service name e ruolo; confrontarlo con docker-compose.yml quando una porta risulta gia occupata.

4. Gateway, proxy e live toolchain

Il gateway-service e il punto di convergenza. Non fa solo proxy HTTP: legge Docker Engine via Unix socket, esegue kubectl/helm tramite ToolCommandClient, aggrega osservabilita e rende stabile il frontend anche quando cluster o strumenti non sono disponibili.

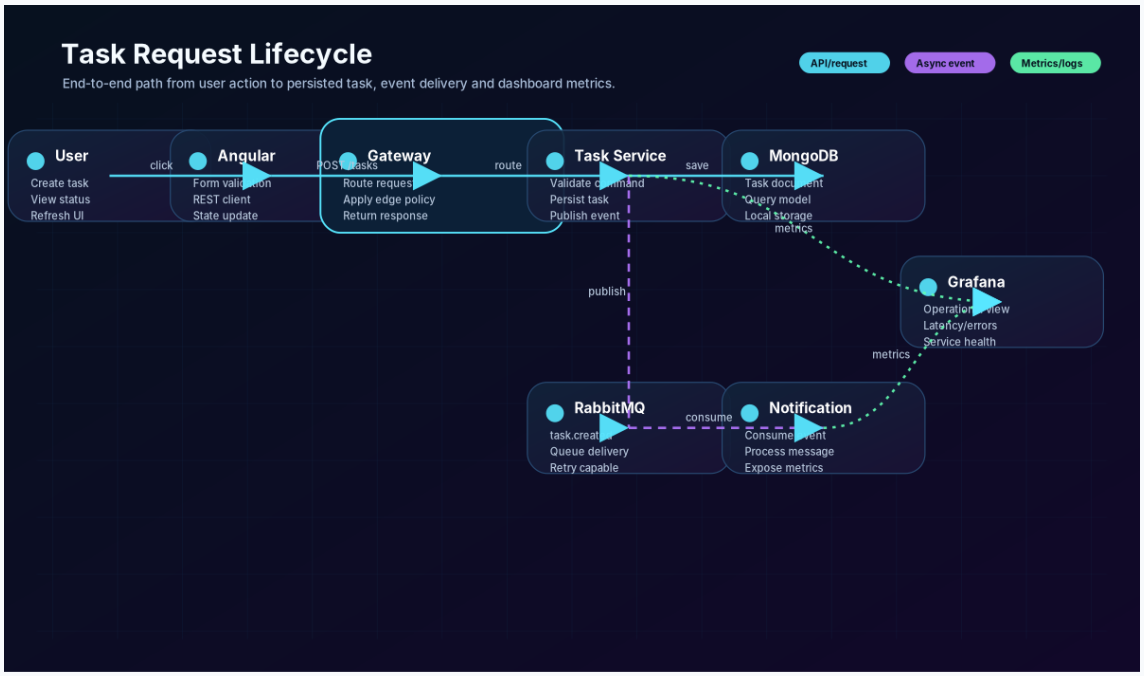

Figura 3 - docs/diagrams/request-flow-sequence.svg: sequenza browser -> nginx -> gateway -> microservizio/tool. Mostra perche gli errori vanno letti prima nei log gateway e poi nel servizio target.

4.1 File chiave del gateway

File	Responsabilita	Errore tipico associato
ProxyController.java	Proxy /api/auth, /api/tasks, /api/pipeline, /api/cost, /api/secrets, /api/registry	502/connection refused se URL del service in application.yml e errato
DockerActionsController.java	Azioni mutate Docker: start, stop, restart, prune, logs, stats	403/permission denied o socket non montato
KubernetesOpsController.java	Snapshot, resources e azioni kubectl OpenLens-like	kubectl not found o kubeconfig localhost non raggiungibile dal container
RuntimeOpsController.java	Read endpoints Docker/Helm/Terraform runtime	liste vuote se tool non disponibile
PlatformLiveController.java	Aggregazioni osservabilita/devsecops/env/terraform	dati partial se Prometheus/Loki/Trivy non rispondono
ToolCommandClient.java	shell wrapper con timeout	comandi lenti/timeout e stderr da riportare alla UI
DockerSocketClient.java	HTTP raw su /var/run/docker.sock	parse JSON fallito o Docker daemon non raggiungibile

5. Script operativi v22.1 - cosa fanno e quando usarli

Questa sezione sostituisce l elenco generico degli script con schede operative. Ogni script indica side effect, prerequisiti e primo comando di debug. Il path e riportato esattamente come nel pacchetto.

5.1 scripts/argocd-apply-local.sh

Crea namespace argocd se manca e applica project/application ArgoCD dal repository.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
kubectl get namespace argocd >/dev/null 2>&1 || kubectl create namespace argocd
kubectl apply -f infrastructure/argocd/project.yaml
kubectl apply -f infrastructure/argocd/application.yaml
```

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT_DIR"

kubectl get namespace argocd >/dev/null 2>&1 || kubectl create namespace argocd
kubectl apply -f infrastructure/argocd/project.yaml
kubectl apply -f infrastructure/argocd/application.yaml

echo "Argo CD Application applied. Check status with: kubectl -n argocd get applications"
```

5.2 scripts/backup-local.sh

Crea snapshot leggero in backups/<timestamp> con Terraform/Grafana e docker compose ps. Non sostituisce un backup dati completo.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
docker compose -f "$ROOT/docker-compose.yml" ps > "$OUT/docker-compose-ps.txt" 2>&1 || true
```

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
STAMP="$(date +%Y%m%d-%H%M%S)"
OUT="$ROOT/backups/$STAMP"
mkdir -p "$OUT"
cp -a "$ROOT/infrastructure/terraform" "$OUT/terraform" 2>/dev/null || true
cp -a "$ROOT/infrastructure/observability/grafana" "$OUT/grafana" 2>/dev/null || true
if command -v docker >/dev/null 2>&1; then
  docker compose -f "$ROOT/docker-compose.yml" ps > "$OUT/docker-compose-ps.txt" 2>&1 || true
fi
ln -sfn "$OUT" "$ROOT/backups/latest"
echo "Backup created: $OUT"
```

5.3 scripts/gitlab-validate.sh

Combina validate-package e validate-yaml su GitLab/ArgoCD; utile prima di pushare pipeline.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
python3 scripts/validate-package.py
python3 scripts/validate-yaml.py .gitlab-ci.yml infrastructure/argocd/application.yaml
  infrastructure/argocd/project.yaml infrastructure/argocd/applicationset.yaml
```

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT_DIR"

command -v python3 >/dev/null || { echo "python3 is required"; exit 1; }
python3 scripts/validate-package.py
python3 scripts/validate-yaml.py .gitlab-ci.yml infrastructure/argocd/application.yaml
  infrastructure/argocd/project.yaml infrastructure/argocd/applicationset.yaml

echo "GitLab and Argo CD files are syntactically valid."
```

5.4 scripts/helm-install-local.sh

helm upgrade --install nebulaops su namespace nebulaops e stampa port-forward utili per frontend/gateway/Grafana/Prometheus.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
helm upgrade --install nebulaops "$ROOT_DIR/infrastructure/helm/nebulaops" \
kubectl -n nebulaops get pods
kubectl -n nebulaops port-forward svc/nebulaops-frontend 4200:4200
kubectl -n nebulaops port-forward svc/nebulaops-gateway 8080:8080
kubectl -n nebulaops port-forward svc/nebulaops-grafana 3000:3000
kubectl -n nebulaops port-forward svc/nebulaops-prometheus 9090:9090
```

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
helm upgrade --install nebulaops "$ROOT_DIR/infrastructure/helm/nebulaops" \
  --namespace nebulaops \
  --create-namespace \
  --set global.imagePullPolicy=IfNotPresent
kubectl -n nebulaops get pods
cat <<'INFO'

Useful port-forwards:
  kubectl -n nebulaops port-forward svc/nebulaops-frontend 4200:4200
  kubectl -n nebulaops port-forward svc/nebulaops-gateway 8080:8080
  kubectl -n nebulaops port-forward svc/nebulaops-grafana 3000:3000
  kubectl -n nebulaops port-forward svc/nebulaops-prometheus 9090:9090

Grafana login: admin / admin
INFO
```

5.5 scripts/helm-render.sh

Renderizza il chart NebulaOps in infrastructure/helm/rendered-nebulaops.yaml per review prima di installare.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
helm template nebulaops "$ROOT_DIR/infrastructure/helm/nebulaops" --namespace nebulaops >
  "$ROOT_DIR/infrastructure/helm/rendered-nebulaops.yaml"
```

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
helm template nebulaops "$ROOT_DIR/infrastructure/helm/nebulaops" --namespace nebulaops >
  "$ROOT_DIR/infrastructure/helm/rendered-nebulaops.yaml"
echo "Rendered Helm manifests: infrastructure/helm/rendered-nebulaops.yaml"
```

5.6 scripts/linux/create-kind-cluster.sh

Crea cluster kind usando infrastructure/kind/cluster.yaml, ma default cluster name è legacy nebulaops-v17.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
kind get clusters | grep -qx "$CLUSTER" || kind create cluster --name "$CLUSTER" --config
  infrastructure/kind/cluster.yaml
kubectl config use-context "kind-${CLUSTER}"
kubectl create namespace nebulaops --dry-run=client -o yaml | kubectl apply -f -
kubectl cluster-info
```

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
CLUSTER=${1:-nebulaops-v17}
kind get clusters | grep -qx "$CLUSTER" || kind create cluster --name "$CLUSTER" --config
  infrastructure/kind/cluster.yaml
kubectl config use-context "kind-${CLUSTER}"
kubectl create namespace nebulaops --dry-run=client -o yaml | kubectl apply -f -
kubectl cluster-info
```

5.7 scripts/linux/helm-install-nebulaops.sh

Script presente nel pacchetto. Verificare path, project name e side effect prima di usarlo in demo: alcuni helper sono legacy o generici.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
helm dependency update infrastructure/helm/nebulaops || true
helm upgrade --install nebulaops-v17 infrastructure/helm/nebulaops -n "$NAMESPACE" --create-
  namespace --values infrastructure/helm/nebulaops/values.yaml
kubectl -n "$NAMESPACE" get pods,svc,ingress
```

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
NAMESPACE=${1:-nebulaops}
helm dependency update infrastructure/helm/nebulaops || true
helm upgrade --install nebulaops-v17 infrastructure/helm/nebulaops -n "$NAMESPACE" --create-
  namespace --values infrastructure/helm/nebulaops/values.yaml
kubectl -n "$NAMESPACE" get pods,svc,ingress
```

5.8 scripts/linux/install-native-toolchain.sh

Installer Ubuntu/Debian per Docker Engine nativo, kubectl, Helm, kind, Java 21, Maven, Node/npm e tool base.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | $SUDO gpg --dearmor -o
  /etc/apt/keyrings/docker.gpg
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.31/deb/Release.key | $SUDO gpg --dearmor -o
  /etc/apt/keyrings/kubernetes-apt-keyring.gpg
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.24.0/kind-linux-amd64 && chmod +x ./kind && $SUDO
  mv ./kind /usr/local/bin/kind
docker version
docker compose version
kubectl version --client=true
helm version
kind version
```

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
if [[ $EUID -eq 0 ]]; then SUDO=""; else SUDO="sudo"; fi
. /etc/os-release
if [[ "${ID}" != "ubuntu" && "${ID_LIKE:-}" != *debian* ]]; then echo "This installer targets
  Ubuntu/Debian based WSL/Linux."; exit 1; fi
$SUDO apt-get update
$SUDO apt-get install -y ca-certificates curl gnupg lsb-release apt-transport-https software-
  properties-common jq yq make git openjdk-21-jdk maven nodejs npm
$SUDO install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | $SUDO gpg --dearmor -o
  /etc/apt/keyrings/docker.gpg
$SUDO chmod a+r /etc/apt/keyrings/docker.gpg
ARCH=$(dpkg --print-architecture)
CODENAME=$(. /etc/os-release && echo "${VERSION_CODENAME:-noble}")
echo "deb [arch=${ARCH} signed-by=/etc/apt/keyrings/docker.gpg]
  https://download.docker.com/linux/ubuntu ${CODENAME} stable" | $SUDO tee
  /etc/apt/sources.list.d/docker.list >/dev/null
$SUDO apt-get update
$SUDO apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
  compose-plugin
$SUDO usermod -aG docker "$USER" || true
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.31/deb/Release.key | $SUDO gpg --dearmor -o
  /etc/apt/keyrings/kubernetes-apt-keyring.gpg
$SUDO chmod 644 /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
  https://pkgs.k8s.io/core:/stable:/v1.31/deb/ /' | $SUDO tee
  /etc/apt/sources.list.d/kubernetes.list >/dev/null
$SUDO apt-get update && $SUDO apt-get install -y kubectl
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.24.0/kind-linux-amd64 && chmod +x ./kind && $SUDO
  mv ./kind /usr/local/bin/kind
$SUDO systemctl enable --now docker 2>/dev/null || $SUDO service docker start
newgrp docker <<'EONG' || true
docker version
docker compose version
kubectl version --client=true
helm version
kind version
EONG
echo "Native Docker Engine, Docker Compose plugin, kubectl, Helm and kind installed. Reopen the
  shell if docker permission is not active yet."
```

5.9 scripts/linux/start-native.sh

Start Linux nativo con COMPOSE_PROJECT_NAME legacy nebulaops-v17. Va adattato a per evitare naming inconsistente.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
docker compose up -d --build
```

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
export COMPOSE_PROJECT_NAME=${COMPOSE_PROJECT_NAME:-nebulaops-v17}
mkdir -p .kube
if [[ -f "$HOME/.kube/config" ]]; then cp "$HOME/.kube/config" .kube/config; chmod 600
  .kube/config; fi
docker compose up -d --build
./scripts/smoke-test.sh || true
echo "Frontend: http://localhost:4200 | Gateway: http://localhost:8080 | Grafana:
  http://localhost:3000 admin/admin"
```

5.10 scripts/linux/start.sh

Script presente nel pacchetto. Verificare path, project name e side effect prima di usarlo in demo: alcuni helper sono legacy o generici.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Non sono stati rilevati comandi shell esterni nelle prime scansioni: leggere il sorgente prima di inserirlo in una pipeline.

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
exec "$SCRIPT_DIR/start-native.sh" "$@"
```

5.11 scripts/local-down.sh

Down del root compose con -v e --remove-orphans; rimuove volumi del progetto .

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
docker compose -p "$PROJECT_NAME" -f "$COMPOSE_FILE" down -v --remove-orphans
```

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
COMPOSE_FILE="$ROOT_DIR/docker-compose.yml"
PROJECT_NAME="                "
cd "$ROOT_DIR"
docker compose -p "$PROJECT_NAME" -f "$COMPOSE_FILE" down -v --remove-orphans
```

5.12 scripts/local-up.sh

Avvio generico dal root compose: crea nebulaops-network, prepara kubeconfig/tools se disponibili e lancia docker compose -p up --build.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
docker network create nebulaops-network
docker compose -p "$PROJECT_NAME" -f "$COMPOSE_FILE" up --build "$@"
```

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
COMPOSE_FILE="$ROOT_DIR/docker-compose.yml"
PROJECT_NAME="                "
export COMPOSE_PARALLEL_LIMIT=${COMPOSE_PARALLEL_LIMIT:-2}
cd "$ROOT_DIR"

# Ensure shared network exists (external: true in docker-compose requires it)
if ! docker network inspect nebulaops-network &>/dev/null; then
  echo "[     ] Creating shared network: nebulaops-network"
  docker network create nebulaops-network
else
  echo "[     ] Network nebulaops-network already exists — skipping"
fi

./scripts/wsl/prepare-kubeconfig-for-docker.sh 2>/dev/null || true
./scripts/wsl/prepare-runtime-tools.sh 2>/dev/null || true
docker compose -p "$PROJECT_NAME" -f "$COMPOSE_FILE" up --build "$@"
```

5.13 scripts/observability-open.sh

Stampa endpoint osservabilità e reminder Grafana admin/admin.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Non sono stati rilevati comandi shell esterni nelle prime scansioni: leggere il sorgente prima di inserirlo in una pipeline.

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
cat <<'INFO'
Observability endpoints after ./scripts/local-up.sh:
  Grafana:    http://localhost:3000  login admin/admin
  Prometheus: http://localhost:9090
  Gateway:    http://localhost:8080/actuator/health
  Task metrics endpoint: http://localhost:8082/actuator/prometheus

Open Grafana, go to Dashboards → NebulaOps → NebulaOps Platform Overview.
INFO
```

5.14 scripts/restore-local.sh

Helper non distruttivo: stampa indicazioni restore e richiede procedura manuale per MongoDB.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Non sono stati rilevati comandi shell esterni nelle prime scansioni: leggere il sorgente prima di inserirlo in una pipeline.

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
SNAPSHOT="${1:-backups/latest}"
echo "Restore helper for $SNAPSHOT"
echo "Review files before overwriting live state. For MongoDB restore use mongorestore according
  to your volume strategy."
```

5.15 scripts/smoke-test.sh

Smoke test gateway/task/RabbitMQ: health, register demo user, crea task, lista task e verifica go-cache-service.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
curl -fsS "$BASE/actuator/health" >/dev/null
curl -fsS -H 'Content-Type: application/json'   -d
  '{"email":"demo@nebulaops.dev","displayName":"Demo
  User","password":"Password123!","organizationId":"demo-org"}'   "$BASE/api/auth/register"
  >/tmp/nebulaops-register.json || true
curl -fsS "$BASE/api/tasks?organizationId=demo-org"
curl -fsS http://localhost:8091/health >/dev/null && echo "go-cache-service OK"
```

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
BASE=${BASE:-http://localhost:8080}
echo "Checking gateway health..."
curl -fsS "$BASE/actuator/health" >/dev/null
printf "Registering demo user... "
curl -fsS -H 'Content-Type: application/json'   -d
  '{"email":"demo@nebulaops.dev","displayName":"Demo
  User","password":"Password123!","organizationId":"demo-org"}'   "$BASE/api/auth/register"
  >/tmp/nebulaops-register.json || true
echo "OK"
echo "Creating task through gateway..."
TASK_RESPONSE=$(curl -fsS -H 'Content-Type: application/json'   -d '{"organizationId":"demo-
  org","projectId":"portfolio","title":"Smoke test RabbitMQ event","description":"Created by
  smoke-test.sh","priority":"HIGH","labels":["smoke","rabbitmq","mongodb"]}'
  "$BASE/api/tasks")
echo "$TASK_RESPONSE"
echo "Listing tasks..."
curl -fsS "$BASE/api/tasks?organizationId=demo-org"
echo
echo "Smoke test completed. Check notifications with: curl $BASE/api/notifications"


echo "Checking Go cache service"
curl -fsS http://localhost:8091/health >/dev/null && echo "go-cache-service OK"
```

5.16 scripts/terraform/apply-local.sh

Come plan-local ma con apply -auto-approve. Da usare solo su ambiente locale controllato e dopo aver verificato il path tfvars.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
terraform init
terraform fmt -recursive
terraform validate
terraform apply -auto-approve -var-file examples/local-kind/terraform.tfvars
```

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/../.." && pwd)"
cd "$ROOT/terraform"
terraform init
terraform fmt -recursive
terraform validate
terraform apply -auto-approve -var-file examples/local-kind/terraform.tfvars
```

5.17 scripts/terraform/plan-local.sh

Esegue terraform init/fmt/validate/plan in terraform/ con examples/local-kind/terraform.tfvars. Nel pacchetto il tfvars atteso è sotto infrastructure/terraform, quindi va verificato.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
terraform init
terraform fmt -recursive
terraform validate
terraform plan -var-file examples/local-kind/terraform.tfvars
```

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/../.." && pwd)"
cd "$ROOT/terraform"
terraform init
terraform fmt -recursive
terraform validate
terraform plan -var-file examples/local-kind/terraform.tfvars
```

5.18 scripts/test-all.sh

Validazione statica e build: validate-package, validate-yaml, bash -n, Go tests, frontend npm build e Maven tests se tool presenti.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
python3 scripts/validate-package.py
python3 scripts/validate-yaml.py
npm run build --silent
mvn -q -f backend/pom.xml test
```

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT_DIR"

python3 scripts/validate-package.py
python3 scripts/validate-yaml.py
find scripts -name "*.sh" -print0 | xargs -0 -I{} bash -n {}

if command -v go >/dev/null 2>&1; then
  (cd go/cache-service && go test ./...)
  (cd go/event-worker && go test ./...)
else
  echo "go not found: skipping Go tests"
fi

if command -v npm >/dev/null 2>&1; then
  (
    cd frontend
    if [ ! -d node_modules ]; then
      if [ -f package-lock.json ]; then npm ci --silent; else npm install --silent; fi
    fi
    npm run build --silent
  )
else
  echo "npm not found: skipping frontend build"
fi

if command -v mvn >/dev/null 2>&1; then
  mvn -q -f backend/pom.xml test
else
  echo "mvn not found: skipping Java Maven tests"
fi

echo "NebulaOps validation completed"
```

5.19 scripts/validate-package.py

Controllo qualità del pacchetto. Nella consegna corrente fallisce su README: mancano GitLab/Argo CD e riferimenti a diagrammi nonostante i file esistano.

Campo	Valore
Tipo	Python/utility
Shebang	#!/usr/bin/env python3
Uso consigliato	supporto/diagnostica/legacy

Non sono stati rilevati comandi shell esterni nelle prime scansioni: leggere il sorgente prima di inserirlo in una pipeline.

Estratto sorgente

```
#!/usr/bin/env python3
from pathlib import Path
import json
import sys
import xml.etree.ElementTree as ET

ROOT = Path(__file__).resolve().parents[1]
errors = []

def check_file(path: str) -> None:
    if not (ROOT / path).exists():
        errors.append(f"missing: {path}")

required_files = [
    "README.md",
    "docs/architecture-animated.svg",
    "docs/diagrams/runtime-architecture.svg",
    "docs/diagrams/gitlab-argocd-flow.svg",
    "docs/diagrams/messaging-cache-flow.svg",
    "docs/diagrams/kubernetes-helm-view.svg",
    "docs/diagrams/request-flow-sequence.svg",
    "docs/diagrams/service-port-map.svg",
    "docs/TECHNICAL_DOCUMENTATION.md",
    "docs/GITLAB_ARGOCD.md",
    "docs/TROUBLESHOOTING.md",
    "infrastructure/docker-compose.yml",
    "infrastructure/helm/nebulaops/Chart.yaml",
    "infrastructure/observability/prometheus/prometheus.yml",
    "infrastructure/observability/grafana/dashboards/nebulaops-overview.json",
    "infrastructure/argocd/project.yaml",
    "infrastructure/argocd/application.yaml",
    "infrastructure/argocd/applicationset.yaml",
    ".gitlab-ci.yml",
    "frontend/package.json",
    "backend/pom.xml",
```

5.20 scripts/validate-yaml.py

Parser PyYAML sui file passati. Utile in CI per .gitlab-ci.yml, compose, ArgoCD e Helm-rendered manifests.

Campo	Valore
Tipo	Python/utility
Shebang	#!/usr/bin/env python3
Uso consigliato	supporto/diagnostica/legacy

Non sono stati rilevati comandi shell esterni nelle prime scansioni: leggere il sorgente prima di inserirlo in una pipeline.

Estratto sorgente

```
#!/usr/bin/env python3
import sys
from pathlib import Path
try:
    import yaml
except ModuleNotFoundError:
    print('PyYAML is required for YAML validation: pip install pyyaml')
    sys.exit(1)
for arg in sys.argv[1:]:
    path = Path(arg)
    with path.open('r', encoding='utf-8') as f:
        list(yaml.safe_load_all(f))
    print(f'YAML OK: {path}')
```

5.21 scripts/verify-local.sh

Script presente nel pacchetto. Verificare path, project name e side effect prima di usarlo in demo: alcuni helper sono legacy o generici.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
docker compose version >/dev/null 2>&1 && echo "OK docker compose" || { echo "MISSING docker
  compose plugin"; missing=1; }
python3 scripts/validate-package.py
```

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT_DIR"
echo "NebulaOps local verification"
missing=0
for c in docker curl; do
  if command -v "$c" >/dev/null 2>&1; then echo "OK $c"; else echo "MISSING $c"; missing=1; fi
done
for c in node npm jq helm kubectl; do
  if command -v "$c" >/dev/null 2>&1; then echo "OK $c"; else echo "OPTIONAL missing $c"; fi
done
docker compose version >/dev/null 2>&1 && echo "OK docker compose" || { echo "MISSING docker
  compose plugin"; missing=1; }
[ -f frontend/package.json ] && echo "OK Angular frontend present" || missing=1
[ -f infrastructure/docker-compose.yml ] && echo "OK Docker Compose present" || missing=1
[ -f infrastructure/helm/nebulaops/Chart.yaml ] && echo "OK Helm chart present" || missing=1
[ -f docs/architecture-animated.svg ] && echo "OK architecture SVG present" || missing=1
python3 scripts/validate-package.py
if [[ "$missing" == "1" ]]; then
  echo "Some required tools/files are missing. See messages above."
  exit 1
fi
echo "Local static verification completed. For WSL use: ./scripts/wsl/check-wsl.sh &&
  ./scripts/wsl/start.sh"
```

5.22 scripts/wsl/check-wsl.sh

Pre-flight WSL: verifica docker, docker compose, kubectl, helm, kind e socket /var/run/docker.sock. Nota: messaggio testuale ancora v17, ma le verifiche sono utili per .

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
docker info >/dev/null 2>&1 && ok "Native Docker daemon reachable" || fail "Docker daemon is not
  reachable. Start it with: sudo service docker start"
docker compose version >/dev/null 2>&1 && ok "docker compose plugin found" || fail "docker
  compose plugin missing"
```

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
ok(){ echo "OK $1"; }
warn(){ echo "WARN $1"; }
fail(){ echo "FAIL $1"; exit 1; }
echo "NebulaOps v17 native WSL pre-flight check"
grep -qi microsoft /proc/version && ok "Running inside WSL" || warn "Not detected as WSL"
command -v docker >/dev/null 2>&1 && ok "docker CLI found" || fail "docker CLI missing. Run
  ./scripts/wsl/install-native-toolchain.sh"
docker info >/dev/null 2>&1 && ok "Native Docker daemon reachable" || fail "Docker daemon is not
  reachable. Start it with: sudo service docker start"
docker compose version >/dev/null 2>&1 && ok "docker compose plugin found" || fail "docker
  compose plugin missing"
command -v kubectl >/dev/null 2>&1 && ok "kubectl found" || fail "kubectl missing"
command -v helm >/dev/null 2>&1 && ok "helm found" || fail "helm missing"
command -v kind >/dev/null 2>&1 && ok "kind found" || warn "kind missing; install native
  toolchain if Kubernetes local cluster is needed"
[[ -S /var/run/docker.sock ]] && ok "Docker socket exists" || fail "Docker socket missing"
```

5.23 scripts/wsl/diagnose-runtime.sh

Esegue command -v dentro gateway/devsecops/pipeline/gitops/environment/terraform/ai-ops per verificare Java e tool montati.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
docker exec "$cid" sh -lc 'echo PATH=$PATH; /opt/java/openjdk/bin/java -version 2>&1 | head -1;
  command -v kubectl || true; command -v docker || true; command -v helm || true; command -v
  trivy || true; command -v terraform || true; command -v argocd || true' || true
```

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
PROJECT_NAME=${COMPOSE_PROJECT_NAME:-                }
SERVICES=(gateway-service devsecops-service pipeline-engine-service gitops-control-service
  environment-manager-service terraform-studio-service ai-ops-service)
for svc in "${SERVICES[@]}"; do
  cid=$(docker compose -p "$PROJECT_NAME" ps -q "$svc" 2>/dev/null || true)
  echo "--- $svc ---"
  if [[ -z "$cid" ]]; then
    echo "container not found"
    continue
  fi
  docker exec "$cid" sh -lc 'echo PATH=$PATH; /opt/java/openjdk/bin/java -version 2>&1 | head
    -1; command -v kubectl || true; command -v docker || true; command -v helm || true; command
    -v trivy || true; command -v terraform || true; command -v argocd || true' || true
done
```

5.24 scripts/wsl/docker-cache-repair.sh

Ripara errori BuildKit snapshot eliminando builder cache, immagini inutilizzate e buildx cache; preserva volumi applicativi.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
docker compose -p "$PROJECT_NAME" -f "$COMPOSE_FILE" down --remove-orphans || true
docker builder prune -af || true
docker image prune -af || true
docker buildx prune -af || true
```

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/../.." && pwd)"
cd "$ROOT_DIR"
COMPOSE_FILE="$ROOT_DIR/infrastructure/docker-compose.yml"
PROJECT_NAME="nebulaops"
cat <<'INFO'
NebulaOps Docker cache repair
This repairs common Docker Desktop/WSL BuildKit snapshot errors such as:
  parent snapshot ... does not exist: not found
  failed to prepare extraction snapshot

It removes build cache and stopped containers. Application data volumes are preserved unless you
  run reset.sh separately.
INFO

docker compose -p "$PROJECT_NAME" -f "$COMPOSE_FILE" down --remove-orphans || true

echo "Pruning BuildKit builder cache..."
docker builder prune -af || true

echo "Pruning unused images/layers..."
docker image prune -af || true

echo "Pruning unused buildx cache..."
docker buildx prune -af || true

echo "Repair completed. Rebuild with:"
echo "  DOCKER_BUILDKIT=1 docker compose -p $PROJECT_NAME -f infrastructure/docker-compose.yml
  build --no-cache frontend"
echo "  docker compose -p $PROJECT_NAME -f infrastructure/docker-compose.yml up -d"
```

5.25 scripts/wsl/gateway-logs.sh

Focus diagnostico sul gateway: status, ultimi 100 log e curl verbose su /actuator/health e /api/health.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
curl -sv --max-time 5 http://localhost:8080/actuator/health 2>&1 | tail -15
curl -sv --max-time 5 http://localhost:8080/api/health 2>&1 | tail -15
```

Estratto sorgente

```
#!/usr/bin/env bash
#        — Show gateway-service startup logs and probe health.
set -uo pipefail
source "$(dirname "${BASH_SOURCE[0]}")/lib/common.sh"

log_step "Gateway container status"
dc ps gateway-service

log_step "Gateway logs (last 100 lines)"
dc logs --tail=100 gateway-service

log_step "Testing gateway health directly"
curl -sv --max-time 5 http://localhost:8080/actuator/health 2>&1 | tail -15
echo
curl -sv --max-time 5 http://localhost:8080/api/health 2>&1 | tail -15
```

5.26 scripts/wsl/health.sh

Health check completo: container status, endpoint Spring Actuator, AI Engine, Grafana, Prometheus, Loki, RabbitMQ, Keycloak e sample API live.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	primario

Non sono stati rilevati comandi shell esterni nelle prime scansioni: leggere il sorgente prima di inserirlo in una pipeline.

Estratto sorgente

```
#!/usr/bin/env bash
#       — Comprehensive platform health check.
set -uo pipefail
source "$(dirname "${BASH_SOURCE[0]}")/lib/common.sh"

log_step "Container status"
dc ps --format "table {{.Service}}\t{{.Status}}\t{{.Ports}}" 2>/dev/null \
  | head -40 || log_warn "Compose not available"

log_step "Endpoint health"


check_keycloak_login() {
  local label="Keycloak custom login"
  local tmp="/tmp/nebulaops-keycloak-auth-health.html"
  local challenge="xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
  local url="http://localhost:8180/realms/nebulaops/protocol/openid-
    connect/auth?client_id=nebulaops-
    frontend&redirect_uri=http%3A%2F%2Flocalhost%3A4200&response_type=code&scope=openid%20profile%20email&state=healthcheck&code_challenge=
  local code
  code=$(curl -sS --max-time 5 -o "$tmp" -w "%{http_code}" "$url" 2>/dev/null || true)
  if [ "$code" = "200" ] && grep -qiE "kc-form-login|NebulaOps v21\.4|name=\"username\"" "$tmp"
    2>/dev/null; then
    printf "  ${C_GREEN}●${C_RESET} %-22s %s\n" "$label" "HTTP 200 login page"
  else
    printf "  ${C_RED}●${C_RESET} %-22s %s\n" "$label" "HTTP ${code:-000}"
    if [ "$code" = "500" ]; then
      printf "    ${C_YELLOW}↳${C_RESET} Custom theme failed. Run: docker compose -p
        $PROJECT_NAME logs keycloak --tail=160 | grep -iE 'freemarker|template|ERROR'\n"
    fi
  fi
}

check_endpoint() {
  local label="$1" url="$2"
  if curl -fsS --max-time 3 "$url" >/dev/null 2>&1; then
    printf "  ${C_GREEN}●${C_RESET} %-22s %s\n" "$label" "$url"
  else
    printf "  ${C_RED}●${C_RESET} %-22s %s\n" "$label" "$url"
```

5.27 scripts/wsl/install-kubectl.sh

Script presente nel pacchetto. Verificare path, project name e side effect prima di usarlo in demo: alcuni helper sono legacy o generici.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
curl \
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key \
kubectl version --client
curl -sfL https://get.k3s.io | sh -
kubectl get nodes
kubectl create namespace nebulaops
kubectl get ns
kubectl get nodes -o wide
```

Estratto sorgente

```
#!/usr/bin/env bash

set -euo pipefail

GREEN='\033[0;32m'
RED='\033[0;31m'
YELLOW='\033[1;33m'
NC='\033[0m'

log() {
  echo -e "${GREEN}[INFO]${NC} $1"
}

warn() {
  echo -e "${YELLOW}[WARN]${NC} $1"
}

error() {
  echo -e "${RED}[ERROR]${NC} $1"
}

echo
log "NebulaOps Kubernetes Bootstrap (k3s)"
echo

# ------------------------------------------------------------------------------
# Dependencies
# ------------------------------------------------------------------------------

sudo apt-get update

sudo apt-get install -y \
  ca-certificates \
  curl \
  apt-transport-https \
```

5.28 scripts/wsl/install-native-toolchain.sh

Script presente nel pacchetto. Verificare path, project name e side effect prima di usarlo in demo: alcuni helper sono legacy o generici.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Non sono stati rilevati comandi shell esterni nelle prime scansioni: leggere il sorgente prima di inserirlo in una pipeline.

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
bash "$SCRIPT_DIR/../linux/install-native-toolchain.sh"
if ! grep -q '^\[boot\]' /etc/wsl.conf 2>/dev/null; then
  sudo tee -a /etc/wsl.conf >/dev/null <<'CONF'
[boot]
systemd=true
CONF
fi
echo "WSL configured for native Docker Engine. Run from PowerShell: wsl --shutdown, then reopen
  Ubuntu."
```

5.29 scripts/wsl/install-prereqs-ubuntu.sh

Script presente nel pacchetto. Verificare path, project name e side effect prima di usarlo in demo: alcuni helper sono legacy o generici.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Non sono stati rilevati comandi shell esterni nelle prime scansioni: leggere il sorgente prima di inserirlo in una pipeline.

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
exec "$(dirname "$0")/install-native-toolchain.sh"
```

5.30 scripts/wsl/keycloak-apply-theme.sh

Script presente nel pacchetto. Verificare path, project name e side effect prima di usarlo in demo: alcuni helper sono legacy o generici.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
curl -sf -X PUT "${KC_URL}/admin/realms/${REALM}" \
```

Estratto sorgente

```
#!/usr/bin/env bash
# Apply NebulaOps theme to the nebulaops realm via Keycloak Admin REST API
# Run this after Keycloak is up: ./scripts/wsl/keycloak-apply-theme.sh

set -euo pipefail
KC_URL="${KC_URL:-http://localhost:8180}"
KC_ADMIN="${KC_ADMIN:-admin}"
KC_ADMIN_PASS="${KC_ADMIN_PASS:-admin}"
REALM="${KC_REALM:-nebulaops}"

echo "▶ Applying NebulaOps theme to Keycloak realm '$REALM'..."

# Get admin token
TOKEN=$(curl -sf -X POST "${KC_URL}/realms/master/protocol/openid-connect/token" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "client_id=admin-cli&username=${KC_ADMIN}&password=${KC_ADMIN_PASS}&grant_type=password" \
  | grep -o '"access_token":"[^"]*' | cut -d'"' -f4)

if [ -z "$TOKEN" ]; then
  echo "✗ Failed to get admin token — is Keycloak running on ${KC_URL}?"
  exit 1
fi
echo "✓ Admin token obtained"

# Apply theme to realm
curl -sf -X PUT "${KC_URL}/admin/realms/${REALM}" \
  -H "Authorization: Bearer ${TOKEN}" \
  -H "Content-Type: application/json" \
  -d
    '{"loginTheme":"nebulaops","accountTheme":"keycloak","adminTheme":"keycloak","emailTheme":"keycloak"}'
    \
  && echo "✓ Theme 'nebulaops' applied to realm '${REALM}'" \
  || echo "✗ Failed to apply theme"

echo ""
echo "  Open http://localhost:8180/realms/${REALM}/account to verify the login page."
```

5.31 scripts/wsl/lib/common.sh

Script presente nel pacchetto. Verificare path, project name e side effect prima di usarlo in demo: alcuni helper sono legacy o generici.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Non sono stati rilevati comandi shell esterni nelle prime scansioni: leggere il sorgente prima di inserirlo in una pipeline.

Estratto sorgente

```
#!/usr/bin/env bash
#       — Shared helpers sourced by all WSL scripts.
# Provides logging, status indicators, project naming, compose helpers.

# Project naming — derived once, consumed by all scripts
export PROJECT_NAME="${COMPOSE_PROJECT_NAME:-                    }"
export ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/../../.." && pwd)"
export COMPOSE_FILE="$ROOT_DIR/docker-compose.yml"
export CONFIG_FILE="$ROOT_DIR/config/platform.yml"

# ANSI colors (no-op when not a TTY)
if [ -t 1 ]; then
  C_RESET='\033[0m'; C_BOLD='\033[1m'
  C_GREEN='\033[32m'; C_YELLOW='\033[33m'; C_RED='\033[31m'
  C_CYAN='\033[36m'; C_DIM='\033[2m'
else
  C_RESET=''; C_BOLD=''; C_GREEN=''; C_YELLOW=''; C_RED=''; C_CYAN=''; C_DIM=''
fi

log_info()  { printf "${C_CYAN}${C_RESET}  %s\n" "$*"; }
log_ok()    { printf "${C_GREEN}${C_RESET}  %s\n" "$*"; }
log_warn()  { printf "${C_YELLOW}${C_RESET}  %s\n" "$*"; }
log_err()   { printf "${C_RED}${C_RESET}  %s\n" "$*" >&2; }
log_step()  { printf "\n${C_BOLD}${C_CYAN}▶ %s${C_RESET}\n" "$*"; }

dc()  { docker compose -p "$PROJECT_NAME" -f "$COMPOSE_FILE" "$@"; }

check_command() {
  command -v "$1" >/dev/null 2>&1 \
    && log_ok "$1 found" \
    || { log_err "$1 missing"; return 1; }
}

# Wait for a HTTP endpoint with timeout
wait_http() {
```

5.32 scripts/wsl/logs.sh

Tailing dei log compose per singolo servizio. Senza argomenti stampa la lista dei servizi di compose.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	primario

Non sono stati rilevati comandi shell esterni nelle prime scansioni: leggere il sorgente prima di inserirlo in una pipeline.

Estratto sorgente

```
#!/usr/bin/env bash
#       — Tail service logs. Usage: ./logs.sh [service-name] [--lines N]
set -euo pipefail
source "$(dirname "${BASH_SOURCE[0]}")/lib/common.sh"

if [ $# -eq 0 ]; then
  log_info "Available services:"
  dc config --services | sort | sed 's/^/  /'
  echo
  log_info "Usage: $0 <service-name> [--lines N]"
  exit 0
fi

SVC="$1"; shift || true
LINES=200
while [ $# -gt 0 ]; do
  case "$1" in
    --lines|-n) LINES="$2"; shift 2 ;;
    *) shift ;;
  esac
done

dc logs --tail="$LINES" -f "$SVC"
```

5.33 scripts/wsl/prepare-kubeconfig-for-docker.sh

Copia KUBECONFIG in .kube/config e sostituisce localhost/127.0.0.1 con IP WSL per renderlo raggiungibile dal container gateway.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
python3 - "$OUT_FILE" "$WSL_IP" <<'PY'
```

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/../.." && pwd)"
OUT_DIR="$ROOT_DIR/.kube"
OUT_FILE="$OUT_DIR/config"
mkdir -p "$OUT_DIR"

SOURCE="${KUBECONFIG:-}"
if [[ -z "$SOURCE" || ! -f "$SOURCE" ]]; then
  if [[ -f /etc/rancher/k3s/k3s.yaml ]]; then
    SOURCE=/etc/rancher/k3s/k3s.yaml
  elif [[ -f "$HOME/.kube/config" ]]; then
    SOURCE="$HOME/.kube/config"
  else
    echo "ERROR: kubeconfig not found. Expected /etc/rancher/k3s/k3s.yaml or ~/.kube/config" >&2
    exit 1
  fi
fi

if [[ ! -r "$SOURCE" ]]; then
  sudo cp "$SOURCE" "$OUT_FILE"
  sudo chown "$USER:$USER" "$OUT_FILE"
else
  cp "$SOURCE" "$OUT_FILE"
fi
chmod 600 "$OUT_FILE"

WSL_IP="$(hostname -I | awk '{print $1}')"
if [[ -z "$WSL_IP" ]]; then
  echo "ERROR: unable to detect WSL IP" >&2
  exit 1
fi

python3 - "$OUT_FILE" "$WSL_IP" <<'PY'
from pathlib import Path
```

5.34 scripts/wsl/prepare-runtime-tools.sh

Copia docker, kubectl, helm, trivy, terraform e argocd in .runtime-tools. Il gateway e i servizi runtime leggono questi binari montati per restituire dati live.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Non sono stati rilevati comandi shell esterni nelle prime scansioni: leggere il sorgente prima di inserirlo in una pipeline.

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/../.." && pwd)"
TOOLS_DIR="$ROOT_DIR/.runtime-tools"
mkdir -p "$TOOLS_DIR"

copy_tool() {
  local name="$1"
  local required="${2:-optional}"
  local src=""
  if command -v "$name" >/dev/null 2>&1; then
    src="$(command -v "$name")"
    cp -L "$src" "$TOOLS_DIR/$name"
    chmod 0755 "$TOOLS_DIR/$name"
    echo "OK mounted runtime tool: $name -> $TOOLS_DIR/$name"
  else
    rm -f "$TOOLS_DIR/$name"
    if [[ "$required" == "required" ]]; then
      echo "WARN required runtime tool missing on host: $name" >&2
    else
      echo "INFO optional runtime tool missing on host: $name" >&2
    fi
  fi
}

copy_tool docker required
copy_tool kubectl required
copy_tool helm optional
copy_tool trivy optional
copy_tool terraform optional
copy_tool argocd optional

cat > "$TOOLS_DIR/README.md" <<'INFO'
This directory is generated by scripts/wsl/prepare-runtime-tools.sh.
It contains host CLI binaries copied into a bind-mounted folder for backend containers.
```

5.35 scripts/wsl/reset.sh

Reset distruttivo di container e volumi per project name nebulaops. Va usato solo quando si accetta di perdere stato locale.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Non sono stati rilevati comandi shell esterni nelle prime scansioni: leggere il sorgente prima di inserirlo in una pipeline.

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/../.." && pwd)"
cd "$ROOT_DIR"
COMPOSE_FILE="$ROOT_DIR/infrastructure/docker-compose.yml"
PROJECT_NAME="nebulaops"
echo "This removes containers and local volumes."
read -r -p "Continue? [y/N] " answer
case "$answer" in y|Y|yes|YES) docker compose -p "$PROJECT_NAME" -f "$COMPOSE_FILE" down -v
  --remove-orphans; echo "Reset complete.";; *) echo "Cancelled.";; esac
```

5.36 scripts/wsl/restart-gateway.sh

Rebuild no-cache e recreate del solo gateway-service. Utile dopo modifiche a ProxyController, KubernetesOpsController, application.yml o runtime tools.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Non sono stati rilevati comandi shell esterni nelle prime scansioni: leggere il sorgente prima di inserirlo in una pipeline.

Estratto sorgente

```
#!/usr/bin/env bash
#       — Fast gateway restart (no-cache rebuild + restart only the gateway).
set -euo pipefail
source "$(dirname "${BASH_SOURCE[0]}")/lib/common.sh"

log_step "Rebuilding gateway-service (no cache)"
dc build --no-cache gateway-service

log_step "Restarting gateway-service"
dc up -d --force-recreate --no-deps gateway-service

log_step "Waiting for health"
wait_http "http://localhost:8080/api/health" 45 "gateway-service"
```

5.37 scripts/wsl/smoke-test.sh

Script presente nel pacchetto. Verificare path, project name e side effect prima di usarlo in demo: alcuni helper sono legacy o generici.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
curl -fsS -H 'Content-Type: application/json' -d
  '{"email":"demo@nebulaops.dev","displayName":"Demo
  User","password":"Password123!","organizationId":"demo-org"}' "$BASE/api/auth/register" ||
  true
curl -fsS -H 'Content-Type: application/json' -d '{"organizationId":"demo-
  org","projectId":"portfolio","title":"WSL smoke test task","description":"Created from
  scripts/wsl/smoke-test.sh","priority":"HIGH","labels":["wsl","rabbitmq","mongodb"]}'
  "$BASE/api/tasks"
curl -fsS http://localhost:8091/health >/dev/null && echo "go-cache-service OK"
```

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
BASE=${BASE:-http://localhost:8080}
echo "Registering demo user..."
curl -fsS -H 'Content-Type: application/json' -d
  '{"email":"demo@nebulaops.dev","displayName":"Demo
  User","password":"Password123!","organizationId":"demo-org"}' "$BASE/api/auth/register" ||
  true
echo
echo "Creating task through gateway..."
curl -fsS -H 'Content-Type: application/json' -d '{"organizationId":"demo-
  org","projectId":"portfolio","title":"WSL smoke test task","description":"Created from
  scripts/wsl/smoke-test.sh","priority":"HIGH","labels":["wsl","rabbitmq","mongodb"]}'
  "$BASE/api/tasks"
echo; echo "Tasks:"; curl -fsS "$BASE/api/tasks?organizationId=demo-org"
echo; echo "Notifications:"; curl -fsS "$BASE/api/notifications" || true
echo; echo "Smoke test completed."


echo "Checking Go cache service"
curl -fsS http://localhost:8091/health >/dev/null && echo "go-cache-service OK"
```

5.38 scripts/wsl/start.sh

Entry point consigliato su WSL. Esegue check-wsl, prepara kubeconfig per container, copia runtime tools, corregge il tema Keycloak, valida provisioning Grafana, crea rete nebulaops-network e avvia docker compose con project .

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	primario

Comandi esterni rilevati

```
curl -fsS --max-time 10 \
curl -fsS --max-time 10 \
docker network create nebulaops-network
```

Estratto sorgente

```
#!/usr/bin/env bash
#       — NebulaOps startup script with custom Keycloak OIDC login auto-check.
# Usage: ./scripts/wsl/start.sh [--rebuild-gateway]
set -euo pipefail
source "$(dirname "${BASH_SOURCE[0]}")/lib/common.sh"

REBUILD_GATEWAY=false
for arg in "$@"; do
  case "$arg" in
    --rebuild-gateway) REBUILD_GATEWAY=true ;;
    -h|--help)
      cat <<USAGE
Usage: $0 [--rebuild-gateway]
  --rebuild-gateway    Force no-cache rebuild of the gateway-service image
                       (use when gateway routes/config changed)
USAGE
      exit 0
      ;;
  esac
done

cd "$ROOT_DIR"

keycloak_theme_dir() {
  printf '%s\n' "$ROOT_DIR/infrastructure/keycloak/themes/nebulaops/login"
}

write_custom_keycloak_login_theme() {
  local theme_dir
  theme_dir="$(keycloak_theme_dir)"
  mkdir -p "$theme_dir/resources/css"

  # Important: do NOT ship/keep a custom template.ftl here.
  # Keycloak parent pages can then safely use the parent theme template,
  # while NebulaOps login.ftl stays fully standalone.
```

5.39 scripts/wsl/status.sh

Script presente nel pacchetto. Verificare path, project name e side effect prima di usarlo in demo: alcuni helper sono legacy o generici.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
docker compose -p "$PROJECT_NAME" -f "$COMPOSE_FILE" ps
```

Estratto sorgente

```
#!/usr/bin/env bash
set -euo pipefail
ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/../.." && pwd)"
cd "$ROOT_DIR"
COMPOSE_FILE="$ROOT_DIR/infrastructure/docker-compose.yml"
PROJECT_NAME="nebulaops"
docker compose -p "$PROJECT_NAME" -f "$COMPOSE_FILE" ps
echo
for url in http://localhost:8080/actuator/health http://localhost:8081/api/auth/healthz
  http://localhost:8082/actuator/health http://localhost:8083/actuator/health
  http://localhost:8084/actuator/health; do
  printf "%-48s" "$url"; curl -fsS "$url" >/dev/null 2>&1 && echo OK || echo not-ready
done
```

5.40 scripts/wsl/stop.sh

Arresta e ferma eventuali progetti legacy v21-2/v21-3 ancora attivi.

Campo	Valore
Tipo	Bash
Shebang	#!/usr/bin/env bash
Uso consigliato	supporto/diagnostica/legacy

Comandi esterni rilevati

```
docker compose -p "$LEGACY" -f "$COMPOSE_FILE" down 2>/dev/null || \
docker rm -f $(docker ps -aq --filter "label=com.docker.compose.project=$LEGACY") 2>/dev/null ||
  true
```

Estratto sorgente

```
#!/usr/bin/env bash
#       — Stop NebulaOps stack.
set -euo pipefail
source "$(dirname "${BASH_SOURCE[0]}")/lib/common.sh"

log_step "Stopping "
dc down "$@"

# Also stop legacy project names if still running
for LEGACY in nebulaops-v21-2-1 nebulaops-v21-3 nebulaops-v21-2; do
  if docker compose -p "$LEGACY" ps -q 2>/dev/null | grep -q .; then
    log_warn "Found running containers under legacy project '$LEGACY' — stopping them too"
    docker compose -p "$LEGACY" -f "$COMPOSE_FILE" down 2>/dev/null || \
      docker rm -f $(docker ps -aq --filter "label=com.docker.compose.project=$LEGACY")
        2>/dev/null || true
  fi
done

log_ok "All services stopped"
```

6. Docker Compose, volumi e rete

Il file di riferimento e docker-compose.yml alla root. La rete nebulaops-network e dichiarata external: true, quindi deve esistere prima dello start. Lo script start.sh la crea. I volumi persistenti root sono mongo-data, redis-data, rabbitmq-data, grafana-data e keycloak-db-data.

6.1 Volumi e persistenza

Volume	Usato da	Cosa conserva	Rischio reset
mongo-data	mongodb	utenti auth-service, task, notifiche, metadata file	Perdita dati applicativi se down -v
redis-data	redis	cache e blacklist/token future	Perdita cache accettabile in dev
rabbitmq-data	rabbitmq	queue/exchange e messaggi non consumati	Perdita eventi asincroni
grafana-data	grafana	dashboard runtime, utenti e impostazioni	Perdita dashboard custom
keycloak-db-data	keycloak-db	realm, client, utenti Keycloak	Reset login OIDC

6.2 Errori probabili al primo compose up

Sintomo	Causa reale	Fix
network nebulaops-network declared as external, but could not be found	La rete non esiste e si sta lanciando docker compose manualmente	docker network create nebulaops-network oppure usare ./scripts/wsl/start.sh
port is already allocated :4200/:8080/:3000	Vecchio stack v21.3/v20 o altro servizio attivo	docker ps; ./scripts/wsl/stop.sh; docker rm -f container
gateway-service healthy ma UI vuota	Frontend chiama /api ma nginx o gateway non risponde a rotta target	curl http://localhost:8080/api/health e logs gateway
Keycloak HTTP 500 su login	Tema FreeMarker custom con template.ftl non compatibile	./scripts/wsl/start.sh rigenera login standalone; controllare logs keycloak
Docker tab vuota	/var/run/docker.sock non montato o permessi	docker exec gateway command -v docker; controllare compose volumes e user

7. Autenticazione: Keycloak OIDC e auth-service JWT

La combina due livelli: Keycloak OIDC/PKCE nel frontend e auth-service con JWT dev/API. Il frontend espone costanti Keycloak in api.config.ts, mentre auth-service mantiene endpoint /api/auth per login/register/me/refresh/logout. Questo va documentato chiaramente: non sono la stessa identita, e in produzione occorre convergere su un identity provider unico o su validation JWT nel gateway.

7.1 Keycloak OIDC frontend

```
/**
 *       — Centralized frontend API configuration.
 * All HTTP calls read URLs from here. Synced from config/platform.yml.
 */
export const API_BASE = '/api';

export const API = {
  health: `${API_BASE}/health`,

  auth: {
    login:    `${API_BASE}/auth/login`,
    register: `${API_BASE}/auth/register`,
    me:       `${API_BASE}/auth/me`,
    refresh:  `${API_BASE}/auth/refresh`,
    logout:   `${API_BASE}/auth/logout`,
  },

  tasks: {
    list:   (org: string) => `${API_BASE}/tasks?organizationId=${encodeURIComponent(org)}`,
    create: `${API_BASE}/tasks`,
    update: (id: string) => `${API_BASE}/tasks/${encodeURIComponent(id)}`,
    delete: (id: string) => `${API_BASE}/tasks/${encodeURIComponent(id)}`,
    status: (id: string, s: string) =>
      `${API_BASE}/tasks/${encodeURIComponent(id)}/status/${s}`,
  },

  kubernetes: {
    snapshot:  `${API_BASE}/kubernetes/snapshot`,
    logs:      `${API_BASE}/kubernetes/logs`,
    deletePod:  (ns: string, name: string) => `${API_BASE}/kubernetes/pods/${ns}/${name}`,
    restartPod: (ns: string, name: string) =>
      `${API_BASE}/kubernetes/pods/${ns}/${name}/restart`,
    podLogs:    (ns: string, name: string) => `${API_BASE}/kubernetes/pods/${ns}/${name}/logs`,
    describePod:(ns: string, name: string) =>
      `${API_BASE}/kubernetes/pods/${ns}/${name}/describe`,
    scaleDeployment:   (ns: string, name: string) =>
      `${API_BASE}/kubernetes/deployments/${ns}/${name}/scale`,
    restartDeployment: (ns: string, name: string) =>
      `${API_BASE}/kubernetes/deployments/${ns}/${name}/restart`,
    deploymentYaml:    (ns: string, name: string) =>
      `${API_BASE}/kubernetes/deployments/${ns}/${name}/yaml`,
    describeDeployment:(ns: string, name: string) =>
      `${API_BASE}/kubernetes/deployments/${ns}/${name}/describe`,
    serviceYaml:    (ns: string, name: string) =>
      `${API_BASE}/kubernetes/services/${ns}/${name}/yaml`,
    describeService:(ns: string, name: string) =>
      `${API_BASE}/kubernetes/services/${ns}/${name}/describe`,
    ingressYaml:    (ns: string, name: string) =>
      `${API_BASE}/kubernetes/ingresses/${ns}/${name}/yaml`,
    describeIngress:(ns: string, name: string) =>
      `${API_BASE}/kubernetes/ingresses/${ns}/${name}/describe`,
    scaleStatefulSet:   (ns: string, name: string) =>
      `${API_BASE}/kubernetes/statefulsets/${ns}/${name}/scale`,
    restartStatefulSet: (ns: string, name: string) =>
      `${API_BASE}/kubernetes/statefulsets/${ns}/${name}/restart`,
    cordonNode:  (name: string) => `${API_BASE}/kubernetes/nodes/${name}/cordon`,
    uncordonNode:(name: string) => `${API_BASE}/kubernetes/nodes/${name}/uncordon`,
    drainNode:   (name: string) => `${API_BASE}/kubernetes/nodes/${name}/drain`,
    describeNode:(name: string) => `${API_BASE}/kubernetes/nodes/${name}/describe`,
    createNamespace: `${API_BASE}/kubernetes/namespaces`,
    apply:  `${API_BASE}/kubernetes/apply`,
    delete: `${API_BASE}/kubernetes/delete`,
  },

  runtime: {
    dockerContainers: `${API_BASE}/runtime/docker/containers`,
    dockerImages:     `${API_BASE}/runtime/docker/images`,
    dockerVolumes:    `${API_BASE}/runtime/docker/volumes`,
    dockerNetworks:   `${API_BASE}/runtime/docker/networks`,
    helmReleases:     `${API_BASE}/runtime/helm/releases?namespace=all`,
    containerStart:   (id: string) => `${API_BASE}/runtime/docker/containers/${id}/start`,
    containerStop:    (id: string) => `${API_BASE}/runtime/docker/containers/${id}/stop`,
    containerRestart: (id: string) => `${API_BASE}/runtime/docker/containers/${id}/restart`,
    containerPause:   (id: string) => `${API_BASE}/runtime/docker/containers/${id}/pause`,
    containerUnpause: (id: string) => `${API_BASE}/runtime/docker/containers/${id}/unpause`,
    containerRemove:  (id: string) => `${API_BASE}/runtime/docker/containers/${id}`,
    containerLogs:    (id: string) => `${API_BASE}/runtime/docker/containers/${id}/logs`,
    containerStats:   (id: string) => `${API_BASE}/runtime/docker/containers/${id}/stats`,
    imageRemove:  (id: string) => `${API_BASE}/runtime/docker/images/${id}`,
    imagesPrune:  `${API_BASE}/runtime/docker/images/prune`,
    volumeRemove: (name: string) => `${API_BASE}/runtime/docker/volumes/${name}`,
    volumesPrune: `${API_BASE}/runtime/docker/volumes/prune`,
    networksPrune: `${API_BASE}/runtime/docker/networks/prune`,
  },

  platform: {
    observability: `${API_BASE}/platform/observability`,
    gitops:        `${API_BASE}/platform/gitops`,
    devsecops:     `${API_BASE}/platform/devsecops`,
    environments:  `${API_BASE}/platform/environments`,
```

```
  },

  aiOps: {
```

7.2 Realm Keycloak

```
{
  "realm": "nebulaops",
  "displayName": "NebulaOps Platform",
  "enabled": true,
  "loginTheme": "nebulaops",
  "accountTheme": "keycloak",
  "adminTheme": "keycloak",
  "emailTheme": "keycloak",
  "sslRequired": "none",
  "registrationAllowed": false,
  "loginWithEmailAllowed": true,
  "duplicateEmailsAllowed": false,
  "resetPasswordAllowed": true,
  "editUsernameAllowed": false,
  "bruteForceProtected": true,
  "accessTokenLifespan": 3600,
  "refreshTokenMaxReuse": 0,
  "clients": [
    {
      "clientId": "nebulaops-frontend",
      "name": "NebulaOps Frontend",
      "enabled": true,
      "publicClient": true,
      "standardFlowEnabled": true,
      "directAccessGrantsEnabled": true,
      "redirectUris": [
        "http://localhost:4200/*",
        "http://localhost:4201/*"
      ],
      "webOrigins": [
        "http://localhost:4200",
        "http://localhost:4201"
      ],
      "attributes": {
        "pkce.code.challenge.method": "S256"
      }
    },
    {
      "clientId": "nebulaops-gateway",
      "name": "NebulaOps Gateway",
      "enabled": true,
      "publicClient": false,
      "secret": "nebulaops-secret-dev-only",
      "standardFlowEnabled": false,
      "serviceAccountsEnabled": true,
      "directAccessGrantsEnabled": true,
      "authorizationServicesEnabled": true
    }
  ],
  "roles": {
    "realm": [
      {
        "name": "nebula-admin",
        "description": "NebulaOps platform administrator with full access"
      },
      {
        "name": "nebula-operator",
        "description": "NebulaOps operator with read/write access to operations"
      },
      {
        "name": "nebula-viewer",
        "description": "NebulaOps read-only viewer"
      }
    ]
  },
```

7.3 auth-service JWT

AuthController.java contiene register, login, refresh, me, logout e healthz. Il dev shortcut admin/admin emette token dev-jwt-admin-* anche senza MongoDB. Questo e utile in locale ma va disabilitato in produzione.

Endpoint	Comportamento	Rischio/nota
POST /api/auth/register	Crea UserAccount in MongoDB con password prefix plain-dev-only	Non usare hashing dev in produzione
POST /api/auth/login	Valida user o shortcut admin/admin	Shortcut da rimuovere prima di demo pubblica non controllata
POST /api/auth/refresh	Accetta Bearer refresh token e genera nuovo access token	Manca rotazione/blacklist Redis

Endpoint	Comportamento	Rischio/nota
GET /api/auth/me	Valida JWT o token dev admin	Gateway non applica ancora auth globale a tutte le route
POST /api/auth/logout	Stateless client-side	Serve revocation server-side in produzione

8. Manuale UI reale: schede operative senza ripetizioni

Ogni scheda segue un formato diverso e concreto: scopo, file sorgente, dati/API, azioni e limite tecnico. Non viene ripetuta la frase generica "aprire, aggiornare, filtrare". Le sezioni foundation sono dichiarate come tali.

8.1 OVERVIEW - Home Command Center, feature launcher e service galaxy

Aggrega homeLaunchers, platformTools e terraformModules. Non è una pagina statica: i pulsanti cambiano activeTab e richiamano caricamenti live come loadK8sFromApi, loadDocker, loadGitOps e loadDevSecOps.

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:97-156; frontend/src/app/app.component.ts:647-746
Maturita	Mista: UI pronta con backend/fallback
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

8.2 INFRA - Hub degli strumenti esterni e accessi rapidi

Espone URL locali di Grafana, Prometheus, Loki, Tempo, RabbitMQ, Mongo Express, Redis Commander, Keycloak e Gateway. Nel codice openInfraLink apre link esterni e il pannello interno instrada alle sezioni operative.

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:157-204; docker-compose.yml
Maturita	Mista: UI pronta con backend/fallback
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

8.3 TASKS - Kanban reale con CRUD REST, MongoDB e RabbitMQ

loadTasks usa API.tasks.list; createTask invia POST /api/tasks; il drag&drop aggiorna lo stato con PATCH /api/tasks/{id}/status/{status}. Ogni mutazione pubblica eventi su RabbitMQ exchange nebulaops.tasks.

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:205-234; TaskController.java; RabbitMqConfig.java
Maturita	Live
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

8.4 CONTAINERS - Docker Desktop locale via Docker Engine API su Unix socket

Mostra container, immagini, volumi e network. Le azioni start/stop/restart/pause/unpause/remove/logs/stats chiamano endpoint gateway, non dati mock. Se il socket non è montato, il service ritorna fallback/toolStatus e la UI mostra stato non-live.

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:235-370; DockerActionsController.java; DockerSocketClient.java
Maturita	Live
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

Scenario demo: avviare la Spring demo via docker compose, cercare container nebulaops-spring-demo, aprire logs, restartare il container e verificare che lo stato cambi. Le azioni chiamano API.runtime.containerRestart/containerLogs definite in frontend/src/app/api.config.ts.

8.5 KUBERNETES - OpenLens-like Kubernetes console

La pagina combina snapshot cluster, lens tree e action grid. Le operazioni reali usano kubectl dentro gateway con /root/.kube/config montato e binario copiato in .runtime-tools.

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:371-533; KubernetesOpsController.java; KubernetesPlatformService.java
Maturita	Live
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

Scenario demo: deploy namespace nebulaops-demo, scalare deployment a 3 repliche dalla UI, verificare in OpenLens-like che il conteggio pods cambi, poi leggere pod logs e describe in caso di ImagePullBackOff.

8.6 TERRAFORM - Runtime map e moduli IaC base

La tab base visualizza moduli, drift e stato plan. Il codice di repository contiene due alberi Terraform: terraform/ e infrastructure/terraform/, con moduli kind, apps, gitops, observability e security.

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:534-555; terraform/*.tf; infrastructure/terraform
Maturita	Mista: UI pronta con backend/fallback
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

8.7 HELM - Release view e deploy chart

loadHelm chiama API.runtime.helmReleases. Se helm è disponibile nel container gateway, RuntimeOpsController ritorna release reali; altrimenti lista vuota/fallback.

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:556-587; RuntimeOpsController.java; infrastructure/helm/nebulaops
Maturita	Live
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

8.8 OBSERVABILITY - Prometheus, Loki, Tempo, Grafana e logs live

loadObservability legge /api/platform/observability. Il gateway arricchisce status, metriche e stream usando Prometheus/Loki quando raggiungibili, con link a Grafana e queries operative.

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:588-637; PlatformLiveController.java; infrastructure/observability
Maturita	Live
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

8.9 GITOPS - ArgoCD application state, sync e drift

La sezione visualizza app, revisioni, drift e stato sync. Gli script argocd-apply-local.sh e GitLab deploy:argocd sono il percorso operativo.

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:638-672; gitops-control-service; infrastructure/argocd
Maturita	Mista: UI pronta con backend/fallback
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

8.10 ENVIRONMENTS - Namespace/ambienti applicativi

loadEnvironments carica /api/platform/environments e mostra namespace con deploy, pods, health e costi stimati. Utile per separare dev/stage/prod/demo.

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:673-701; environment-manager-service
Maturita	Mista: UI pronta con backend/fallback
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

8.11 TERRAFORM STUDIO - Plan preview, graph e moduli

Espone plan, graph e modules. Lo stato attuale è foundation: endpoint leggibili e preview, ma apply remoto deve essere protetto da policy e approval.

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:702-742; TerraformStudioServiceController.java
Maturita	Mista: UI pronta con backend/fallback
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

8.12 AI OPS - RCA, console operativa e patch suggerite

runAiOpsAnalysis chiama /api/ai-ops/analyze, produce confidence, rootCause, blastRadius e YAML. autoFix è deliberatamente controllato: prima genera proposta, poi eventuale azione.

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:743-830; ai-engine/app/main.py; AiOpsController.java
Maturita	Mista: UI pronta con backend/fallback
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

8.13 CICD - Pipeline designer con canvas e GitLab/ArgoCD bridge

La UI genera YAML pipeline, seleziona nodi e può inviare trigger. La pipeline reale nel repository ha stage test, build, package, deploy e verify.

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:831-867; .gitlab-ci.yml; PipelineEngineController.java
Maturita	Mista: UI pronta con backend/fallback
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

Scenario demo: mostrare pipelineYaml generato, confrontarlo con .gitlab-ci.yml reale della demo e spiegare differenza tra designer UI e pipeline GitLab eseguibile.

8.14 SECURITY/COMPLIANCE/VULNERABILITIES - DevSecOps dashboard con scan, secrets e CVE

RUN SCAN usa API.platform.devsecops e secrets endpoint. Il valore operativo dipende da trivy copiato in .runtime-tools e dal mount docker.sock per leggere immagini/container.

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:868-948; DevSecOpsController.java; PlatformLiveController.java
Maturita	Mista: UI pronta con backend/fallback
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

Scenario demo: eseguire RUN SCAN dopo prepare-runtime-tools. Se trivy manca, mostrare come diagnose-runtime evidenzia il tool mancante invece di falsificare risultati.

8.15 FINOPS - Costo stimato e unit economics

Foundation service: CostController espone summary, breakdown e POST entries. Serve per spiegare metriche di costo locale/cluster ma non è ancora integrato in modo completo nel compose root.

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:949-968; cost-analytics-service
Maturita	Foundation/visuale con endpoint parziali
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

8.16 BACKUPS - Backup locale e restore assistito

La UI è un pannello; lo script backup-local copia terraform/grafana e salva compose ps. Non effettua dump MongoDB: il manuale include procedura mongodump/mongorestore da aggiungere.

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:969-989; scripts/backup-local.sh; scripts/restore-local.sh
Maturita	Foundation/visuale con endpoint parziali
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

8.17 REGISTRY - Registry images e immagini locali

La sezione è ponte tra immagini Docker locali e concetto di registry. Nel pacchetto non c’è registry container dedicato; si usa Docker locale/CI registry esterno.

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:990-1021; ProxyController /registry/images; Docker Runtime
Maturita	Mista: UI pronta con backend/fallback
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

8.18 SERVICE MESH - Mappa traffic policy foundation

Sezione visuale foundation: non installa Istio/Linkerd. Deve essere documentata come design cockpit e non come service mesh operativo già presente.

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:1022-1046
Maturita	Foundation/visuale con endpoint parziali
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

8.19 SECRETS - Secrets inventory e rotazione policy

Legge secret scanning, non espone valori segreti. Il runbook indica di non committare .env e di usare Kubernetes Secret o Docker secrets in produzione.

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:1047-1067; /api/platform/devsecops/secrets
Maturita	Mista: UI pronta con backend/fallback
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

8.20 DATABASES - MongoDB e data services

MongoDB porta 27017 e Mongo Express 8088. I servizi task/auth/file/notification persistono documenti; il manuale include collection attese e health check.

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:1068-1089; docker-compose.yml
Maturita	Mista: UI pronta con backend/fallback
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

8.21 QUEUES - RabbitMQ e flussi event-driven

RabbitMQ esposto su 5672/15672. task-service pubblica eventi TaskCreated/Updated/Deleted, notification-service li consuma e produce notifiche.

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:1090-1105; RabbitMqConfig.java; notification-service
Maturita	Mista: UI pronta con backend/fallback
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

8.22 SLO - SLO cockpit foundation

Il cockpit usa metriche Prometheus/Actuator. Per produzione servono alert rules e recording rules; il manuale propone SLI HTTP, availability e error rate.

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:1106-1122; Prometheus
Maturita	Mista: UI pronta con backend/fallback
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

8.23 INCIDENTS - Incident console e correlazione

Sezione correlata a AI OPS. Serve per annotare incident timeline, causa radice e comandi di remediation.

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:1123-1143; AI OPS; Observability
Maturita	Mista: UI pronta con backend/fallback
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

8.24 AUDIT - Audit events e governance

La rotta esiste come proxy/foundation. In produzione va collegata a database append-only con correlation id, utente, endpoint, before/after e outcome.

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:1144-1163; ProxyController /audit/events
Maturita	Foundation/visuale con endpoint parziali
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

8.25 DOCS - Documentazione interna

Mostra documenti aggiornati e note. La revisione corrente evita di trattare file legacy come fonte primaria quando contraddicono .

Aspetto	Dettaglio
Sorgente UI	frontend/src/app/app.component.html:1164-1175; docs/*.md
Maturita	Mista: UI pronta con backend/fallback
Uso demo	Mostrare flusso dati reale, poi usare log/gateway per provare che non e mock.

9. Docker Desktop in NebulaOps

La sezione Docker e implementata come pannello runtime collegato al Docker Engine locale. Nel codice il gateway comunica con Docker attraverso DockerSocketClient e DockerActionsController. I read endpoints sono in RuntimeOpsController e PlatformLiveController; le mutazioni sono dedicate in DockerActionsController.

9.1 Endpoint Docker reali

Metodo	Path	Controller/File
POST	/api/runtime/docker/containers/{id}/start	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/DockerActionsController.java
POST	/api/runtime/docker/containers/{id}/stop	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/DockerActionsController.java
POST	/api/runtime/docker/containers/{id}/restart	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/DockerActionsController.java
POST	/api/runtime/docker/containers/{id}/pause	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/DockerActionsController.java
POST	/api/runtime/docker/containers/{id}/unpause	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/DockerActionsController.java
DELETE	/api/runtime/docker/containers/{id}	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/DockerActionsController.java
GET	/api/runtime/docker/containers/{id}/logs	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/DockerActionsController.java
GET	/api/runtime/docker/containers/{id}/stats	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/DockerActionsController.java
DELETE	/api/runtime/docker/images/{id}	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/DockerActionsController.java
POST	/api/runtime/docker/images/prune	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/DockerActionsController.java
DELETE	/api/runtime/docker/volumes/{name}	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/DockerActionsController.java
POST	/api/runtime/docker/volumes/prune	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/DockerActionsController.java
POST	/api/runtime/docker/networks/prune	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/DockerActionsController.java
GET	/api/platform/docker/containers	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/PlatformLiveController.java
GET	/api/platform/docker/images	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/PlatformLiveController.java
GET	/api/platform/docker/volumes	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/PlatformLiveController.java
GET	/api/platform/docker/networks	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/PlatformLiveController.java
GET	/api/platform/docker/builds	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/PlatformLiveController.java
GET	/api/platform/docker/scout	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/PlatformLiveController.java
GET	/api/runtime/docker/containers	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/RuntimeOpsController.java
GET	/api/runtime/docker/images	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/RuntimeOpsController.java
GET	/api/runtime/docker/volumes	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/RuntimeOpsController.java
GET	/api/runtime/docker/stats	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/RuntimeOpsController.java
GET	/api/runtime/docker/networks	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/RuntimeOpsController.java
GET	/api/runtime/docker/builds	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/RuntimeOpsController.java

9.2 Runbook Docker demo

```
# Nel progetto demo
cd nebulaops-spring-demo-
./scripts/build-local.sh
./scripts/run-docker.sh

# Nel progetto NebulaOps
./scripts/wsl/health.sh
curl http://localhost:8080/api/runtime/docker/containers | jq
curl http://localhost:8080/api/runtime/docker/containers/<id>/logs?tail=100 | jq -r .logs
```

9.3 Errori concreti e diagnosi

Errore	Comando di conferma	Fix
Docker containers empty ma docker ps mostra servizi	gateway non ha socket o DockerSocketClient fallisce	controllare docker-compose.yml volume /var/run/docker.sock e logs gateway
restart container ritorna 409	container in stato non compatibile o in rimozione	refresh dopo 3s, usare docker inspect
logs illeggibili	Docker stream multiplexed contiene header binari	DockerActionsController rimuove header; verificare funzione logsRaw/parse
remove volume fallisce	volume in uso da container	fermare container collegati prima di DELETE volume

10. OpenLens/Kubernetes in NebulaOps

La sezione Kubernetes emula un subset operativo di OpenLens: lens tree, snapshot, risorse, logs, describe, scale, restart, YAML get/apply, node cordon/uncordon/drain e namespace create. Il controller costruisce anche fallback da Docker quando il cluster non e live, ma le azioni reali richiedono kubectl funzionante nel gateway.

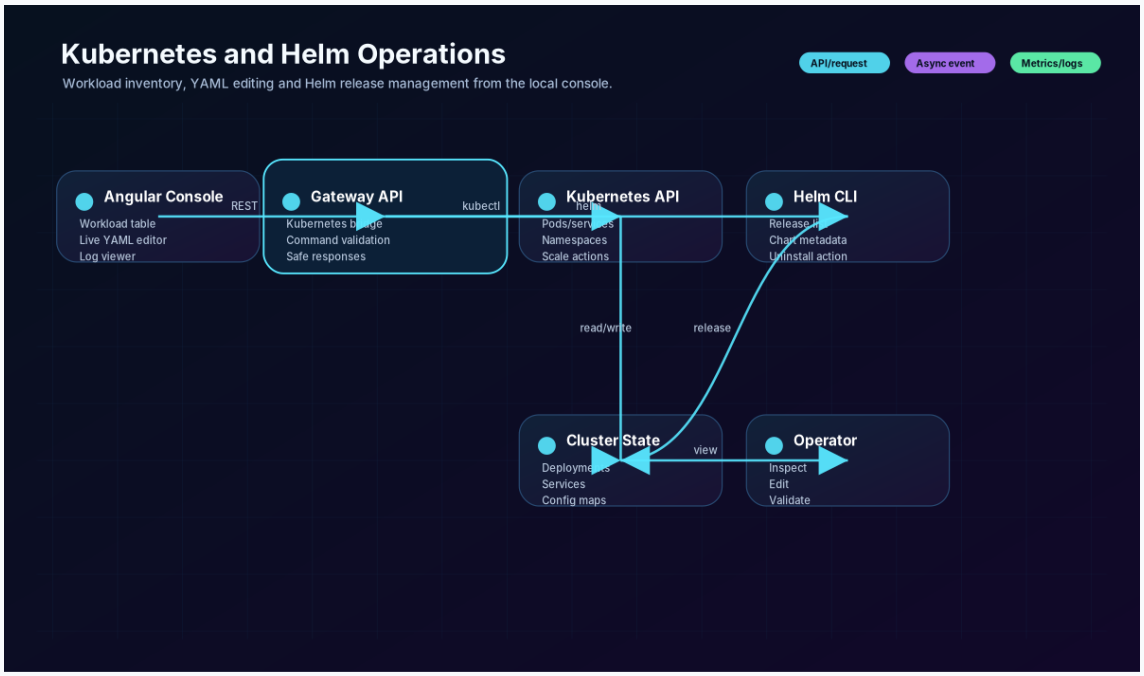

Figura 4 - docs/diagrams/kubernetes-helm-view.svg: relazione tra Kubernetes console, Helm chart e release; aiuta a distinguere deploy manuale kubectl da deploy Helm/GitOps.

10.1 Endpoint Kubernetes reali (1-28)

Metodo	Path	File
GET	/api/kubernetes/snapshot	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
GET	/api/kubernetes/logs	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
GET	/api/kubernetes/health	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
GET	/api/kubernetes/nodes	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
GET	/api/kubernetes/namespaces	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
GET	/api/kubernetes/resources	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
GET	/api/kubernetes/resources/{resourceId}	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
GET	/api/kubernetes/kind/{kind}	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
DELETE	/api/kubernetes/pods/{ns}/{name}	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
POST	/api/kubernetes/pods/{ns}/{name}/restart	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
GET	/api/kubernetes/pods/{ns}/{name}/logs	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
GET	/api/kubernetes/pods/{ns}/{name}/describe	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
POST	/api/kubernetes/deployments/{ns}/{name}/scale	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
POST	/api/kubernetes/deployments/{ns}/{name}/restart	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
GET	/api/kubernetes/deployments/{ns}/{name}/yaml	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
POST	/api/kubernetes/deployments/{ns}/{name}/yaml	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
GET	/api/kubernetes/deployments/{ns}/{name}/describe	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
GET	/api/kubernetes/services/{ns}/{name}/yaml	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
POST	/api/kubernetes/services/{ns}/{name}/yaml	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
GET	/api/kubernetes/services/{ns}/{name}/describe	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
GET	/api/kubernetes/ingresses/{ns}/{name}/yaml	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
POST	/api/kubernetes/ingresses/{ns}/{name}/yaml	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
GET	/api/kubernetes/ingresses/{ns}/{name}/describe	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java

Metodo	Path	File
POST	/api/kubernetes/statefulsets/{ns}/{name}/scale	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
POST	/api/kubernetes/statefulsets/{ns}/{name}/restart	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
GET	/api/kubernetes/configmaps/{ns}/{name}/yaml	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
POST	/api/kubernetes/configmaps/{ns}/{name}/yaml	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
POST	/api/kubernetes/nodes/{name}/cordon	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java

10.1 Endpoint Kubernetes reali (29-35)

Metodo	Path	File
POST	/api/kubernetes/nodes/{name}/uncordon	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
POST	/api/kubernetes/nodes/{name}/drain	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
GET	/api/kubernetes/nodes/{name}/describe	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
POST	/api/kubernetes/namespaces	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
GET	/api/kubernetes/namespaces/{name}/describe	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
POST	/api/kubernetes/apply	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
POST	/api/kubernetes/delete	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java

10.2 Come leggere una risorsa in OpenLens-like

Quando si seleziona una riga, selectedResourcePanel mostra YAML, status, CPU/RAM/restarts e azioni contestuali. Deployment e StatefulSet possono scalare o fare rollout restart; Pod puo leggere logs/describe o delete; Node puo cordon/uncordon/drain. La UI espone le operazioni supportate dagli endpoint presenti nel gateway; funzioni non collegate a endpoint dedicati, come un port-forward applicativo da bottone, restano fuori dal flusso operativo documentato.

10.3 Runbook ImagePullBackOff

```
kubectl -n nebulaops-demo get pods
kubectl -n nebulaops-demo describe pod <pod>
kubectl -n nebulaops-demo get deploy nebulaops-spring-demo -o yaml | grep image:
# Se Kind non ha immagine locale:
kind load docker-image nebulaops-spring-demo:        --name
kubectl -n nebulaops-demo rollout restart deploy/nebulaops-spring-demo
```

11. Observability stack

Lo stack osservabilità e composto da Prometheus, Loki, Tempo, OTel Collector e Grafana. Il punto chiave non e solo aprire Grafana: bisogna sapere quali target vengono scrape-ati e quali endpoint Actuator/Prometheus espone ogni servizio.

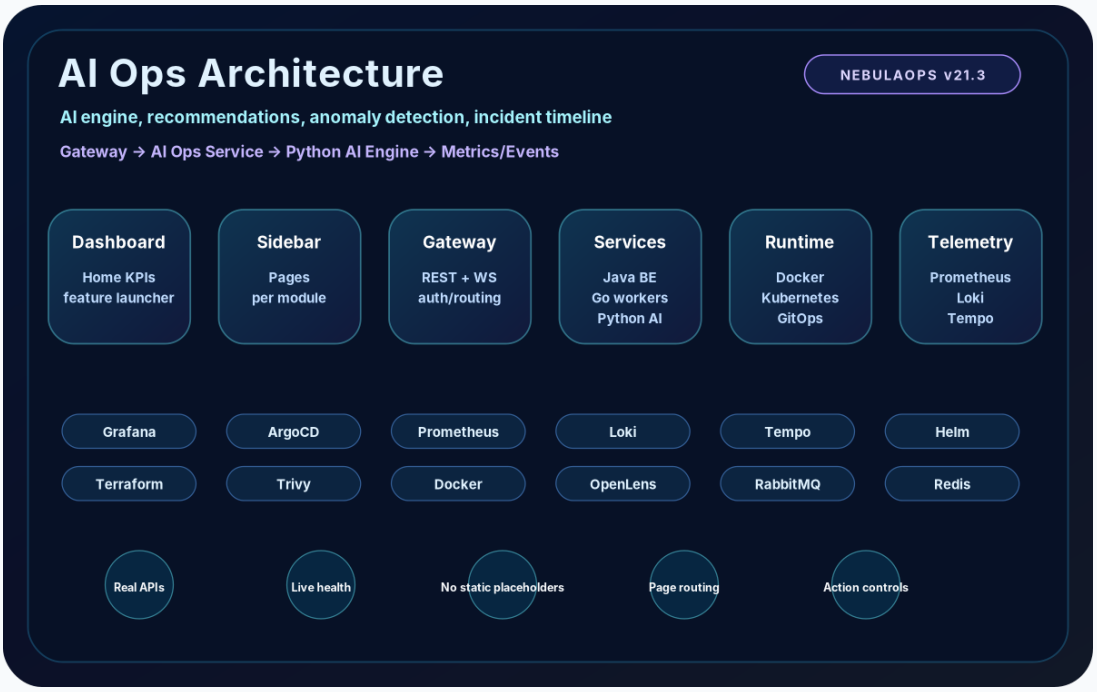

Figura 5 - frontend/src/assets/ ·ai-ops-architecture.svg: dependency graph usato dalla UI per spiegare correlazione tra eventi, AI OPS e osservabilità.

11.x infrastructure/observability/prometheus/prometheus.yml

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: prometheus
    static_configs:
      - targets: [ "prometheus:9090" ]

  - job_name: gateway-service
    metrics_path: /actuator/prometheus
    static_configs:
      - targets: [ "gateway-service:8080" ]

  - job_name: auth-service
    metrics_path: /actuator/prometheus
    static_configs:
      - targets: [ "auth-service:8081" ]

  - job_name: task-service
    metrics_path: /actuator/prometheus
    static_configs:
      - targets: [ "task-service:8082" ]

  - job_name: notification-service
    metrics_path: /actuator/prometheus
    static_configs:
      - targets: [ "notification-service:8083" ]

  - job_name: file-service
    metrics_path: /actuator/prometheus
    static_configs:
      - targets: [ "file-service:8084" ]

  - job_name: go-cache-service
    metrics_path: /health
    static_configs:
      - targets: [ "go-cache-service:8091" ]
```

11.x infrastructure/observability/otel-collector/config.yaml

```
receivers:
  otlp:
    protocols:
      http:
      grpc:
exporters:
  otlp:
    endpoint: tempo:4317
    tls:
      insecure: true
  logging: { }
service:
  pipelines:
    traces:
      receivers: [ otlp ]
      exporters: [ otlp, logging ]
```

11.x infrastructure/observability/tempo/tempo.yaml

```
server:
  http_listen_port: 3200
distributor:
  receivers:
    otlp:
      protocols:
        grpc:
        http:
storage:
  trace:
    backend: local
    local:
      path: /tmp/tempo/traces
```

11.1 Query operative

Sistema	Query/URL	Uso
Prometheus	up	Verifica target live
Prometheus	jvm_memory_used_bytes	Controllo memoria JVM servizi Spring
Prometheus	http_server_requests_seconds_count	Volume chiamate API
Loki	{container="gateway-service"}	Log gateway se configurati
Grafana	Dashboards -> NebulaOps Platform Overview	Vista portfolio e screenshot demo

12. DevSecOps, secrets e vulnerabilita

La parte DevSecOps ha due livelli: service dedicato backend/devsecops-service e aggregazione gateway /api/platform/devsecops. Le scansioni reali dipendono da trivy e docker disponibili nel container; in assenza degli strumenti va mostrato toolStatus/fallback, non un risultato inventato.

Metodo	Path	File
GET	/api/devsecops/secrets	backend/devsecops-service/src/main/java/dev/nebulaops/devsecops/DevSecOpsController.java
GET	/api/devsecops/image	backend/devsecops-service/src/main/java/dev/nebulaops/devsecops/DevSecOpsController.java
GET	/api/platform/devsecops	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/PlatformLiveController.java
GET	/api/platform/devsecops/secrets	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/PlatformLiveController.java
GET	/api/secrets/list	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/ProxyController.java

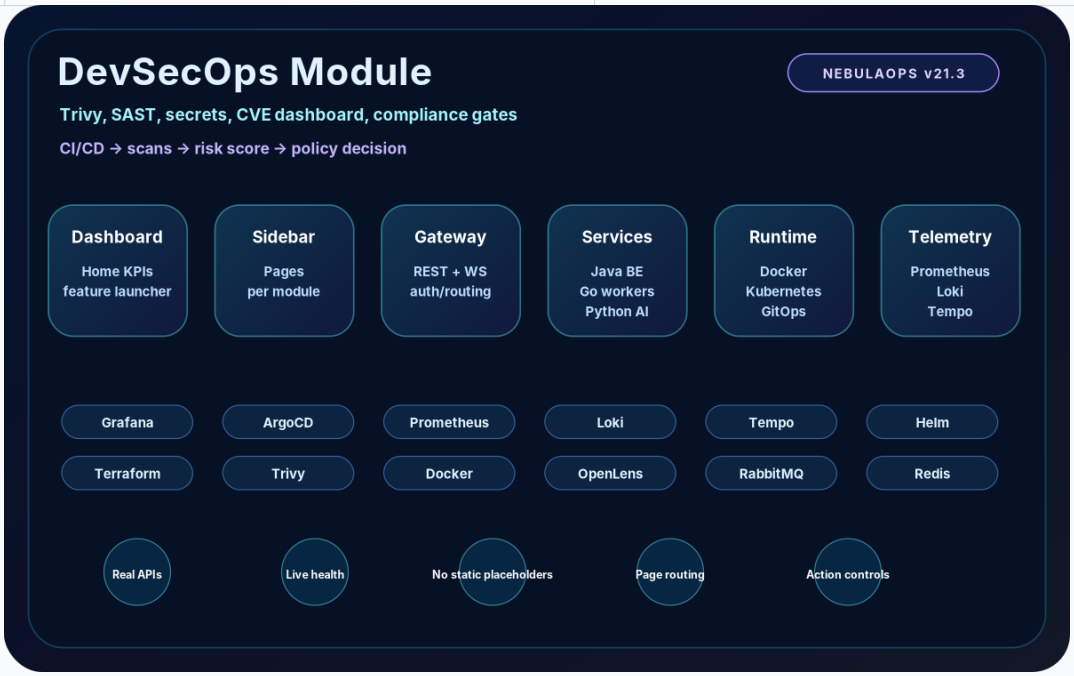

Figura 6 - frontend asset devsecops: illustra risk score, radar, CVE e compliance, ma il dato operativo arriva da DevSecOpsController/PlatformLiveController.

12.1 Hardening minimo

- Non montare docker.sock in produzione senza proxy autorizzativo.
- Sostituire password dev con Keycloak realm user/roles e secret manager.
- Aggiungere audit append-only per ogni azione Docker/Kubernetes distruttiva.
- Limitare kubectl con ServiceAccount/RBAC e namespace allow-list.
- Gestire Trivy DB cache e aggiornamenti in pipeline.

13. CI/CD, GitLab, Helm e ArgoCD

La pipeline reale del progetto root e nella .gitlab-ci.yml, mentre il progetto demo contiene una pipeline completa test/build/package/deploy/verify. L integrazione GitOps si appoggia ai manifest infrastructure/argocd e a scripts/argocd-apply-local.sh.

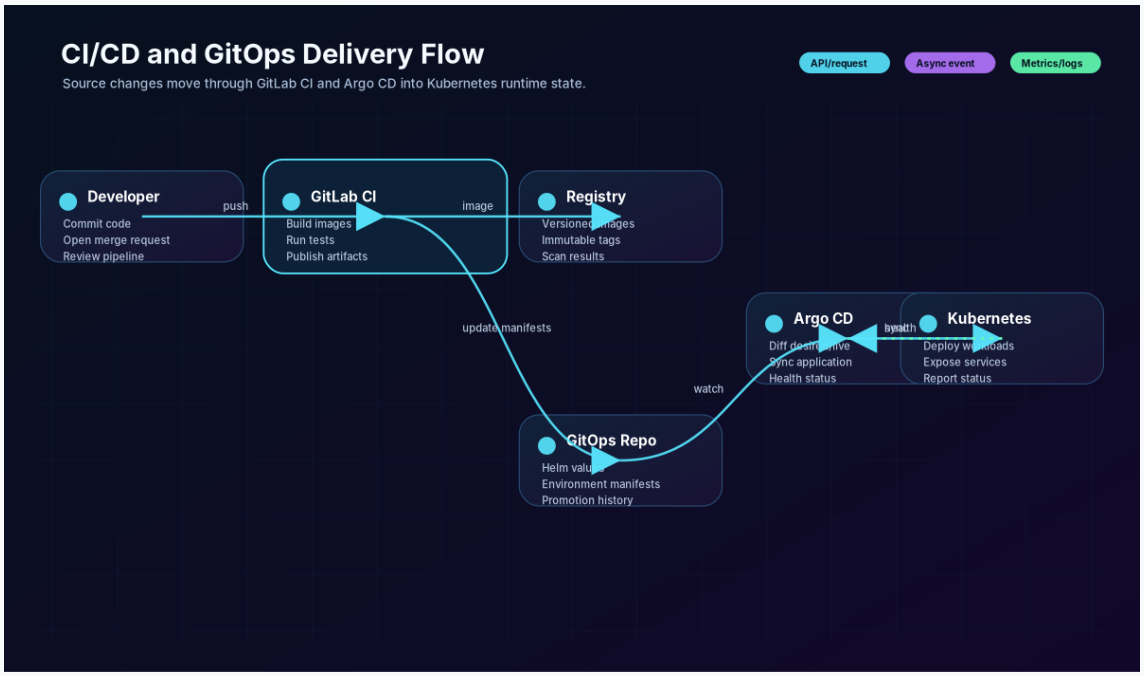

Figura 7 - docs/diagrams/gitlab-argocd-flow.svg: flusso GitLab -> registry -> Helm chart -> ArgoCD sync -> cluster. Questo e il percorso da mostrare in portfolio.

13.x .gitlab-ci.yml

```
# GitLab CI/CD pipeline for NebulaOps.
# Required protected variables in GitLab CI/CD settings:
# - ARGOCD_SERVER, ARGOCD_AUTH_TOKEN: used by the argocd-sync job.
# - NEBULAOPS_BASE_URL: optional deployed gateway URL for smoke tests.

stages:
  - validate
  - test
  - build
  - package
  - deploy
  - verify

variables:
  DOCKER_TLS_CERTDIR: ""
  DOCKER_DRIVER: overlay2
  IMAGE_TAG: "$CI_COMMIT_SHORT_SHA"
  HELM_CHART_DIR: infrastructure/helm/nebulaops
  HELM_RELEASE_NAME: nebulaops
  HELM_RELEASE_NAMESPACE: nebulaops

workflow:
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH
    - if: $CI_COMMIT_TAG

cache:
  key: "$CI_COMMIT_REF_SLUG"
  paths:
    - .m2/repository
    - frontend/node_modules/
    - go/pkg/mod/

validate:package:
  stage: validate
  image: python:3.12-alpine
  before_script:
    - pip install pyyaml
  script:
    - python scripts/validate-package.py
  artifacts:
    when: always
    paths:
      - docs/QUALITY_REPORT.md
    expire_in: 1 week

validate:shell:
  stage: validate
  image: bash:5.2-alpine3.20
  script:
    - find scripts -name "*.sh" -print0 | xargs -0 -I{} bash -n {}

test:go:
  stage: test
  image: golang:1.22-alpine
  script:
    - cd go/cache-service && go test ./...
    - cd ../event-worker && go test ./...

test:frontend:
  stage: test
  image: node:20-alpine
  script:
    - cd frontend
    - npm ci
    - npm run build
  artifacts:
    paths:
      - frontend/dist/
    expire_in: 1 week

test:backend:
  stage: test
  image: maven:3.9.8-eclipse-temurin-21
  script:
    - cd backend
    - mvn -B -ntp test
  artifacts:
    when: always
```

13.x infrastructure/argocd/project.yaml

```
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: nebulaops
  namespace: argocd
spec:
  description: NebulaOps GitOps project
  sourceRepos:
    - '*'
  destinations:
    - namespace: nebulaops
      server: https://kubernetes.default.svc
    - namespace: nebulaops-dev
      server: https://kubernetes.default.svc
    - namespace: nebulaops-staging
      server: https://kubernetes.default.svc
    - namespace: nebulaops-prod
      server: https://kubernetes.default.svc
  clusterResourceWhitelist:
    - group: ''
      kind: Namespace
  namespaceResourceWhitelist:
    - group: '*'
      kind: '*'
  roles:
    - name: cicd-sync
      description: Allows GitLab CI to sync NebulaOps applications
      policies:
        - p, proj:nebulaops:cicd-sync, applications, sync, nebulaops/*, allow
        - p, proj:nebulaops:cicd-sync, applications, get, nebulaops/*, allow
```

13.x infrastructure/argocd/application.yaml

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: nebulaops
  namespace: argocd
  labels:
    app.kubernetes.io/name: nebulaops
    app.kubernetes.io/part-of: nebulaops-platform
spec:
  project: nebulaops
  source:
    repoURL: https://gitlab.com/your-group/nebulaops.git
    targetRevision: main
    path: infrastructure/helm/nebulaops
    helm:
      releaseName: nebulaops
      valueFiles:
        - values.yaml
  destination:
    server: https://kubernetes.default.svc
    namespace: nebulaops
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
      - PruneLast=true
      - ApplyOutOfSyncOnly=true
  revisionHistoryLimit: 5
```

13.x infrastructure/argocd/applicationset.yaml

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: nebulaops-environments
  namespace: argocd
spec:
  generators:
    - list:
        elements:
          - env: dev
            namespace: nebulaops-dev
            revision: develop
          - env: staging
            namespace: nebulaops-staging
            revision: main
          - env: prod
            namespace: nebulaops-prod
            revision: main
  template:
    metadata:
      name: 'nebulaops-{{env}}'
      labels:
        environment: '{{env}}'
    spec:
      project: nebulaops
      source:
        repoURL: https://gitlab.com/your-group/nebulaops.git
        targetRevision: '{{revision}}'
        path: infrastructure/helm/nebulaops
        helm:
          releaseName: 'nebulaops-{{env}}'
          valueFiles:
            - values.yaml
          parameters:
            - name: global.namespace
              value: '{{namespace}}'
      destination:
        server: https://kubernetes.default.svc
        namespace: '{{namespace}}'
      syncPolicy:
        automated:
          prune: true
          selfHeal: true
        syncOptions:
          - CreateNamespace=true
```

13.x infrastructure/helm/nebulaops/Chart.yaml

```
apiVersion: v2
name: nebulaops
description: Helm chart for the NebulaOps Angular + Spring Boot + MongoDB + RabbitMQ + Redis
  platform
type: application
version: 21.2.1
appVersion: "21.2.1"
keywords:
  - spring-boot
  - angular
  - mongodb
  - rabbitmq
  - redis
  - grafana
  - portfolio
maintainers:
  - name: Peyman Eshghi Malayeri
```

13.x infrastructure/helm/nebulaops/values.yaml

```
global:
  imageRegistry: ""
  imagePullPolicy: IfNotPresent
  namespace: nebulaops

frontend:
  enabled: true
  image: nebulaops/frontend:local
  replicas: 1
  port: 80
  servicePort: 4200

services:
  gateway:
    image: nebulaops/gateway-service:local
    replicas: 1
    port: 8080
  auth:
    image: nebulaops/auth-service:local
    replicas: 1
    port: 8081
    mongoDatabase: nebula_auth
  task:
    image: nebulaops/task-service:local
    replicas: 1
    port: 8082
    mongoDatabase: nebula_task
  notification:
    image: nebulaops/notification-service:local
    replicas: 1
    port: 8083
    mongoDatabase: nebula_notification
  file:
    image: nebulaops/file-service:local
    replicas: 1
    port: 8084
    mongoDatabase: nebula_file

mongodb:
  enabled: true
  image: mongo:7
  username: nebula
  password: nebula
  port: 27017
  storage: 1Gi

observability:
  prometheus:
    enabled: true
    image: prom/prometheus:v2.53.0
    port: 9090
  grafana:
    enabled: true
    image: grafana/grafana:11.1.0
    port: 3000
    adminUser: admin
    adminPassword: admin

ingress:
  enabled: false
  className: nginx
  host: nebulaops.local

redis:
  enabled: true
  image: redis:7-alpine
  port: 6379

rabbitmq:
  enabled: true
  image: rabbitmq:3.13-management-alpine
  port: 5672
  managementPort: 15672
  username: guest
  password: guest

goServices:
  cache:
    image: nebulaops/go-cache-service:local
    replicas: 1
```

14. Terraform e Smart Terraform Studio

Il repository contiene due zone Terraform: terraform/ e infrastructure/terraform/. La UI Terraform Studio punta a endpoint /api/terraform-studio/graph, /plan e /modules. Gli script terraform/plan-local.sh e apply-local.sh entrano in terraform/ ma citano examples/local-kind/terraform.tfvars; nel pacchetto reale il tfvars disponibile e sotto infrastructure/terraform/examples/local-kind. Questa incongruenza richiede normalizzazione del path prima di una demo live.

14.x terraform/main.tf

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    local = { source = "hashicorp/local", version = "~> 2.5" }
    null  = { source = "hashicorp/null", version = "~> 3.2" }
  }
}

provider "local" {}
provider "null" {}

module "kind" {
  source       = "./modules/kind"
  cluster_name = var.cluster_name
  namespace    = var.namespace
}

module "observability" {
  source    = "./modules/observability"
  namespace = var.namespace
}

module "gitops" {
  source      = "./modules/gitops"
  namespace   = var.namespace
  argocd_path = "../infrastructure/argocd/application.yaml"
}

module "security" {
  source    = "./modules/security"
  namespace = var.namespace
}

module "apps" {
  source    = "./modules/apps"
  namespace = var.namespace
}
```

14.x terraform/variables.tf

```
variable "cluster_name" { type = string, default = "nebulaops-v18" }
variable "namespace" { type = string, default = "nebulaops" }
variable "target" { type = string, default = "kind", description = "Logical demo target used by
  docs and FE." }
```

14.x terraform/outputs.tf

```
output "cluster_name" { value = var.cluster_name }
output "namespace" { value = var.namespace }
output "kind_manifest" { value = module.kind.kind_config_path }
output "generated_values" { value = module.apps.values_path }
output "next_steps" { value = "./scripts/terraform/apply-local.sh && ./scripts/helm-install-
  local.sh" }
```

14.x infrastructure/terraform/main.tf

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    local = { source = "hashicorp/local", version = "~> 2.5" }
    null  = { source = "hashicorp/null", version = "~> 3.2" }
  }
}

provider "local" {}
provider "null" {}

module "kind" {
  source       = "./modules/kind"
  cluster_name = var.cluster_name
  namespace    = var.namespace
}

module "observability" {
  source    = "./modules/observability"
  namespace = var.namespace
}

module "gitops" {
  source      = "./modules/gitops"
  namespace   = var.namespace
  argocd_path = "../infrastructure/argocd/application.yaml"
}

module "security" {
  source    = "./modules/security"
  namespace = var.namespace
}

module "apps" {
  source    = "./modules/apps"
  namespace = var.namespace
}
```

14.x infrastructure/terraform/variables.tf

```
variable "cluster_name" { type = string, default = "nebulaops-v18" }
variable "namespace" { type = string, default = "nebulaops" }
variable "target" { type = string, default = "kind", description = "Logical demo target used by
  docs and FE." }
```

14.x infrastructure/terraform/examples/local-kind/terraform.tfvars

```
cluster_name = "nebulaops-v18"
namespace    = "nebulaops"
target       = "kind"
```

15. Tasks, RabbitMQ e notifiche

La sezione Tasks e una delle piu reali del pacchetto: il task-service salva su MongoDB e pubblica eventi su RabbitMQ. Questo permette di dimostrare CRUD, persistence e messaging senza dover deployare un app esterna.

Metodo	Path	File
GET	/api/tasks	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/ProxyController.java
POST	/api/tasks	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/ProxyController.java
PUT	/api/tasks/{id}	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/ProxyController.java
DELETE	/api/tasks/{id}	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/ProxyController.java
PATCH	/api/tasks/{id}/status/{status}	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/ProxyController.java
GET	/api/notifications/live	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/ProxyController.java
PATCH	/api/notifications/{id}/read	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/ProxyController.java
GET	/api/tasks/{id}	backend/task-service/src/main/java/dev/nebulaops/task/api/TaskController.java
PUT	/api/tasks/{id}	backend/task-service/src/main/java/dev/nebulaops/task/api/TaskController.java
DELETE	/api/tasks/{id}	backend/task-service/src/main/java/dev/nebulaops/task/api/TaskController.java
PATCH	/api/tasks/{id}/status/{status}	backend/task-service/src/main/java/dev/nebulaops/task/api/TaskController.java

15.x backend/task-service/src/main/java/dev/nebulaops/task/api/TaskController.java

```
package dev.nebulaops.task.api;

import dev.nebulaops.task.domain.*;
import dev.nebulaops.task.repo.TaskRepository;

import java.time.Instant;
import java.util.*;

import dev.nebulaops.task.config.RabbitMqConfig;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/tasks")
public class TaskController {
    private final TaskRepository repo;
    private final RabbitTemplate rabbit;

    public TaskController(TaskRepository repo, RabbitTemplate rabbit) {
        this.repo = repo;
        this.rabbit = rabbit;
    }

    @GetMapping
    public List<TaskDocument> list(@RequestParam(defaultValue = "default-org") String
      organizationId) {
        return repo.findByOrganizationId(organizationId);
    }

    @PostMapping
    public TaskDocument create(@RequestBody CreateTaskRequest r) {
        if (r.title() == null || r.title().isBlank()) {
            throw new IllegalArgumentException("title is required");
        }
        var now = Instant.now();
        var organizationId = r.organizationId() == null || r.organizationId().isBlank() ?
          "default-org" : r.organizationId();
        var saved = repo.save(new TaskDocument(
                null,
                organizationId,
                r.projectId() == null ? "portfolio" : r.projectId(),
                r.title(),
                r.description(),
                TaskStatus.TODO,
                r.priority() == null ? TaskPriority.MEDIUM : r.priority(),
                r.assigneeId(),
                r.labels() == null ? List.of() : r.labels(),
                r.dueAt(),
                now,
                now));

        publish("TaskCreated", saved);
        return saved;
    }


    @GetMapping("/{id}")
    public TaskDocument get(@PathVariable String id) {
        return repo.findById(id).orElseThrow();
    }

    @PutMapping("/{id}")
    public TaskDocument update(@PathVariable String id, @RequestBody UpdateTaskRequest r) {
        var existing = repo.findById(id).orElseThrow();
        var updated = new TaskDocument(
                existing.id(),
                existing.organizationId(),
                r.projectId() == null ? existing.projectId() : r.projectId(),
                r.title() == null || r.title().isBlank() ? existing.title() : r.title(),
                r.description() == null ? existing.description() : r.description(),
                r.status() == null ? existing.status() : r.status(),
                r.priority() == null ? existing.priority() : r.priority(),
                r.assigneeId() == null ? existing.assigneeId() : r.assigneeId(),
                r.labels() == null ? existing.labels() : r.labels(),
                r.dueAt() == null ? existing.dueAt() : r.dueAt(),
                existing.createdAt(),
                Instant.now());
        var saved = repo.save(updated);
        publish("TaskUpdated", saved);
        return saved;
    }
```

15.x backend/task-service/src/main/java/dev/nebulaops/task/config/RabbitMqConfig.java

```
package dev.nebulaops.task.config;

import org.springframework.amqp.core.Binding;
import org.springframework.amqp.core.BindingBuilder;
import org.springframework.amqp.core.DirectExchange;
import org.springframework.amqp.core.Queue;
import org.springframework.amqp.rabbit.connection.ConnectionFactory;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.amqp.support.converter.Jackson2JsonMessageConverter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RabbitMqConfig {
    public static final String TASK_EXCHANGE = "nebula.task.exchange";
    public static final String TASK_EVENTS_QUEUE = "nebula.task.events";
    public static final String TASK_ROUTING_KEY = "task.event";

    @Bean
    DirectExchange taskExchange() {
        return new DirectExchange(TASK_EXCHANGE, true, false);
    }

    @Bean
    Queue taskEventsQueue() {
        return new Queue(TASK_EVENTS_QUEUE, true);
    }

    @Bean
    Binding taskEventsBinding(Queue taskEventsQueue, DirectExchange taskExchange) {
        return BindingBuilder.bind(taskEventsQueue).to(taskExchange).with(TASK_ROUTING_KEY);
    }

    @Bean
    Jackson2JsonMessageConverter jsonMessageConverter() {
        return new Jackson2JsonMessageConverter();
    }

    @Bean
    RabbitTemplate rabbitTemplate(ConnectionFactory connectionFactory,
      Jackson2JsonMessageConverter converter) {
        RabbitTemplate template = new RabbitTemplate(connectionFactory);
        template.setMessageConverter(converter);
        return template;
    }
}
```

15.x backend/notification-service/src/main/java/dev/nebulaops/notification/api/NotificationController.java

```
package dev.nebulaops.notification.api;

import java.time.Instant;
import java.util.Map;
import java.util.concurrent.CopyOnWriteArrayList;

import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/notifications")
public class NotificationController {
    private final CopyOnWriteArrayList<Map<String, Object>> inbox = new
      CopyOnWriteArrayList<>();

    @RabbitListener(queues = "nebula.task.events")
    public void onTaskEvent(Map<String, Object> event) {
        inbox.add(Map.of("type", "TASK_EVENT", "payload", event, "receivedAt",
          Instant.now().toString()));
    }

    @GetMapping
    public Object list() {
        return inbox;
    }
}
```

16. AI OPS e RCA

AI OPS usa ai-ops-service come wrapper e ai-engine FastAPI come componente Python. In app.component.ts runAiOpsAnalysis gestisce fallback locale con messaggi chiari quando backend o AI engine non rispondono. Il valore della sezione in demo e mostrare diagnosi, blast radius, suggerimento e YAML patch, non promettere auto-remediation non governata.

Metodo	Path	File
GET	/api/aiops/diagnose	backend/ai-ops-service/src/main/java/dev/nebulaops/aiops/AiOpsController.java
GET	/api/ai-ops/diagnose	backend/ai-ops-service/src/main/java/dev/nebulaops/aiops/AiOpsController.java
POST	/api/aiops/analyze	backend/ai-ops-service/src/main/java/dev/nebulaops/aiops/AiOpsController.java
POST	/api/ai-ops/analyze	backend/ai-ops-service/src/main/java/dev/nebulaops/aiops/AiOpsController.java
POST	/api/aiops/autofix	backend/ai-ops-service/src/main/java/dev/nebulaops/aiops/AiOpsController.java
POST	/api/ai-ops/autofix	backend/ai-ops-service/src/main/java/dev/nebulaops/aiops/AiOpsController.java
GET	/api/aiops/logs	backend/ai-ops-service/src/main/java/dev/nebulaops/aiops/AiOpsController.java
GET	/api/ai-ops/logs	backend/ai-ops-service/src/main/java/dev/nebulaops/aiops/AiOpsController.java
POST	/api/aiops/actions/{action}	backend/ai-ops-service/src/main/java/dev/nebulaops/aiops/AiOpsController.java
POST	/api/ai-ops/actions/{action}	backend/ai-ops-service/src/main/java/dev/nebulaops/aiops/AiOpsController.java
POST	/api/runtime/docker/containers/{id}/start	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/DockerActionsController.java
POST	/api/runtime/docker/containers/{id}/stop	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/DockerActionsController.java
POST	/api/runtime/docker/containers/{id}/restart	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/DockerActionsController.java
POST	/api/runtime/docker/containers/{id}/pause	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/DockerActionsController.java
POST	/api/runtime/docker/containers/{id}/unpause	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/DockerActionsController.java
DELETE	/api/runtime/docker/containers/{id}	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/DockerActionsController.java
GET	/api/runtime/docker/containers/{id}/logs	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/DockerActionsController.java
GET	/api/runtime/docker/containers/{id}/stats	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/DockerActionsController.java
POST	/api/kubernetes/nodes/{name}/drain	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/KubernetesOpsController.java
GET	/api/platform/docker/containers	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/PlatformLiveController.java
POST	/api/ai-ops/analyze	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/ProxyController.java
POST	/api/ai-ops/autofix	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/ProxyController.java
GET	/api/runtime/docker/containers	backend/gateway-service/src/main/java/dev/nebulaops/gateway/api/RuntimeOpsController.java

16.x ai-engine/app/main.py

```
from datetime import datetime
from typing import Any
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="NebulaOps AI Engine", version="19.1")

class AnalyzeRequest(BaseModel):
    prompt: str = ""
    logs: list[dict[str, Any]] = []
    resources: list[dict[str, Any]] = []

def has(text: str, word: str) -> bool:
    return word.lower() in text.lower()

@app.get("/health")
def health():
    return {"status": "ok", "engine": "nebulaops-ai-engine", "version": "19.1"}

@app.post("/analyze")
def analyze(req: AnalyzeRequest):
    text = " ".join([req.prompt] + [str(x) for x in req.logs] + [str(x) for x in req.resources])
    root = "Readiness probe timeout and rollout instability."
    fix = "Patch readiness probe, verify rollout and restart gateway-service."
    if has(text, "image") or has(text, "ImagePullBackOff"):
        root = "Image tag, registry credential or pull policy mismatch."
        fix = "Verify image tag, imagePullSecrets and Helm values, then restart rollout."
    if has(text, "oom") or has(text, "OOMKilled"):
        root = "Container memory limit too low or memory leak after deploy."
        fix = "Raise memory limits, inspect heap profile and scale replicas temporarily."
    if has(text, "crash") or has(text, "CrashLoopBackOff"):
        severity = "CRITICAL"
        health = 18
    else:
        severity = "HIGH"
        health = 46
    now = datetime.utcnow().strftime("%H:%M:%S")
    return {
        "incidentId": "AIOPS-19-1-" + datetime.utcnow().strftime("%H%M%S"),
        "summary": "AI engine correlated logs, Kubernetes events and service dependencies into
          an incident summary.",
        "rootCause": root,
        "confidence": 0.92,
        "blastRadius": ["frontend", "gateway-service", "task-service", "notification-service"],
        "fix": fix,
        "yaml": "apiVersion: apps/v1\nkind: Deployment\nmetadata:\n  name: gateway-service\n
          namespace: nebulaops\nspec:\n  template:\n    spec:\n      containers:\n        -
          name: gateway-service\n          readinessProbe:\n            httpGet:\n
          path: /actuator/health\n              port: 8080\n            initialDelaySeconds:
          25\n            periodSeconds: 10\n",
        "events": [
            {"time": now, "service": "gateway-service", "severity": severity, "title":
              "CrashLoopBackOff / readiness failure", "status": "active", "recommendation":
              fix},
            {"time": now, "service": "frontend", "severity": "HIGH", "title": "5xx propagation",
              "status": "degraded", "recommendation": "Send traffic to healthy gateway
              replicas"},
            {"time": now, "service": "task-service", "severity": "MEDIUM", "title": "Queue
              latency spike", "status": "watch", "recommendation": "Scale workers if queue depth
              grows"},
            {"time": now, "service": "mongodb", "severity": "LOW", "title": "No storage
              anomaly", "status": "stable", "recommendation": "No action"}
        ],
        "nodes": [
            {"id": "frontend", "label": "Frontend", "type": "edge", "health": 72, "x": 14, "y":
              24, "z": 1, "status": "warn"},
            {"id": "gateway", "label": "Gateway", "type": "api", "health": health, "x": 42, "y":
              38, "z": 4, "status": "critical"},
            {"id": "tasks", "label": "Tasks", "type": "svc", "health": 66, "x": 69, "y": 22,
              "z": 2, "status": "warn"},
            {"id": "mongo", "label": "MongoDB", "type": "db", "health": 96, "x": 78, "y": 66,
              "z": 1, "status": "ok"},
            {"id": "notify", "label": "Notify", "type": "svc", "health": 61, "x": 30, "y": 72,
              "z": 3, "status": "warn"}
        ]
    }
```

16.x backend/ai-ops-service/src/main/java/dev/nebulaops/aiops/AiOpsController.java

```
package dev.nebulaops.aiops;

import dev.nebulaops.aiops.service.AiOpsService;
import org.springframework.web.bind.annotation.*;

import java.util.Map;

@RestController
@RequestMapping({"/api/aiops", "/api/ai-ops"})
public class AiOpsController {
    private final AiOpsService service;

    public AiOpsController(AiOpsService service) {
        this.service = service;
    }

    @GetMapping("/diagnose")
    public Map<String, Object> diagnose(@RequestParam(required = false) String namespace) {
        return service.diagnose(namespace);
    }

    @PostMapping("/analyze")
    public Map<String, Object> analyze(@RequestBody(required = false) Map<String, Object> body)
      {
        return service.analyze(body == null ? Map.of() : body);
    }

    @PostMapping("/autofix")
    public Map<String, Object> autofix(@RequestBody(required = false) Map<String, Object> body)
      {
        return service.autofix(body == null ? Map.of() : body);
    }

    @GetMapping("/logs")
    public Map<String, Object> logs(@RequestParam(required = false) String selector,
      @RequestParam(defaultValue = "default") String namespace) {
        return service.logs(selector, namespace);
    }

    @PostMapping("/actions/{action}")
    public Map<String, Object> action(@PathVariable String action, @RequestParam String
      deployment, @RequestParam(defaultValue = "default") String namespace) {
        return service.action(namespace, deployment, action);
    }
}
```

17. Applicazione demo Spring Boot v22.1

La demo e progettata per essere osservata sia nel modulo Docker Desktop sia nel modulo OpenLens/Kubernetes. Include Actuator, Prometheus metrics, Dockerfile multi-stage, Docker Compose, manifest Kubernetes, Helm chart, ArgoCD Application e GitLab CI/CD.

17.x demo/README.md

```
# NebulaOps Spring Demo

Applicazione demo Spring Boot 3.3 / Java 21 pensata per essere installata, deployata e osservata
  tramite .

## Cosa contiene

- API REST `/api/demo/hello`, `/api/demo/orders`, `/api/demo/load/{items}`
- Actuator: `/actuator/health`, `/actuator/prometheus`, readiness/liveness probes
- Dockerfile multi-stage
- Docker Compose collegato alla rete `nebulaops-network`
- Manifest Kubernetes per namespace, deployment, service, HPA e ingress
- Helm chart
- ArgoCD Application
- GitLab CI/CD per test, build image, render Helm, sync ArgoCD e verify

## Avvio Docker locale

```bash
./scripts/build-local.sh
./scripts/run-docker.sh
```

Apri NebulaOps > Docker Desktop e cerca il container `nebulaops-spring-demo`.

Deploy Kubernetes locale

```bash
./scripts/deploy-k8s-local.sh
./scripts/port-forward.sh
```

Poi in un altro terminale:

```bash
./scripts/smoke-test.sh
```

Apri NebulaOps > OpenLens/Kubernetes e verifica namespace `nebulaops-demo`, deployment
  `nebulaops-spring-demo`, pod, service e HPA.

CI/CD GitLab

1. Crea un nuovo repository GitLab.
2. Carica questo progetto.
3. Configura le variabili CI/CD: `ARGOCD_SERVER`, `ARGOCD_AUTH_TOKEN`, `DEMO_BASE_URL`.
4. Modifica `argocd/application.yaml` con il tuo `repoURL`.
5. Crea/sincronizza l'app in ArgoCD.
6. Esegui la pipeline: test -> build -> package -> deploy -> verify.

Endpoint principali

```bash
curl http://localhost:18080/actuator/health
curl http://localhost:18080/api/demo/hello
curl -H 'Content-Type: application/json' -d '{"product":"test","quantity":1}'
  http://localhost:18080/api/demo/orders
curl http://localhost:18080/actuator/prometheus
```
```

17.x demo/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
  instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
           https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.5</version>
    <relativePath/>
  </parent>
  <groupId>dev.nebulaops.demo</groupId>
  <artifactId>nebulaops-spring-demo</artifactId>
  <version>      </version>
  <name>nebulaops-spring-demo</name>
  <description>Applicazione Spring Boot demo da monitorare e gestire con
          </description>
  <properties>
    <java.version>21</java.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>
    <dependency>
      <groupId>io.micrometer</groupId>
      <artifactId>micrometer-registry-prometheus</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

17.x demo/Dockerfile

```
# Multi-stage Dockerfile per NebulaOps Spring Demo
FROM maven:3.9.8-eclipse-temurin-21 AS build
WORKDIR /workspace
COPY pom.xml .
RUN mvn -B -ntp dependency:go-offline
COPY src ./src
RUN mvn -B -ntp clean package -DskipTests

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
RUN addgroup -S spring && adduser -S spring -G spring
COPY --from=build /workspace/target/nebulaops-spring-demo-      .jar /app/app.jar
USER spring:spring
EXPOSE 8080
HEALTHCHECK --interval=20s --timeout=3s --retries=10 CMD wget -qO-
  http://127.0.0.1:8080/actuator/health/readiness || exit 1
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "/app/app.jar"]
```

17.x demo/docker-compose.yml

```
services:
  nebulaops-spring-demo:
    build: .
    image: nebulaops-spring-demo:
    container_name: nebulaops-spring-demo
    environment:
      APP_ENVIRONMENT: docker-compose
      SERVER_PORT: 8080
    ports:
      - "18080:8080"
    networks:
      - nebulaops-network
    labels:
      dev.nebulaops.demo: "true"
      dev.nebulaops.version: "
    healthcheck:
      test: ["CMD", "wget", "-qO-", "http://127.0.0.1:8080/actuator/health/readiness"]
      interval: 20s
      timeout: 3s
      retries: 10
networks:
  nebulaops-network:
    external: true
```

17.x demo/src/main/java/dev/nebulaops/demo/api/DemoController.java

```
package dev.nebulaops.demo.api;

import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
import jakarta.validation.Valid;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.UUID;
import java.util.concurrent.CopyOnWriteArrayList;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/demo")
public class DemoController {
  private final List<OrderDto> orders = new CopyOnWriteArrayList<>();
  private final Counter requestCounter;

  @Value("${spring.application.name:nebulaops-spring-demo}")
  private String applicationName;

  @Value("${app.environment:local}")
  private String environment;

  public DemoController(MeterRegistry meterRegistry) {
    this.requestCounter = Counter.builder("nebulaops_demo_requests_total")
        .description("Numero totale di richieste gestite dalla demo NebulaOps")
        .register(meterRegistry);
    orders.add(new OrderDto(UUID.randomUUID().toString(), "starter-plan", 1,
      Instant.now().toString()));
  }

  @GetMapping("/hello")
  public Map<String, Object> hello() {
    requestCounter.increment();
    return Map.of(
        "message", "NebulaOps Spring Demo is alive",
        "application", applicationName,
        "environment", environment,
        "timestamp", Instant.now().toString()
    );
  }

  @GetMapping("/orders")
  public List<OrderDto> orders() {
    requestCounter.increment();
    return new ArrayList<>(orders);
  }

  @PostMapping("/orders")
  public ResponseEntity<OrderDto> createOrder(@Valid @RequestBody CreateOrderRequest request) {
    requestCounter.increment();
    OrderDto created = new OrderDto(UUID.randomUUID().toString(), request.product(),
      request.quantity(), Instant.now().toString());
    orders.add(created);
    return ResponseEntity.status(201).body(created);
  }

  @GetMapping("/load/{items}")
  public Map<String, Object> syntheticLoad(@PathVariable @Min(1) int items) {
    requestCounter.increment();
    long checksum = 0;
    int max = Math.min(items, 200_000);
    for (int i = 1; i <= max; i++) {
      checksum += (long) i * 31L;
    }
    return Map.of("items", max, "checksum", checksum, "timestamp", Instant.now().toString());
  }

  public record CreateOrderRequest(@NotBlank String product, @Min(1) int quantity) {}
  public record OrderDto(String id, String product, int quantity, String createdAt) {}
}
```

17.x demo/src/main/resources/application.yml

```
spring:
  application:
    name: nebulaops-spring-demo
server:
  port: ${SERVER_PORT:8080}
app:
  environment: ${APP_ENVIRONMENT:local}
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus,env
  endpoint:
    health:
      probes:
        enabled: true
      show-details: always
  info:
    env:
      enabled: true
info:
  app:
    name: nebulaops-spring-demo
    version:
    purpose: Demo applicativa per Docker Desktop, OpenLens/Kubernetes, Prometheus e CI/CD
      NebulaOps
logging:
  level:
    root: INFO
    dev.nebulaops.demo: DEBUG
```

17.x demo/.gitlab-ci.yml

```
stages:
  - test
  - build
  - package
  - deploy
  - verify

variables:
  DOCKER_TLS_CERTDIR: ""
  DOCKER_DRIVER: overlay2
  IMAGE_TAG: "$CI_COMMIT_SHORT_SHA"
  IMAGE_NAME: "$CI_REGISTRY_IMAGE/nebulaops-spring-demo"
  HELM_CHART_DIR: helm/nebulaops-spring-demo

test:maven:
  stage: test
  image: maven:3.9.8-eclipse-temurin-21
  script:
    - mvn -B -ntp test
  artifacts:
    when: always
    paths:
      - target/surefire-reports/
    expire_in: 1 week

build:image:
  stage: build
  image: docker:27
  services:
    - name: docker:27-dind
      command: ["--tls=false"]
  before_script:
    - echo "$CI_REGISTRY_PASSWORD" | docker login "$CI_REGISTRY" -u "$CI_REGISTRY_USER"
      --password-stdin
  script:
    - docker build -t "$IMAGE_NAME:$IMAGE_TAG" -t "$IMAGE_NAME:latest" .
    - docker push "$IMAGE_NAME:$IMAGE_TAG"
    - docker push "$IMAGE_NAME:latest"
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH

package:helm:
  stage: package
  image: alpine/helm:3.15.4
  script:
    - helm lint "$HELM_CHART_DIR"
    - helm template nebulaops-spring-demo "$HELM_CHART_DIR" --namespace nebulaops-demo --set
      image.repository="$IMAGE_NAME" --set image.tag="$IMAGE_TAG" > rendered-demo.yaml
  artifacts:
    paths:
      - rendered-demo.yaml
    expire_in: 1 week

deploy:argocd:
  stage: deploy
  image: argoproj/argocd:v2.12.3
  script:
    - argocd login "$ARGOCD_SERVER" --auth-token "$ARGOCD_AUTH_TOKEN" --grpc-web
    - argocd app set nebulaops-spring-demo -p image.repository="$IMAGE_NAME" -p
      image.tag="$IMAGE_TAG"
    - argocd app sync nebulaops-spring-demo --prune --timeout 300
    - argocd app wait nebulaops-spring-demo --health --sync --timeout 300
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH && $ARGOCD_SERVER && $ARGOCD_AUTH_TOKEN
      when: manual

verify:http:
  stage: verify
  image: curlimages/curl:8.9.1
  script:
    - curl -fsS "$DEMO_BASE_URL/actuator/health"
    - curl -fsS "$DEMO_BASE_URL/api/demo/hello"
  rules:
    - if: $DEMO_BASE_URL
      when: manual
```

17.x demo/argocd/application.yaml

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: nebulaops-spring-demo
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://gitlab.example.com/your-group/nebulaops-spring-demo.git
    targetRevision: main
    path: helm/nebulaops-spring-demo
    helm:
      valueFiles:
        - values.yaml
  destination:
    server: https://kubernetes.default.svc
    namespace: nebulaops-demo
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

17.x demo/k8s/02-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nebulaops-spring-demo
  namespace: nebulaops-demo
  labels:
    app: nebulaops-spring-demo
    app.kubernetes.io/name: nebulaops-spring-demo
    app.kubernetes.io/part-of: nebulaops
spec:
  replicas: 2
  revisionHistoryLimit: 5
  selector:
    matchLabels:
      app: nebulaops-spring-demo
  template:
    metadata:
      labels:
        app: nebulaops-spring-demo
        app.kubernetes.io/name: nebulaops-spring-demo
        app.kubernetes.io/part-of: nebulaops
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "8080"
        prometheus.io/path: /actuator/prometheus
    spec:
      containers:
        - name: nebulaops-spring-demo
          image: nebulaops-spring-demo:
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 8080
          envFrom:
            - configMapRef:
                name: nebulaops-spring-demo-config
          resources:
            requests:
              cpu: 100m
              memory: 256Mi
            limits:
              cpu: 500m
              memory: 512Mi
          readinessProbe:
            httpGet:
              path: /actuator/health/readiness
              port: http
            initialDelaySeconds: 15
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /actuator/health/liveness
              port: http
            initialDelaySeconds: 40
            periodSeconds: 20
```

17.x demo/helm/nebulaops-spring-demo/values.yaml

```
replicaCount: 2
image:
  repository: nebulaops-spring-demo
  tag: "      "
  pullPolicy: IfNotPresent
service:
  type: ClusterIP
  port: 8080
ingress:
  enabled: false
  className: nginx
  host: demo.nebulaops.local
resources:
  requests:
    cpu: 100m
    memory: 256Mi
  limits:
    cpu: 500m
    memory: 512Mi
env:
  APP_ENVIRONMENT: helm
  SERVER_PORT: "8080"
```

18. Deploy demo in Docker Desktop

Questa procedura prova la parte Docker senza Kubernetes. L applicazione entra nella stessa rete esterna nebulaops-network e compare nella UI Containers di NebulaOps.

```
cd nebulaops-spring-demo-
./scripts/build-local.sh
./scripts/run-docker.sh
./scripts/smoke-test.sh

# Verifica da NebulaOps
curl http://localhost:8080/api/runtime/docker/containers | jq '.[] | select(.name|test("spring-
  demo"))'
curl http://localhost:18080/api/demo/hello
curl http://localhost:18080/actuator/prometheus | grep nebulaops_demo_requests_total
```

18.x demo/scripts/build-local.sh

```
#!/usr/bin/env bash
set -euo pipefail
IMAGE=${IMAGE:-nebulaops-spring-demo:      }
mvn -B -ntp clean test package
docker build -t "$IMAGE" .
echo "Built $IMAGE"
```

18.x demo/scripts/run-docker.sh

```
#!/usr/bin/env bash
set -euo pipefail
if ! docker network inspect nebulaops-network >/dev/null 2>&1; then
  docker network create nebulaops-network
fi
docker compose up --build -d
curl -fsS http://localhost:18080/actuator/health
curl -fsS http://localhost:18080/api/demo/hello
cat <<INFO
Demo Docker container ready.
NebulaOps Docker Desktop tab should show container: nebulaops-spring-demo
Try logs: docker logs -f nebulaops-spring-demo
INFO
```

18.x demo/scripts/smoke-test.sh

```
#!/usr/bin/env bash
set -euo pipefail
BASE=${BASE:-http://localhost:18080}
curl -fsS "$BASE/actuator/health" | sed 's/.*/health OK/'
curl -fsS "$BASE/api/demo/hello"; echo
curl -fsS -H 'Content-Type: application/json' -d '{"product":"nebulaops-license","quantity":2}'
  "$BASE/api/demo/orders"; echo
curl -fsS "$BASE/api/demo/orders"; echo
curl -fsS "$BASE/actuator/prometheus" | grep -E
  '^nebulaops_demo_requests_total|^http_server_requests_seconds_count' | head
```

18.1 Cosa guardare nel modulo Docker Desktop

Elemento UI	Cosa deve mostrare	Conferma CLI
Containers	nebulaops-spring-demo running/healthy	docker ps --filter name=nebulaops-spring-demo
Logs	startup Spring Boot e richieste /api/demo	docker logs -f nebulaops-spring-demo
Stats	CPU/RAM del container	docker stats nebulaops-spring-demo
Restart	container ricreato e health torna UP	curl :18080/actuator/health

19. Deploy demo in Kubernetes/OpenLens

Questa procedura prova la parte OpenLens-like. Il punto critico e l immagine locale: su kind bisogna caricare nebulaops-spring-demo: nel cluster oppure usare un registry raggiungibile.

```
cd nebulaops-spring-demo-
./scripts/deploy-k8s-local.sh
./scripts/port-forward.sh
# in un altro terminale
BASE=http://localhost:18080 ./scripts/smoke-test.sh

kubectl -n nebulaops-demo get deploy,pods,svc,hpa
kubectl -n nebulaops-demo logs deploy/nebulaops-spring-demo --tail=80
kubectl -n nebulaops-demo describe deploy nebulaops-spring-demo
```

19.x demo/scripts/deploy-k8s-local.sh

```
#!/usr/bin/env bash
set -euo pipefail
IMAGE=${IMAGE:-nebulaops-spring-demo:      }
NAMESPACE=${NAMESPACE:-nebulaops-demo}
KIND_CLUSTER=${KIND_CLUSTER:-                }

mvn -B -ntp clean package -DskipTests
docker build -t "$IMAGE" .
if command -v kind >/dev/null 2>&1 && kind get clusters | grep -qx "$KIND_CLUSTER"; then
  kind load docker-image "$IMAGE" --name "$KIND_CLUSTER"
else
  echo "WARN: kind cluster '$KIND_CLUSTER' not found. Continuing with
    imagePullPolicy=IfNotPresent."
fi
kubectl apply -f k8s/
kubectl -n "$NAMESPACE" rollout status deploy/nebulaops-spring-demo --timeout=180s
kubectl -n "$NAMESPACE" get pods,svc,hpa -l app=nebulaops-spring-demo
cat <<INFO
Demo Kubernetes app deployed.
Port-forward: ./scripts/port-forward.sh
NebulaOps OpenLens tab should show namespace: $NAMESPACE and deployment/pods: nebulaops-spring-
  demo
INFO
```

19.x demo/scripts/deploy-helm-local.sh

```
#!/usr/bin/env bash
set -euo pipefail
IMAGE_REPOSITORY=${IMAGE_REPOSITORY:-nebulaops-spring-demo}
IMAGE_TAG=${IMAGE_TAG:-      }
helm upgrade --install nebulaops-spring-demo helm/nebulaops-spring-demo \
  --namespace nebulaops-demo --create-namespace \
  --set image.repository="$IMAGE_REPOSITORY" \
  --set image.tag="$IMAGE_TAG" \
  --set image.pullPolicy=IfNotPresent
kubectl -n nebulaops-demo rollout status deploy/nebulaops-spring-demo --timeout=180s
kubectl -n nebulaops-demo get pods,svc,hpa
```

19.x demo/scripts/port-forward.sh

```
#!/usr/bin/env bash
set -euo pipefail
kubectl -n nebulaops-demo port-forward svc/nebulaops-spring-demo 18080:8080
```

19.x demo/scripts/cleanup.sh

```
#!/usr/bin/env bash
set -euo pipefail
docker compose down --remove-orphans 2>/dev/null || true
kubectl delete namespace nebulaops-demo --ignore-not-found=true
```

19.x demo/k8s/00-namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: nebulaops-demo
  labels:
    app.kubernetes.io/part-of: nebulaops
    app.kubernetes.io/name: nebulaops-spring-demo
```

19.x demo/k8s/03-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: nebulaops-spring-demo
  namespace: nebulaops-demo
  labels:
    app: nebulaops-spring-demo
spec:
  type: ClusterIP
  selector:
    app: nebulaops-spring-demo
  ports:
    - name: http
      port: 8080
      targetPort: http
```

19.x demo/k8s/04-hpa.yaml

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: nebulaops-spring-demo
  namespace: nebulaops-demo
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nebulaops-spring-demo
  minReplicas: 1
  maxReplicas: 5
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

19.x demo/k8s/05-ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: nebulaops-spring-demo
  namespace: nebulaops-demo
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
    - host: demo.nebulaops.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: nebulaops-spring-demo
                port:
                  number: 8080
```

19.1 Cosa guardare nel modulo OpenLens

Sezione	Risorsa attesa	Azione demo
Namespaces	nebulaops-demo	describe namespace
Deployments	nebulaops-spring-demo replicas=2	scale +1 e rollout restart
Pods	2/2 o 3/3 running	logs e describe pod
Services	ClusterIP nebulaops-spring-demo:8080	mostrare selector app=nebulaops-spring-demo
HPA	min 1 max 5 target CPU 70%	spiegare autoscaling e metric-server requirement

20. Attivazione CI/CD e GitOps sulla demo

La pipeline GitLab della demo e completa ma richiede registry e ArgoCD reali. L esecuzione end-to-end richiede variabili CI, repoURL corretto, token ArgoCD e immagine raggiungibile dal cluster.

20.1 Variabili GitLab richieste

Variabile	Esempio	Uso
CI_REGISTRY_IMAGE	registry.gitlab.com/group/nebulaops-spring-demo	Base image repository
ARGOCD_SERVER	argocd.example.local o localhost tunnel	Login argocd CLI
ARGOCD_AUTH_TOKEN	token account ArgoCD	argocd app set/sync/wait
DEMO_BASE_URL	http://demo.nebulaops.local o port-forward	verify:http stage

20.2 Comandi GitOps

```
# Nel repository demo, modificare argocd/application.yaml
# source.repoURL: repository GitLab reale
kubectl create namespace argocd --dry-run=client -o yaml | kubectl apply -f -
kubectl apply -f argocd/application.yaml
kubectl -n argocd get application nebulaops-spring-demo -o wide
# Se si usa argocd CLI:
argocd app sync nebulaops-spring-demo --prune
argocd app wait nebulaops-spring-demo --health --sync --timeout 300
```

21. API reference da controller

La tabella e generata dai controller Java presenti in backend/*/src/main/java. Non include endpoint inventati. Per leggibilita e divisa per servizio.

21.x ai-ops-service

Metodo	Path	File/Metodo
GET	/api/aiops/diagnose	AiOpsController.java
GET	/api/ai-ops/diagnose	AiOpsController.java
POST	/api/aiops/analyze	AiOpsController.java
POST	/api/ai-ops/analyze	AiOpsController.java
POST	/api/aiops/autofix	AiOpsController.java
POST	/api/ai-ops/autofix	AiOpsController.java
GET	/api/aiops/logs	AiOpsController.java
GET	/api/ai-ops/logs	AiOpsController.java
POST	/api/aiops/actions/{action}	AiOpsController.java
POST	/api/ai-ops/actions/{action}	AiOpsController.java

21.x auth-service

Metodo	Path	File/Metodo
POST	/api/auth/register	AuthController.java
POST	/api/auth/login	AuthController.java
POST	/api/auth/refresh	AuthController.java
GET	/api/auth/me	AuthController.java
POST	/api/auth/logout	AuthController.java
GET	/api/auth/healthz	AuthController.java

21.x cost-analytics-service

Metodo	Path	File/Metodo
GET	/api/cost/summary	CostController.java
GET	/api/cost/breakdown	CostController.java
POST	/api/cost/entries	CostController.java

21.x devsecops-service

Metodo	Path	File/Metodo
GET	/api/devsecops/secrets	DevSecOpsController.java
GET	/api/devsecops/image	DevSecOpsController.java

21.x environment-manager-service

Metodo	Path	File/Metodo
GET	/api/environments/{namespace}	EnvironmentManagerServiceController.java
GET	/api/environments/{namespace}/pods	EnvironmentManagerServiceController.java

21.x gateway-service

Metodo	Path	File/Metodo
POST	/api/runtime/docker/containers/{id}/start	DockerActionsController.java
POST	/api/runtime/docker/containers/{id}/stop	DockerActionsController.java
POST	/api/runtime/docker/containers/{id}/restart	DockerActionsController.java
POST	/api/runtime/docker/containers/{id}/pause	DockerActionsController.java
POST	/api/runtime/docker/containers/{id}/unpause	DockerActionsController.java
DELETE	/api/runtime/docker/containers/{id}	DockerActionsController.java
GET	/api/runtime/docker/containers/{id}/logs	DockerActionsController.java
GET	/api/runtime/docker/containers/{id}/stats	DockerActionsController.java
DELETE	/api/runtime/docker/images/{id}	DockerActionsController.java
POST	/api/runtime/docker/images/prune	DockerActionsController.java
DELETE	/api/runtime/docker/volumes/{name}	DockerActionsController.java
POST	/api/runtime/docker/volumes/prune	DockerActionsController.java
POST	/api/runtime/docker/networks/prune	DockerActionsController.java
GET	/api/health	HealthController.java
GET	/api/ping	HealthController.java
GET	/api/kubernetes/snapshot	KubernetesOpsController.java
GET	/api/kubernetes/logs	KubernetesOpsController.java
GET	/api/kubernetes/health	KubernetesOpsController.java
GET	/api/kubernetes/nodes	KubernetesOpsController.java
GET	/api/kubernetes/namespaces	KubernetesOpsController.java
GET	/api/kubernetes/resources	KubernetesOpsController.java
GET	/api/kubernetes/resources/{resourceId}	KubernetesOpsController.java
GET	/api/kubernetes/kind/{kind}	KubernetesOpsController.java
DELETE	/api/kubernetes/pods/{ns}/{name}	KubernetesOpsController.java
POST	/api/kubernetes/pods/{ns}/{name}/restart	KubernetesOpsController.java
GET	/api/kubernetes/pods/{ns}/{name}/logs	KubernetesOpsController.java
GET	/api/kubernetes/pods/{ns}/{name}/describe	KubernetesOpsController.java
POST	/api/kubernetes/deployments/{ns}/{name}/scale	KubernetesOpsController.java
POST	/api/kubernetes/deployments/{ns}/{name}/restart	KubernetesOpsController.java
GET	/api/kubernetes/deployments/{ns}/{name}/yaml	KubernetesOpsController.java
POST	/api/kubernetes/deployments/{ns}/{name}/yaml	KubernetesOpsController.java
GET	/api/kubernetes/deployments/{ns}/{name}/describe	KubernetesOpsController.java
GET	/api/kubernetes/services/{ns}/{name}/yaml	KubernetesOpsController.java
POST	/api/kubernetes/services/{ns}/{name}/yaml	KubernetesOpsController.java
GET	/api/kubernetes/services/{ns}/{name}/describe	KubernetesOpsController.java
GET	/api/kubernetes/ingresses/{ns}/{name}/yaml	KubernetesOpsController.java
POST	/api/kubernetes/ingresses/{ns}/{name}/yaml	KubernetesOpsController.java
GET	/api/kubernetes/ingresses/{ns}/{name}/describe	KubernetesOpsController.java
POST	/api/kubernetes/statefulsets/{ns}/{name}/scale	KubernetesOpsController.java
POST	/api/kubernetes/statefulsets/{ns}/{name}/restart	KubernetesOpsController.java
GET	/api/kubernetes/configmaps/{ns}/{name}/yaml	KubernetesOpsController.java
POST	/api/kubernetes/configmaps/{ns}/{name}/yaml	KubernetesOpsController.java
POST	/api/kubernetes/nodes/{name}/cordon	KubernetesOpsController.java
POST	/api/kubernetes/nodes/{name}/uncordon	KubernetesOpsController.java
POST	/api/kubernetes/nodes/{name}/drain	KubernetesOpsController.java
GET	/api/kubernetes/nodes/{name}/describe	KubernetesOpsController.java
POST	/api/kubernetes/namespaces	KubernetesOpsController.java
GET	/api/kubernetes/namespaces/{name}/describe	KubernetesOpsController.java
POST	/api/kubernetes/apply	KubernetesOpsController.java
POST	/api/kubernetes/delete	KubernetesOpsController.java
GET	/api/platform/observability	PlatformLiveController.java
GET	/api/platform/observability/prometheus	PlatformLiveController.java
GET	/api/platform/observability/loki	PlatformLiveController.java
GET	/api/platform/devsecops	PlatformLiveController.java

Metodo	Path	File/Metodo
GET	/api/platform/devsecops/secrets	PlatformLiveController.java
GET	/api/platform/environments	PlatformLiveController.java
GET	/api/platform/k8s/cluster	PlatformLiveController.java
GET	/api/platform/k8s/nodes	PlatformLiveController.java
GET	/api/platform/k8s/{kind}	PlatformLiveController.java
GET	/api/platform/helm/releases	PlatformLiveController.java
GET	/api/platform/docker/containers	PlatformLiveController.java
GET	/api/platform/docker/images	PlatformLiveController.java
GET	/api/platform/docker/volumes	PlatformLiveController.java
GET	/api/platform/docker/networks	PlatformLiveController.java
GET	/api/platform/docker/builds	PlatformLiveController.java
GET	/api/platform/docker/scout	PlatformLiveController.java
GET	/api/platform/terraform/plan	PlatformLiveController.java
GET	/api/platform/terraform/graph	PlatformLiveController.java
GET	/api/platform/terraform/modules	PlatformLiveController.java
POST	/api/auth/login	ProxyController.java
POST	/api/auth/register	ProxyController.java
GET	/api/tasks	ProxyController.java
POST	/api/tasks	ProxyController.java
PUT	/api/tasks/{id}	ProxyController.java
DELETE	/api/tasks/{id}	ProxyController.java
PATCH	/api/tasks/{id}/status/{status}	ProxyController.java
POST	/api/ai-ops/analyze	ProxyController.java
POST	/api/ai-ops/autofix	ProxyController.java
GET	/api/pipeline/runs	ProxyController.java
POST	/api/pipeline/runs/{id}/trigger	ProxyController.java
GET	/api/notifications/live	ProxyController.java
PATCH	/api/notifications/{id}/read	ProxyController.java
GET	/api/cost/summary	ProxyController.java
GET	/api/audit/events	ProxyController.java
GET	/api/secrets/list	ProxyController.java
GET	/api/registry/images	ProxyController.java
GET	/api/runtime/docker/containers	RuntimeOpsController.java
GET	/api/runtime/docker/images	RuntimeOpsController.java
GET	/api/runtime/docker/volumes	RuntimeOpsController.java
GET	/api/runtime/docker/stats	RuntimeOpsController.java
GET	/api/runtime/docker/networks	RuntimeOpsController.java
GET	/api/runtime/docker/builds	RuntimeOpsController.java
GET	/api/runtime/helm/releases	RuntimeOpsController.java
GET	/api/runtime/grafana/health	RuntimeOpsController.java
GET	/api/runtime/terraform/validate	RuntimeOpsController.java
GET	/api/runtime/terraform/plan	RuntimeOpsController.java

21.x gitops-control-service

Metodo	Path	File/Metodo
GET	/api/gitops/{app}	GitOpsControlServiceController.java
POST	/api/gitops/{app}/sync	GitOpsControlServiceController.java
POST	/api/gitops/{app}/rollback/{revision}	GitOpsControlServiceController.java

21.x observability-service

Metodo	Path	File/Metodo
GET	/api/observability/prometheus	ObservabilityServiceController.java
GET	/api/observability/loki	ObservabilityServiceController.java
GET	/api/observability/grafana	ObservabilityServiceController.java

21.x pipeline-engine-service

Metodo	Path	File/Metodo
GET	/api/pipeline/git-status	PipelineEngineController.java
POST	/api/pipeline/argocd/{app}/sync	PipelineEngineController.java

21.x task-service

Metodo	Path	File/Metodo
GET	/api/tasks/{id}	TaskController.java
PUT	/api/tasks/{id}	TaskController.java
DELETE	/api/tasks/{id}	TaskController.java
PATCH	/api/tasks/{id}/status/{status}	TaskController.java

21.x terraform-studio-service

Metodo	Path	File/Metodo
GET	/api/terraform-studio/graph	TerraformStudioServiceController.java
GET	/api/terraform-studio/plan	TerraformStudioServiceController.java
GET	/api/terraform-studio/modules	TerraformStudioServiceController.java

22. Troubleshooting decisionale

Ogni problema e riportato con causa, comando di conferma e fix. Questo formato evita frasi generiche e rende il manuale usabile durante una demo reale.

22.1 Keycloak login HTTP 500

Voce	Dettaglio
Causa piu probabile	Tema custom FreeMarker non compatibile o template.ftl rimasto nel tema nebulaops/login.
Comando di conferma	docker compose -p logs keycloak --tail=180 | grep -iE "freemarker|template|ERROR"
Fix operativo	Usare ./scripts/wsl/start.sh: rimuove template.ftl e scrive login.ftl standalone. Poi riavvia keycloak e riprova auth URL.

```
docker compose -p logs keycloak --tail=180 | grep -iE
  "freemarker|template|ERROR"
```

22.2 Frontend carica ma tutte le API falliscono

Voce	Dettaglio
Causa piu probabile	Gateway non raggiungibile, nginx proxy /api rotto o gateway non healthy.
Comando di conferma	curl -sv http://localhost:8080/api/health; ./scripts/wsl/gateway-logs.sh
Fix operativo	Riavviare gateway con ./scripts/wsl/restart-gateway.sh, verificare frontend/nginx.conf e api.config.ts.

```
curl -sv http://localhost:8080/api/health; ./scripts/wsl/gateway-logs.sh
```

22.3 Docker Desktop vuoto

Voce	Dettaglio
Causa piu probabile	Docker socket non montato o permessi insufficienti dentro gateway.
Comando di conferma	docker compose -p exec gateway-service sh -lc "ls -l /var/run/docker.sock; command -v docker"
Fix operativo	Montare /var/run/docker.sock e preparare runtime tools. Su Docker Desktop verificare integrazione WSL.

```
docker compose -p exec gateway-service sh -lc "ls -l /var/run/docker.sock;
  command -v docker"
```

22.4 OpenLens non mostra cluster reale

Voce	Dettaglio
Causa piu probabile	Kubeconfig usa localhost non raggiungibile dal container oppure kubectl manca.
Comando di conferma	./scripts/wsl/prepare-kubeconfig-for-docker.sh; ./scripts/wsl/diagnose-runtime.sh
Fix operativo	Rigenerare .kube/config con IP WSL, copiare kubectl in .runtime-tools e ricreare gateway.

```
./scripts/wsl/prepare-kubeconfig-for-docker.sh; ./scripts/wsl/diagnose-runtime.sh
```

22.5 ImagePullBackOff demo

Voce	Dettaglio
Causa piu probabile	Kind non vede immagine locale o imagePullPolicy errata.
Comando di conferma	kubectl -n nebulaops-demo describe pod <pod> | sed -n "/Events/,$p"
Fix operativo	kind load docker-image nebulaops-spring-demo: --name oppure push su registry e aggiornare values.

```
kubectl -n nebulaops-demo describe pod <pod> | sed -n "/Events/,$p"
```

22.6 Grafana provisioning failed

Voce	Dettaglio
Causa piu probabile	Piu datasource con isDefault: true o file provisioning non valido.
Comando di conferma	grep -R "isDefault: true" infrastructure/observability/grafana/provisioning/datasources
Fix operativo	Lasciare un solo default datasource. start.sh blocca l avvio se trova conteggio diverso da 1.

```
grep -R "isDefault: true" infrastructure/observability/grafana/provisioning/datasources
```

22.7 validate-package.py fallisce

Voce	Dettaglio
Causa piu probabile	Gap documentali nel README, non necessariamente runtime.
Comando di conferma	python3 scripts/validate-package.py
Fix operativo	Aggiungere termini GitLab/Argo CD e riferimenti ai diagrammi runtime/service-port nel README.

```
python3 scripts/validate-package.py
```

22.8 Terraform plan-local non trova tfvars

Voce	Dettaglio
Causa piu probabile	Script entra in terraform/ ma tfvars reale e sotto infrastructure/terraform/examples/local-kind.
Comando di conferma	find . -path "*terraform.tfvars" -print
Fix operativo	Correggere path o eseguire Terraform da infrastructure/terraform.

```
find . -path "*terraform.tfvars" -print
```

22.9 RabbitMQ eventi non arrivano

Voce	Dettaglio
Causa piu probabile	Exchange/routing key o connessione RabbitMQ non pronta.
Comando di conferma	curl -u guest:guest http://localhost:15672/api/overview; logs task-service notification-service
Fix operativo	Verificare RabbitMqConfig, health RabbitMQ e retry startup dei servizi.

```
curl -u guest:guest http://localhost:15672/api/overview; logs task-service notification-service
```

22.10 Prometheus target down

Voce	Dettaglio
Causa piu probabile	Actuator non esposto, service name errato o scrape config non aggiornato.
Comando di conferma	curl http://localhost:9090/api/v1/targets | jq
Fix operativo	Controllare prometheus.yml e endpoint /actuator/prometheus del servizio.

```
curl http://localhost:9090/api/v1/targets | jq
```

23. Produzione, security hardening e gap tecnici

e ottimo come piattaforma portfolio/lab. Per essere presentato come enterprise production-ready bisogna dichiarare i gap e definire remediation. Questa sezione trasforma i gap in backlog tecnico prioritizzato.

Gap	Evidenza file	Rischio	Fix consigliato
Versioning misto	config/platform.yml 21.3; backend POM 21.3.0; UI labels v21.3; Linux scripts v17	Confusione in demo, image names errati	Normalizzare versioni in platform.yml, POM parent, UI labels, README e scripts
Auth doppia	Keycloak OIDC + auth-service dev JWT	Identita non unificata	Validare token Keycloak nel gateway o federare auth-service
docker.sock montato	Docker actions reali	Privilegio host elevato	Socket proxy, allow-list azioni, audit e role check
kubectl shell commands	KubernetesOpsController usa command shell	Rischio injection/over-permission	Usare Kubernetes Java client o RBAC + validazione strict + namespace allow-list
Script Terraform path	scripts/terraform usa tfvars non coerente	Plan/apply falliscono	Spostare script sotto infrastructure/terraform o duplicare examples
README validation	validate-package.py fallisce su termini/diagrammi	SEO e qualità pacchetto	Aggiornare README con diagrammi e GitLab/ArgoCD
Audit foundation	ProxyController /audit/events foundation	Manca tracciamento reale	Audit service con Mongo append-only o Loki stream firmato
Service Mesh foundation	UI visuale senza Istio/Linkerd	Feature overclaim	Etichettare come roadmap o integrare Istio read-only

23.1 Checklist pre-demo

• Eseguire ./scripts/wsl/health.sh e salvare output.

• Aprire frontend, Keycloak login, Grafana, RabbitMQ e Mongo Express.

• Avviare demo Docker e verificare visibilita nel modulo Containers.

• Deployare demo Kubernetes e verificare namespace nebulaops-demo nel modulo KUBERNETES.

• Preparare un errore controllato ImagePullBackOff e mostrarne diagnosi in OpenLens-like.

• Mostrare .gitlab-ci.yml demo, ArgoCD application e Helm values come percorso CI/CD realistico.

• Distinguere le funzioni foundation, come service mesh o audit completo, dalle integrazioni operative gia implementate.

24. Appendici operative

24.1 Comandi completi di controllo piattaforma

```
# Stato generale
docker compose -p                ps
./scripts/wsl/health.sh
./scripts/wsl/diagnose-runtime.sh

# Gateway
curl -s http://localhost:8080/api/health | jq
./scripts/wsl/logs.sh gateway-service --lines 200

# Docker API attraverso gateway
curl -s http://localhost:8080/api/runtime/docker/containers | jq
curl -s http://localhost:8080/api/runtime/docker/images | jq
curl -s http://localhost:8080/api/runtime/docker/volumes | jq

# Kubernetes API attraverso gateway
curl -s http://localhost:8080/api/kubernetes/snapshot | jq
curl -s http://localhost:8080/api/kubernetes/nodes | jq
curl -s 'http://localhost:8080/api/kubernetes/resources?kind=pods&namespace=nebulaops-demo' | jq

# Observability
curl -s http://localhost:9090/-/healthy
curl -s 'http://localhost:9090/api/v1/query?query=up' | jq
curl -s http://localhost:3100/loki/api/v1/status/buildinfo | jq

# Demo app
curl -s http://localhost:18080/api/demo/hello | jq
curl -s http://localhost:18080/actuator/prometheus | grep nebulaops_demo_requests_total
```

24.2 Endpoint count summary

Servizio	Endpoint rilevati
ai-ops-service	10
auth-service	6
cost-analytics-service	3
devsecops-service	2
environment-manager-service	2
gateway-service	96
gitops-control-service	3
observability-service	3
pipeline-engine-service	2
task-service	4
terraform-studio-service	3

24.3 Inventario documentazione e diagrammi

Path	Nota
docs/diagrams/architecture-animated.svg	diagramma SVG
docs/diagrams/docker-kubernetes-runtime-3d.svg	diagramma SVG
docs/diagrams/frontend-operations-dashboard.svg	diagramma SVG
docs/diagrams/gitlab-argocd-flow.svg	diagramma SVG
docs/diagrams/kubernetes-helm-view.svg	diagramma SVG
docs/diagrams/messaging-cache-flow.svg	diagramma SVG
docs/diagrams/nebulaops-v14-advanced-architecture.svg	diagramma SVG
docs/diagrams/nebulaops-v16-spatial-architecture.svg	diagramma SVG
docs/diagrams/nebulaops-v18-terraform-control-plane.svg	diagramma SVG
docs/diagrams/nebulaops-v19-1-ai-ops-architecture.svg	diagramma SVG
docs/diagrams/nebulaops-v19-2-ai-ops-architecture.svg	diagramma SVG
docs/diagrams/nebulaops-v19-2-kubernetes-visual-cluster.svg	diagramma SVG
docs/diagrams/nebulaops-v19-3-ai-ops-architecture.svg	diagramma SVG
docs/diagrams/nebulaops-v19-3-devsecops-module.svg	diagramma SVG
docs/diagrams/nebulaops-v19-3-home-feature-launcher.svg	diagramma SVG
docs/diagrams/nebulaops-v19-3-kubernetes-visual-cluster.svg	diagramma SVG
docs/diagrams/nebulaops-v19-4-infra-hub.svg	diagramma SVG
docs/diagrams/nebulaops-v19-5-gitops-control-plane.svg	diagramma SVG
docs/diagrams/nebulaops-v19-5-infra-hub.svg	diagramma SVG
docs/diagrams/nebulaops-v19-5-multi-environment-manager.svg	diagramma SVG
docs/diagrams/nebulaops-v19-5-observability-stack.svg	diagramma SVG
docs/diagrams/nebulaops-v19-5-smart-terraform-studio.svg	diagramma SVG
docs/diagrams/nebulaops-v20-2-ai-ops-architecture.svg	diagramma SVG
docs/diagrams/nebulaops-v20-2-cicd-pipeline-designer.svg	diagramma SVG
docs/diagrams/nebulaops-v20-2-devsecops-module.svg	diagramma SVG
docs/diagrams/nebulaops-v20-2-full-platform-overview.svg	diagramma SVG
docs/diagrams/nebulaops-v20-2-gitops-control-plane.svg	diagramma SVG
docs/diagrams/nebulaops-v20-2-infra-hub.svg	diagramma SVG
docs/diagrams/nebulaops-v20-2-kubernetes-visual-cluster.svg	diagramma SVG
docs/diagrams/nebulaops-v20-2-multi-environment-manager.svg	diagramma SVG
docs/diagrams/nebulaops-v20-2-observability-stack.svg	diagramma SVG
docs/diagrams/nebulaops-v20-2-smart-terraform-studio.svg	diagramma SVG
docs/diagrams/nebulaops-v20-6-corrected-real-services.svg	diagramma SVG
docs/diagrams/nebulaops-v20-6-live-toolchain-flow.svg	diagramma SVG
docs/diagrams/observability-grafana-flow.svg	diagramma SVG
docs/diagrams/openlens-like-console-animated.svg	diagramma SVG
docs/diagrams/request-flow-sequence.svg	diagramma SVG
docs/diagrams/runtime-architecture.svg	diagramma SVG
docs/diagrams/service-port-map.svg	diagramma SVG
docs/diagrams/v17-runtime-control-3d.svg	diagramma SVG
docs/diagrams/v17-saas-flow-3d.svg	diagramma SVG

Appendice sorgente - docker-compose.yml

Estratto incluso per rendere il manuale autosufficiente durante installazione e troubleshooting. Verificare sempre il file nel repository prima di applicare modifiche.

```
name:

services:
  keycloak-db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: keycloak
      POSTGRES_USER: keycloak
      POSTGRES_PASSWORD: keycloak
    volumes:
      - keycloak-db-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U keycloak"]
      interval: 10s
      timeout: 5s
      retries: 10

  keycloak:
    image: quay.io/keycloak/keycloak:24.0.4
    command: start-dev --import-realm
    environment:
      KC_DB: postgres
      KC_DB_URL: jdbc:postgresql://keycloak-db:5432/keycloak
      KC_DB_USERNAME: keycloak
      KC_DB_PASSWORD: keycloak
      KC_HOSTNAME_URL: http://localhost:8180
      KC_HOSTNAME_ADMIN_URL: http://localhost:8180
      KC_HOSTNAME_STRICT: "false"
      KC_HOSTNAME_STRICT_HTTPS: "false"
      KC_HTTP_ENABLED: "true"
      KEYCLOAK_ADMIN: admin
      KEYCLOAK_ADMIN_PASSWORD: admin
      KC_HEALTH_ENABLED: "true"
      KC_SPI_THEME_CACHE_THEMES: "false"
      KC_SPI_THEME_CACHE_TEMPLATES: "false"
      KC_SPI_THEME_STATIC_MAX_AGE: "-1"
    volumes:
      - ./infrastructure/keycloak/realm-nebulaops.json:/opt/keycloak/data/import/realm-
        nebulaops.json:ro
      - ./infrastructure/keycloak/themes/nebulaops:/opt/keycloak/themes/nebulaops:ro
    depends_on:
      keycloak-db:
        condition: service_healthy
    ports:
      - "8180:8080"
    healthcheck:
      test: ["CMD-SHELL", "exec 3<>/dev/tcp/localhost/8080 && echo -e 'GET /health/ready
        HTTP/1.0\r\n\r\n' >&3 && grep -q '\"status\":\"UP\"' <&3 || exit 1"]
      interval: 20s
      timeout: 15s
      retries: 15
      start_period: 60s


  mongodb:
    image: mongo:7
    healthcheck:
      test:
        - CMD
        - mongosh
        - --eval
        - db.adminCommand('ping')
      interval: 10s
      timeout: 5s
      retries: 10
    environment:
      MONGO_INITDB_ROOT_USERNAME: nebula
      MONGO_INITDB_ROOT_PASSWORD: nebula
    ports:
      - 27017:27017
    volumes:
      - mongo-data:/data/db
  mongo-express:
    image: mongo-express:1.0.2-20
    environment:
      ME_CONFIG_MONGODB_ADMINUSERNAME: nebula
      ME_CONFIG_MONGODB_ADMINPASSWORD: nebula
      ME_CONFIG_MONGODB_URL: mongodb://nebula:nebula@mongodb:27017/?authSource=admin
      ME_CONFIG_BASICAUTH_USERNAME: admin
      ME_CONFIG_BASICAUTH_PASSWORD: admin
    depends_on:
```

```
    mongodb:
      condition: service_healthy
  ports:
    - 8088:8081
redis-commander:
  image: rediscommander/redis-commander:latest
  environment:
    REDIS_HOSTS: local:redis:6379
    HTTP_USER: admin
    HTTP_PASSWORD: admin
  depends_on:
    redis:
      condition: service_healthy
  ports:
    - 8089:8081
auth-service:
  image:                -auth-service:latest
  build:
    context: .
    dockerfile: backend/auth-service/Dockerfile
  environment:
    MONGO_URI: mongodb://nebula:nebula@mongodb:27017/nebula_auth?authSource=admin
  depends_on:
    mongodb:
      condition: service_healthy
  ports:
    - 8081:8081
task-service:
  image:                -task-service:latest
  build:
    context: .
    dockerfile: backend/task-service/Dockerfile
  environment:
    MONGO_URI: mongodb://nebula:nebula@mongodb:27017/nebula_task?authSource=admin
    RABBITMQ_HOST: rabbitmq
    RABBITMQ_PORT: '5672'
    RABBITMQ_USER: guest
    RABBITMQ_PASSWORD: guest
  depends_on:
    mongodb:
      condition: service_healthy
    rabbitmq:
      condition: service_healthy
  ports:
    - 8082:8082
notification-service:
  image:                -notification-service:latest
  build:
    context: .
    dockerfile: backend/notification-service/Dockerfile
  environment:
```

Appendice sorgente - frontend/nginx.conf

Estratto incluso per rendere il manuale autosufficiente durante installazione e troubleshooting. Verificare sempre il file nel repository prima di applicare modifiche.

```
server {
  listen 80;
  server_name _;
  root /usr/share/nginx/html;
  index index.html;

  location / {
    try_files $uri $uri/ /index.html;
  }

  location /api/ {
    proxy_pass http://gateway-service:8080/api/;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }
}
```

Appendice sorgente - backend/gateway-service/src/main/resources/application.yml

Estratto incluso per rendere il manuale autosufficiente durante installazione e troubleshooting. Verificare sempre il file nel repository prima di applicare modifiche.

```
# v21.3 — Gateway service config.
# Service URLs centralized: defaults match docker-compose hostnames,
# can be overridden via environment variables (see docker-compose.yml).
server:
  port: 8080

spring:
  application:
    name: gateway-service

# Proxy targets — keep in sync with /config/platform.yml
proxy:
  auth:         ${AUTH_SERVICE_URL:http://auth-service:8081}
  task:         ${TASK_SERVICE_URL:http://task-service:8082}
  notification: ${NOTIFICATION_SERVICE_URL:http://notification-service:8083}
  file:         ${FILE_SERVICE_URL:http://file-service:8084}
  ai-ops:       ${AI_OPS_SERVICE_URL:http://ai-ops-service:8085}
  devsecops:    ${DEVSECOPS_SERVICE_URL:http://devsecops-service:8086}
  pipeline:     ${PIPELINE_ENGINE_SERVICE_URL:http://pipeline-engine-service:8087}
  observability: ${OBSERVABILITY_SERVICE_URL:http://observability-service:8092}
  gitops:       ${GITOPS_CONTROL_SERVICE_URL:http://gitops-control-service:8093}
  environment:  ${ENVIRONMENT_MANAGER_SERVICE_URL:http://environment-manager-service:8094}
  terraform:    ${TERRAFORM_STUDIO_SERVICE_URL:http://terraform-studio-service:8096}
  # v21.3 NEW
  cost:         ${COST_ANALYTICS_SERVICE_URL:http://cost-analytics-service:8097}
  notify-ws:    ${NOTIFICATION_WS_SERVICE_URL:http://notification-ws-service:8098}

management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  metrics:
    tags:
      application: gateway-service
```

Appendice sorgente - backend/pom.xml

Estratto incluso per rendere il manuale autosufficiente durante installazione e troubleshooting. Verificare sempre il file nel repository prima di applicare modifiche.

```
<project xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns="http://maven.apache.org/POM/4.0.0"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
           https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.3.5</version>
        <relativePath/>
    </parent>

    <groupId>dev.nebulaops</groupId>
    <artifactId>nebulaops-backend</artifactId>
    <version>21.3.0</version>
    <packaging>pom</packaging>

    <modules>
        <module>auth-service</module>
        <module>task-service</module>
        <module>file-service</module>
        <module>notification-service</module>
        <module>gateway-service</module>
        <module>ai-ops-service</module>
        <module>devsecops-service</module>
        <module>pipeline-engine-service</module>
        <module>terraform-studio-service</module>
        <module>environment-manager-service</module>
        <module>gitops-control-service</module>
        <module>observability-service</module>
    </modules>

    <properties>
        <java.version>21</java.version>
        <spring-cloud.version>2023.0.3</spring-cloud.version>
        <springdoc.version>2.6.0</springdoc.version>
    </properties>

    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>org.springframework.cloud</groupId>
                <artifactId>spring-cloud-dependencies</artifactId>
                <version>${spring-cloud.version}</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
        </dependencies>
    </dependencyManagement>

    <build>
        <pluginManagement>
            <plugins>
                <plugin>
                    <groupId>org.springframework.boot</groupId>
                    <artifactId>spring-boot-maven-plugin</artifactId>
                    <executions>
                        <execution>
                            <goals>
                                <goal>repackage</goal>
                            </goals>
                        </execution>
                    </executions>
                </plugin>
            </plugins>
        </pluginManagement>
    </build>
</project>
```

Appendice sorgente - frontend/package.json

Estratto incluso per rendere il manuale autosufficiente durante installazione e troubleshooting. Verificare sempre il file nel repository prima di applicare modifiche.

```
{
  "name": "nebulaops-angular-frontend",
  "version": "21.2.1",
  "private": true,
  "scripts": {
    "start": "ng serve --host 0.0.0.0 --port 4200",
    "build": "ng build --configuration production",
    "test": "ng test --watch=false",
    "lint": "ng lint",
    "typecheck": "ng build --configuration production --aot"
  },
  "dependencies": {
    "@angular/animations": "^18.2.0",
    "@angular/common": "^18.2.0",
    "@angular/compiler": "^18.2.0",
    "@angular/core": "^18.2.0",
    "@angular/forms": "^18.2.0",
    "@angular/platform-browser": "^18.2.0",
    "@angular/platform-browser-dynamic": "^18.2.0",
    "@angular/router": "^18.2.0",
    "@angular/cdk": "^18.2.0",
    "rxjs": "~7.8.1",
    "tslib": "^2.6.3",
    "zone.js": "~0.14.10"
  },
  "devDependencies": {
    "@angular-devkit/build-angular": "^18.2.0",
    "@angular/cli": "^18.2.0",
    "@angular/compiler-cli": "^18.2.0",
    "typescript": "~5.5.4"
  }
}
```

Appendice sorgente - infrastructure/kubernetes/deployments.yaml

Estratto incluso per rendere il manuale autosufficiente durante installazione e troubleshooting. Verificare sempre il file nel repository prima di applicare modifiche.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: gateway-service
  namespace: nebulaops
spec:
  replicas: 2
  selector: { matchLabels: { app: gateway-service } }
  template:
    metadata: { labels: { app: gateway-service } }
    spec:
      containers:
        - name: gateway-service
          image: nebulaops/gateway-service:latest
          ports: [ { containerPort: 8080 } ]
          envFrom:
            - configMapRef: { name: nebulaops-config }
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: auth-service
  namespace: nebulaops
spec:
  replicas: 2
  selector: { matchLabels: { app: auth-service } }
  template:
    metadata: { labels: { app: auth-service } }
    spec:
      containers:
        - name: auth-service
          image: nebulaops/auth-service:latest
          ports: [ { containerPort: 8081 } ]
          envFrom:
            - secretRef: { name: nebulaops-secrets }
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: task-service
  namespace: nebulaops
spec:
  replicas: 2
  selector: { matchLabels: { app: task-service } }
  template:
    metadata: { labels: { app: task-service } }
    spec:
      containers:
        - name: task-service
          image: nebulaops/task-service:latest
          ports: [ { containerPort: 8082 } ]
          envFrom:
            - configMapRef: { name: nebulaops-config }
            - secretRef: { name: nebulaops-secrets }
```

Appendice sorgente - infrastructure/kubernetes/services.yaml

Estratto incluso per rendere il manuale autosufficiente durante installazione e troubleshooting. Verificare sempre il file nel repository prima di applicare modifiche.

```
apiVersion: v1
kind: Service
metadata: { name: gateway-service, namespace: nebulaops }
spec:
  selector: { app: gateway-service }
  ports: [ { port: 8080, targetPort: 8080 } ]
---
apiVersion: v1
kind: Service
metadata: { name: auth-service, namespace: nebulaops }
spec:
  selector: { app: auth-service }
  ports: [ { port: 8081, targetPort: 8081 } ]
---
apiVersion: v1
kind: Service
metadata: { name: task-service, namespace: nebulaops }
spec:
  selector: { app: task-service }
  ports: [ { port: 8082, targetPort: 8082 } ]
```

Appendice sorgente - infrastructure/kubernetes/ingress.yaml

Estratto incluso per rendere il manuale autosufficiente durante installazione e troubleshooting. Verificare sempre il file nel repository prima di applicare modifiche.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: nebulaops-ingress
  namespace: nebulaops
spec:
  rules:
    - host: nebulaops.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: gateway-service
                port:
                  number: 8080
```

Appendice sorgente - infrastructure/keycloak/realm-nebulaops.json

Estratto incluso per rendere il manuale autosufficiente durante installazione e troubleshooting. Verificare sempre il file nel repository prima di applicare modifiche.

```
{
  "realm": "nebulaops",
  "displayName": "NebulaOps Platform",
  "enabled": true,
  "loginTheme": "nebulaops",
  "accountTheme": "keycloak",
  "adminTheme": "keycloak",
  "emailTheme": "keycloak",
  "sslRequired": "none",
  "registrationAllowed": false,
  "loginWithEmailAllowed": true,
  "duplicateEmailsAllowed": false,
  "resetPasswordAllowed": true,
  "editUsernameAllowed": false,
  "bruteForceProtected": true,
  "accessTokenLifespan": 3600,
  "refreshTokenMaxReuse": 0,
  "clients": [
    {
      "clientId": "nebulaops-frontend",
      "name": "NebulaOps Frontend",
      "enabled": true,
      "publicClient": true,
      "standardFlowEnabled": true,
      "directAccessGrantsEnabled": true,
      "redirectUris": [
        "http://localhost:4200/*",
        "http://localhost:4201/*"
      ],
      "webOrigins": [
        "http://localhost:4200",
        "http://localhost:4201"
      ],
      "attributes": {
        "pkce.code.challenge.method": "S256"
      }
    },
    {
      "clientId": "nebulaops-gateway",
      "name": "NebulaOps Gateway",
      "enabled": true,
      "publicClient": false,
      "secret": "nebulaops-secret-dev-only",
      "standardFlowEnabled": false,
      "serviceAccountsEnabled": true,
      "directAccessGrantsEnabled": true,
      "authorizationServicesEnabled": true
    }
  ],
  "roles": {
    "realm": [
      {
        "name": "nebula-admin",
        "description": "NebulaOps platform administrator with full access"
      },
      {
        "name": "nebula-operator",
        "description": "NebulaOps operator with read/write access to operations"
      },
      {
        "name": "nebula-viewer",
        "description": "NebulaOps read-only viewer"
      }
    ]
  },
  "users": [
    {
      "username": "admin",
      "email": "admin@nebulaops.local",
      "firstName": "Admin",
      "lastName": "NebulaOps",
      "enabled": true,
      "emailVerified": true,
      "credentials": [
        {
          "type": "password",
          "value": "admin",
          "temporary": false
        }
      ],
      "realmRoles": ["nebula-admin", "default-roles-nebulaops"]
```

```
    },
    {
      "username": "peyman",
      "email": "peyman@nebulaops.local",
      "firstName": "Peyman",
      "lastName": "Operator",
      "enabled": true,
      "emailVerified": true,
      "credentials": [
        {
          "type": "password",
          "value": "admin",
          "temporary": false
        }
      ],
      "realmRoles": ["nebula-operator", "default-roles-nebulaops"]
    }
  ],
  "defaultRoles": ["default-roles-nebulaops"],
  "requiredCredentials": ["password"]
}
```

Conclusione

Il manuale collega a file, endpoint, script, limiti e procedure demo verificabili nel repository. Le integrazioni incomplete sono classificate come foundation o backlog tecnico, mentre le funzionalita operative sono descritte con comandi e flussi di validazione.

La documentazione deve restare allineata a api.config.ts, controller gateway, docker-compose.yml, script WSL, Helm chart e demo app. Ogni modifica in questi componenti richiede aggiornamento della sezione tecnica corrispondente.